APPASERVER Data Structure

March 12, 2022

SECURITY

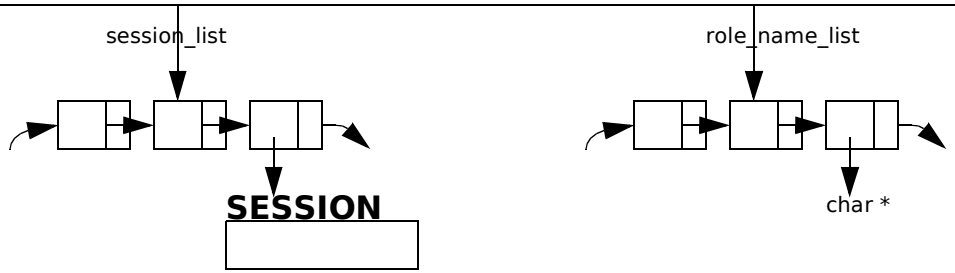
```
enum password_function password_function;{ no_encryption
                                          old_password_function
                                          regular_password_function
                                          sha2_function

enum password_function
    security_database_password_function(
        char *database_password );

char *security_encrypted_password(
    char *password_sql_injection_escape,
    enum password_function password_function );

char *security_mysql_version( void );

enum password_function
    security_mysql_version_password_function(
        security_mysql_version() );

char *security_encryption_select_clause(
    enum password_function password_function,
    char *password );

boolean security_password_match(
    database_password,
    security_encrypted_password(
        security_sql_injection_escape(
            typed_in_password ),
            security_database_password_function() );

boolean security_password_encrypted(
    char *database_password );

char *security_replace_special_characters(
    char *data );

char *security_sql_injection_escape(
    char *data )
{
    char destination[ STRING_WHERE_BUFFER ];

    return strdup(
        string_escape_character_array(
            destination,
            data,
            “‘’$;%=_” ) );
}

boolean security_system_role(
    char *role_name );

/* Public */
LIST *security_sql_injection_escape_list(
    LIST *data_list );
```

SECURITY_ENTITY

```
/* Usage */
SECURITY_ENTITY *security_entity_new(
    char *login_name,
    boolean folder->non_owner_forbid,
    boolean role->override_row_restrictions );

/* Input */
char *login_name;
boolean non_owner_forbid;
boolean override_row_restrictions;

/* Process */
char *login_name_only;
char *full_name_only;
char *street_address_only;

/* Returns null if no ENTITY->login_name */
char *security_login_name_full_name_only(
    &this->street_address_only,
    this->login_name );

/* Returns full_name_only */
char *security_entity_fetch(
    &this->street_address_only,
    this->login_name );

char *security_entity_where(
    SECURITY_ENTITY *security_entity,
    LIST *folder_attribute_list );
```

APPASERVER_USER

```
/* Usage */
APPASERVER_USER *appaserver_user_fetch(
    char *login_name,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

/* Attributes */
char *login_name;
char *full_name;
char *typed_in_password;
char *database_password;
char *user_date_format;

/* Process */
APPASERVER_USER *appaserver_user_parse(
    char *input,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

/* Public */
LIST *appaserver_user_session_list(
    char *login_name );

LIST *appaserver_user_role_name_list(
    char *login_name );

FILE *appaserver_user_update_open(
    void );

void appaserver_user_update(
    appaserver_user_update_open(),
    security_encrypted_password(
        security_sql_injection_escape(
            database_password ),
        security_mysql_version_password_function(
            security_mysql_version() ) ),
    login_name );

char *appaserver_user_default_role_name(
    char *login_name );
```



APPASERVER_PARAMETER

```
/* Usage */
APPASERVER_PARAMETER *
    appaserver_parameter_new(
        void );

APPASERVER_PARAMETER *
    appaserver_parameter_application(
        char *application_name );

/* Process */
char *appaserver_parameter_filename(
    char *application_name );

FILE *appaserver_parameter_open_file(
    char *appaserver_parameter_filename() );

DICTIONARY *appaserver_parameter_dictionary(
    FILE *appaserver_parameter_open_file() );

APPASERVER_PARAMETER *
    appaserver_parameter_fetch(
        appaserver_parameter_application(),
        appaserver_parameter_dictionary() );

/* Attributes */
char *filename;
char *mysql_user;
char *password;
char *flags;
char *MYSQL_HOST;
char *MYSQL_TCP_PORT;
char *appaserver_mount_point;
char *apache_cgi_directory;
char *cgi_home;
char *document_root;
char *appaserver_error_directory;
char *appaserver_data_directory;
char *upload_directory;
boolean mysql_password_syntax;

/* Public */
char *appaserver_parameter_mount_point(
    void );

char *appaserver_parameter_cgi_directory(
    void );

char *appaserver_parameter_error_directory(
    void );

char *appaserver_parameter_data_directory(
    void );

char *appaserver_parameter_cgi_home(
    void );

char *appaserver_parameter_document_root(
    void );

char *appaserver_parameter_upload_directory(
    void );
```

POST_DICTIONARY

```
/* Usage */
POST_DICTIONARY *post_dictionary_stdin_new(
    /* Used when expecting a spooled file */
    char *appaserver_parameter_upload_directory(),
    char *session_key );

POST_DICTIONARY *post_dictionary_string_new(
    char *dictionary_string );

/* Process */
char *post_dictionary_apache_key(
    string_input() );

DICTIONARY *original_post_dictionary =
    post_dictionary_fetch(
        FILE *stdin,
        char *upload_directory,
        char *session_key,
        post_dictionary_apache_key() );

char *post_dictionary_apache_label(
    char *input );

char *post_dictionary_attribute_name(
    char *input );

char *post_dictionary_input_filename(
    char *input );

void post_dictionary_stream_set(
    DICTIONARY *original_post_dictionary /* out */,
    FILE *stdin,
    post_dictionary_attribute_name(),
    post_dictionary_apache_key() );

char *post_dictionary_store_filename(
    char *post_dictionary_input_filename(),
    char *session_key );

char *post_dictionary_spool_filename(
    post_dictionary_store_filename(),
    upload_directory );

void post_dictionary_spool_file(
    post_dictionary_spool_filename(),
    FILE *stdin,
    char *apache_key );

dictionary_set(
    original_post_dictionary,
    strdup( post_dictionary_attribute_name() ),
    post_dictionary_store_filename() );
```

FRAMESET (1)

```
/* Usage */
FRAMESET *frameset_new(
    char *applicatation_name,
    char *session_key,
    char *login_name,
    boolean frameset_menu_horizontal(
        application_name,
        login_name ) );

/* Process */
APPASERVER_PARAMETER *
    appaserver_parameter =
        appaserver_parameter_new(
            application_name );

if ( !frameset_menu_horizontal )
{
    FRAMESET_FRAME *frameset_frame_menu =
        frameset_frame_new(
            application_name,
            appaserver_parameter->
                document_root,
            session_key,
            FRAMESET_MENU_FRAME );
}

FRAMESET_FRAME *frameset_frame_prompt =
    frameset_frame_new(
        application_name,
        appaserver_parameter->
            document_root_directory,
        session_key,
        FRAMESET_PROMPT_FRAME );

FRAMESET_FRAME *frameset_frame_edit =
    frameset_frame_new(
        application_name,
        appaserver_parameter->
            document_root_directory,
        session_key,
        FRAMESET_EDIT_FRAME );

char *frameset_html(
    frameset_frame_menu->html,
    frameset_frame_prompt->html,
    frameset_frame_edit->html,
    boolean frameset_menu_horizontal );

char *frameset_blank_system_string(
    char *output_filename );

char *blank_edit_frame_system_string =
    frameset_blank_system_string(
        frameset_frame_edit->output_filename );

if ( !frameset_menu_horizontal )
{
    char *blank_prompt_frame_system_string =
        frameset_blank_system_string(
            frameset_frame_prompt->
                output_filename );
}
```

FRAMESET_FRAME

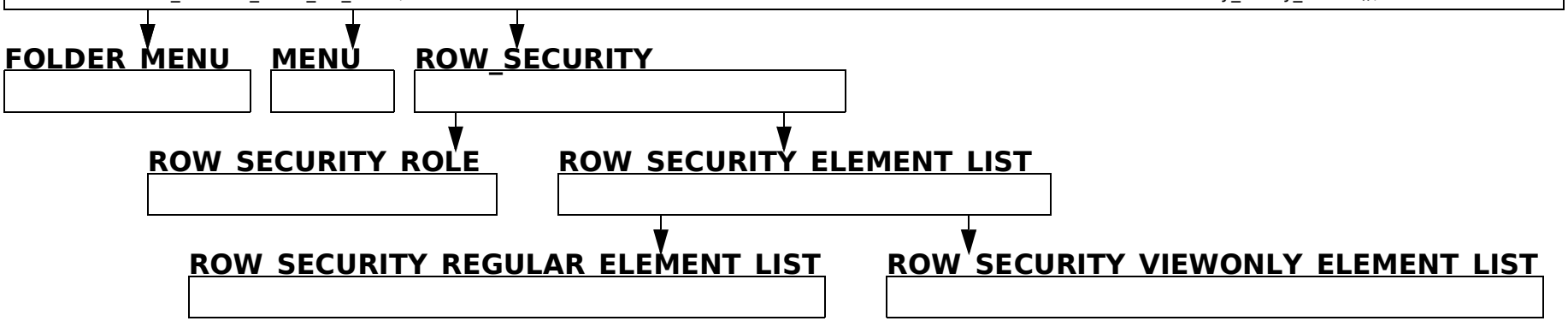
```
/* Usage */
FRAMESET_FRAME *frameset_frame_new(
    char *application_name,
    char *document_root_directory,
    char *session_key,
    char *frame_name );

/* Process */
APPASERVER_LINK *appaserver_file =
    appaserver_file_new();

char *prompt_filename =
    appaserver_link_prompt_filename();

char *output_filename =
    appaserver_link_output_filename();

char *frameset_frame_html(
    char *frame_name,
    char *prompt_filename );
```

FRAMESET (2)

```
/* Process */
char *frameset_role_name(
        LIST *role_name_list(
            role_list_fetch(
                login_name ) ),
        char *appaserver_user_default_role_name(
            login_name ) );

char *frameset_output_filename(
    FRAMESET_FRAME *frameset_frame_menu,
    FRAMESET_FRAME *frameset_frame_prompt );

if ( frameset_role_name() )
{
    char *menu_output_system_string(
        MENU_OUTPUT_EXECUTABLE,
        session_key,
        login_name,
        frameset_role_name(),
        frameset_menu_horizontal(),
        frameset_output_filename() );
}
else
{
    char *choose_role_output_system_string(
        CHOOSE_ROLE_OUTPUT_EXECUTABLE,
        session_key,
        login_name,
        frameset_menu_horizontal(),
        frameset_output_filename() );
}

/* Public */
boolean frameset_menu_horizontal(
    char *application_name,
    char *login_name );

char *frameset_output_system_string(
    char *FRAMESET_OUTPUT_EXECUTABLE,
    char *session_key,
    char *login_name,
    char *post_login_output_pipe_string(),
    char *appaserver_error_filename() );
```

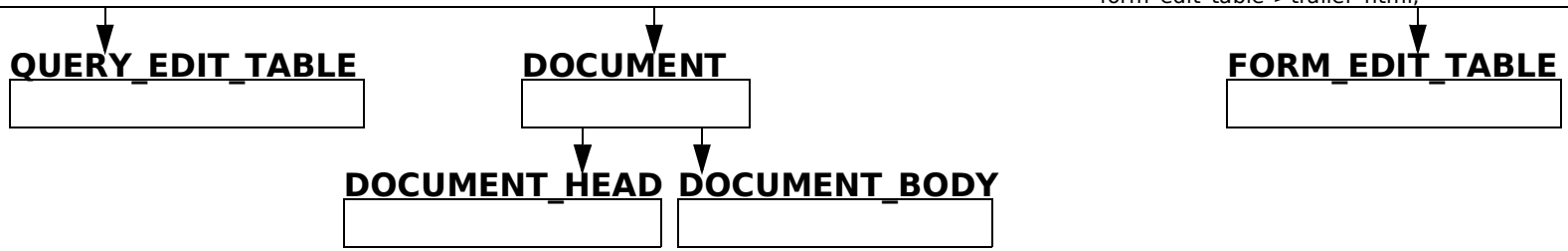
EDIT_TABLE (1)

```
/* Usage */
EDIT_TABLE *edit_table_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    boolean menu_boolean(
        char *current_frame,
        boolean frameset_menu_horizontal(
            application_name,
            login_name ) ),
    DICTIONARY *query_dictionary,
    DICTIONARY *ignore_dictionary,
    DICTIONARY *non_prefixed_dictionary,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *sort_dictionary,
    LIST *dictionary_separate->
        ignore_select_attribute_name_list );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        role_exclude_lookup_attribute_name_list(
            role->attribute_exclude_list ),
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        1 /* fetch_role_operation_list */ );

int folder_attribute_date_name_list_length =
    list_length(
        folder_attribute_date_name_list(
            folder->folder_attribute_append_isa_list ) );

LIST *folder->relation_join_one2m_list =
    relation_join_one2m_list(
        folder->relation_one2m_recursive_list,
        ignore_dictionary );

if ( menu_boolean() )
{
    FOLDER_MENU *folder_menu =
        folder_menu_fetch(
            application_name,
            session_key,
            appaserver_parameter_data_directory(),
            role_name );

    MENU *menu =
        menu_fetch(
            application_name,
            login_name,
            session_key,
            role_name,
            target_frame,
            folder_menu->lookup_count_list );
}

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

char *security_entity_where(
    security_entity,
    folder->folder_attribute_list );

char *edit_table_state(
    LIST *folder->role_folder_list() );

boolean edit_table_primary_keys_non_edit(
    list_length(
        folder->relation_mto1_isa_list ) );

ROW_SECURITY *
    row_security =
        row_security_new(
            folder_name,
            folder->folder_attribute_append_isa_list,
            folder->relation_mto1_non_isa_list,
            folder->relation_join_one2m_list,
            folder->post_change_javascript,
            drillthru_dictionary,
            edit_table_primary_keys_non_edit(),
            folder->role_operation_list,
            ignore_select_attribute_name_list,
            edit_table_state(),
            role_exclude_update_attribute_name_list(),
            role_exclude_lookup_attribute_name_list(),
            folder->non_owner_forbid,
            role->override_row_restrictions,
            login_name,
            security_entity_where();
```



EDIT_TABLE (2)

```
/* Process (continued) */
QUERY_EDIT_TABLE *query_edit_table =
    query_edit_table_new(
        application_name,
        folder_name,
        login_name,
        security_entity_where(),
        folder->relation_join_one2m_list,
        ignore_select_attribute_name_list,
        role_exclude_lookup_attribute_name_list(),
        folder->folder_attribute_append_isa_list,
        folder->relation_mto1_non_isa_list,
        folder->relation_mto1_isa_list,
        query_dictionary,
        sort_dictionary,
        (row_security_role)
            ? row_security_role->folder_name
            : (char *)0,
        (row_security_role)
            ? row_security_role->relation
            : (RELATION *)0,
        (row_security_role)
            ? row_security_role->attribute_not_null
            : (char *)0 );

int dictionary_list_length =
    list_length( query_edit_table->dictionary_list );

int edit_table_row_insert_count(
    char *ROW_INSERTED_COUNT_KEY,
    DICTIONARY *non_prefixed_dictionary );

int edit_table_cell_update_count(
    char *COLUMNS_UPDATED_KEY,
    DICTIONARY *non_prefixed_dictionary );

char *edit_table_cell_update_folder_list_string(
    char *COLUMNS_UPDATED_CHANGED_FOLDER_KEY,
    DICTIONARY *non_prefixed_dictionary );

char *edit_table_results_string(
    char *RESULTS_STRING_KEY,
    DICTIONARY *non_prefixed_dictionary );

char *edit_table_submit_action_string(
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *folder_name,
    char *state,
    char *target_frame,
    (char *)0 /* detail_base_folder_name */ );

LIST *edit_table_heading_name_list(
    LIST *row_security->
        row_security_element_list->
        regular->
        element_list,
    LIST *row_security->
        row_security_element_list->
        viewonly->
        element_list );

char *edit_table_title_html(
    char *folder_name,
    edit_table_state() );

char *edit_table_message_html(
    int edit_table_row_insert_count(),
    int edit_table_cell_update_count(),
    char *edit_table_results_string() );

int folder_attribute_date_name_list_length =
    list_length(
        folder_attribute_date_name_list(
            folder->folder_attribute_append_isa_list ) );

DOCUMENT *document =
    document_new(
        application_name,
        application_title_string(),
        edit_table_title_html(),
        edit_table_message_html(),
        (char *)0 /* subsubtitle_html */,
        (char *)0 /* javascript_replace */,
        menu_boolean(),
        menu,
        document_head_menu_setup_string(
            menu_boolean() ),
        document_head_calendar_setup_string(
            folder_attribute_date_name_list_length ),
        document_head_javascript_include_string(),
        (char *)0 /* input_onload_string */ );

FORM_EDIT_TABLE *form_edit_table =
    form_edit_table_new(
        folder_name,
        folder->post_change_javascript,
        dictionary_list_length,
        edit_table_submit_action_string(),
        folder->role_operation_list,
        edit_table_heading_name_list(),
        target_frame,
        DICTIONARY *query_dictionary,
        DICTIONARY *sort_dictioanry,
        DICTIONARY *drillthru_dictionary,
        DICTIOANRY *ignore_dictionary  );

char *edit_table_html(
    document->html,
    form_edit_table->html );

char *edit_table_trailer_html(
    form_edit_table->trailer_html,
```

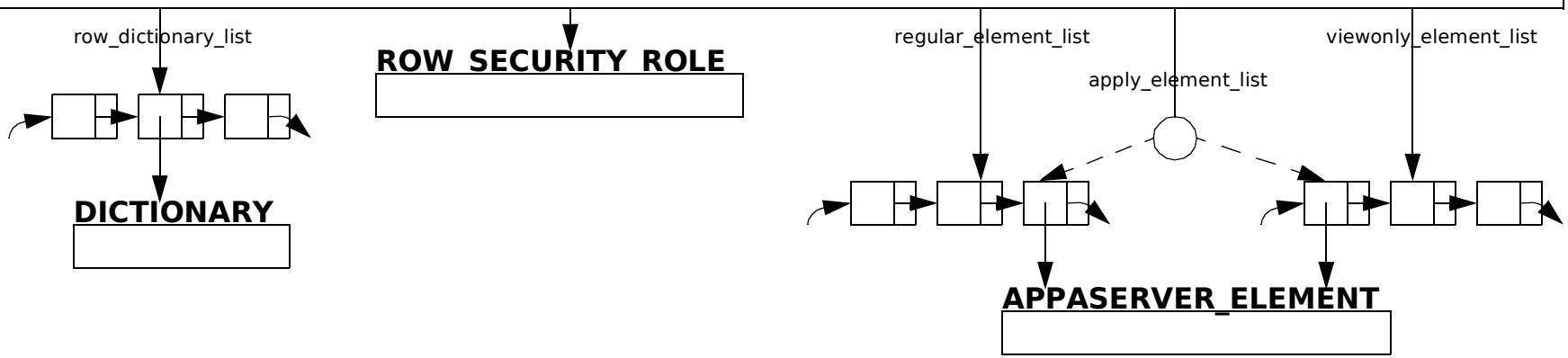


EDIT_TABLE (3)

```
/* Usage */
void edit_table_output(
    FILE *output_stream,
    char *application_name,
    char *edit_table_html(),
    char *form_edit_table_html(),
    LIST *row_dictionary_list,
    LIST *row_security_edit_table->
            row_security_element_list->
            regular_element_list,
    LIST *row_security_edit_table->
            row_security_element_list->
            viewonly_element_list,
    ROW_SECURITY_ROLE *row_security_role,
    char *state,
    char *form_edit_table_trailer_html(),
    char *edit_table_trailer_html() );

/* Process */
fprintf(output_stream,
        "%s\n%s\n",
        edit_table_html(),
        form_edit_table_html() );

edit_table_regular_output(
    output_stream,
    application_name,
    row_dictionary_list,
    regular_element_list,
    viewonly_element_list,
    row_security_role,
    state );

fprintf(output_stream,
        "%s\n",
        element_table_close_html() );

edit_table_hidden_output(
    output_stream,
    row_dictionary_list,
    regular_element_list );

fprintf(output_stream,
        "%s\n%s\n",
        form_edit_table_trailer_html(),
        edit_table_trailer_html() );
```

EDIT_TABLE (4)

```
/* Usage */
void edit_table_regular_output(
    FILE *output_stream,
    char *application_name,
    LIST *row_dictionary_list,
    LIST *row_security_edit_table->
            row_security_element_list->
            row_security_regular_element_list->
            element_list,
    LIST *row_security_edit_table->
            row_security_element_list->
            row_security_viewonly_element_list->
            element_list,
    ROW_SECURITY_ROLE *row_security_role,
    char *state );

/* Process */
for (   row_number = 1;
        row_number <= list_length( row_dictionary_list );
        list_next( row_dictionary_list ), row_number++ )
{
    DICTIONARY *dictionary_row = list_get( row_dictionary_list );

    LIST *edit_table_apply_element_list(
        LIST *regular_element_list,
        LIST *viewonly_element_list,
        DICTIONARY *row_dictionary,
        ROW_SECURITY_ROLE *row_security_role );

    fprintf(  output_stream,
              "%s\n",
              char *edit_table_row_html(
                  LIST *edit_table_apply_element_list(),
                  application_name,
                  char *edit_table_background_color(
                      char *application_name ),
                      char *state,
                      DICTIONARY *row_dictionary,
                      int row_number ) );
}

/* Usage */
void edit_table_hidden_output(
    FILE *output_stream,
    LIST *row_dictionary_list,
    LIST *row_security_edit_table->
            row_security_element_list->
            regular_element_list );

/* Process */
for (   row_number = 1;
        row_number <= list_length( row_dictionary_list );
        list_next( row_dictionary_list ), row_number++ )
{
    DICTIONARY *dictionary_row = list_get( row_dictionary_list );

    fprintf(  output_stream,
              "%s\n",
              char *edit_table_hidden_row_html(
                  DICTIONARY *row_dictionary,
                  LIST *regular_element_list,
                  int row_number ) );
}

/* Private */
char **edit_table_background_color_array(
    int *background_color_array_length,
    environment_application_name() );

char *edit_table_background_color( void );
```



EDIT_TABLE (5)

```
/* Public */
char *edit_table_output_system_string(
    char *EDIT_TABLE_OUTPUT_EXECUTABLE,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    char *dictionary_separate_send_dictionary(),
    char *appaserver_error_filename() );
```

EDIT_TABLE_POST

```
/* Usage */
EDIT_TABLE_POST *edit_table_post_new(
    int argc,
    char **argv,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *state,
    char *target_frame,
    char *detail_base_folder_name );

/* Attributes */
char *target_frame;
char *detail_base_folder_name;

/* Process */
SESSION_FOLDER *session_folder =
    /* Sets appaserver environment and outputs argv */
    /* Each parameter is security inspected */
    session_folder_integrity_exit(
        int argc,
        char **argv,
        char *application_name,
        char *session_key,
        char *login_name,
        char *role_name,
        char *folder_name,
        char *state );

ROLE *role ...
```

INSERT_TABLE

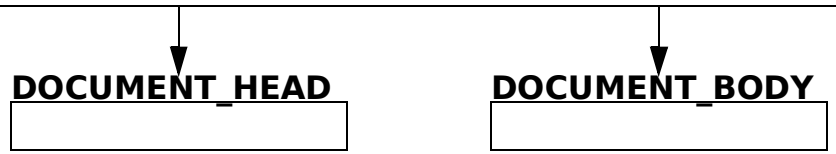
```
/* Usage */
INSERT_TABLE *insert_table_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    boolean menu_boolean(
        char *current_frame,
        boolean frameset_menu_horizontal(
            application_name,
            login_name ) ),
    DICTIONARY *query_dictionary,
    DICTIONARY *ignore_dictionary,
    DICTIONARY *non_prefixed_dictionary,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *sort_dictionary,
    LIST *ignore_select_attribute_name_list );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        role_exclude_lookup_attribute_name_list(
            role->attribute_exclude_list ),
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        1 /* fetch_role_operation_list */ );

if ( menu_boolean() )
{
    FOLDER_MENU *folder_menu =
        folder_menu_fetch(
            application_name,
            session_key,
            appaserver_parameter_data_directory(),
            role_name );

    MENU *menu =
        menu_fetch(
            application_name,
            login_name,
            session_key,
            role_name,
            target_frame,
            folder_menu->lookup_count_list );
}

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );
```

INSERT_TABLE_POST

```
/* Usage */
INSERT_TABLE_POST *insert_table_post_new(
    int argc,
    char **argv,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *folder_name,
    char *state,
    char *target_frame );

/* Attributes */
char *target_frame;

/* Process */
SESSION *session =
    /* Sets appaserver environment and outputs argv */
    /* Each parameter is security inspected */
    session_folder_integrity_exit(
        int argc,
        char **argv,
        char *application_name,
        char *login_name,
        char *session_key,
        char *folder_name,
        char *role_name,
        char *state );

ROLE *role ...
```

DOCUMENT

```
/* Usage */
DOCUMENT *document_new(
    char *application_name,
    char *application_title_string(),
    char *title_html,
    char *subtitle_html,
    char *subsubtitle_html,
    char *javascript_replace(),
    boolean menu_boolean(),
    MENU *menu,
    char *document_head_menu_setup_string(
            menu_boolean() ),
    char *document_head_calendar_setup_string(
            folder_attribute_date_name_list_length ),
    char *document_head_javascript_include_string(),
    char *input_onload_string  );

DOCUMENT *document_quick_new(
    char *application_name,
    char *application_title_string() );

/* Process */
DOCUMENT_HEAD *document_head =
    document_head_new(
        application_name,
        application_title_string(),
        menu_setup_string,
        calendar_setup_string,
        javascript_include_string );

DOCUMENT_BODY *document_body =
    document_body_new(
        menu_boolean,
        menu,
        application_title_string(),
        title_html,
        subtitle_html,
        subsubtitle_html,
        javascript_replace,
        input_onload_string );

char *document_type_html( void );

char *document_standard_html( void );

char *document_html(
    char *document_type_html(),
    char *document_standard_html() );

/* Public */
void document_output_html_stream(
    FILE *output_stream );

char *document_content_type_html( void );

void document_output_content_type( void );

void document_quick_output(
    char *application_name );

char *document_close_html( void );

void document_close( void );

char *document_form_html(
    char *document->html,
    char *document->document_head->html,
    char *document_head_close_html(),
    char *document->document_body->html,
    char *form_html,
    char *document_body_close_html(),
    char *document_close_html() );
```



DOCUMENT_HEAD

```
/* Usage */
DOCUMENT_HEAD *document_head_new(
    char *application_name,
    char *application_title_string,
    document_head_menu_setup_string(),
    document_head_calendar_setup_string(),
    document_head_javascript_include_string() );

/* Process */
char *document_head_meta_string(
    void );

char *document_head_stylesheet_string(
    char *application_name );

char *document_head_title_tag(
    char *application_title_string );

char *document_head_html(
    char *document_head_meta_string,
    char *document_head_stylesheet_string,
    char *document_head_title_tag,
    char *menu_setup_string,
    char *calendar_setup_string,
    char *javascript_include_string );

/* Public */
char *document_head_menu_setup_string(
    boolean menu_boolean() );

char *document_head_calendar_setup_string(
    int list_length(
        folder_attribute_date_name_list() ) );

char *document_head_javascript_include_string(
    void );

char *document_head_close_html( void );
```

DOCUMENT_BODY

```
/* Usage */
DOCUMENT_BODY *document_body_new(
    boolean menu_boolean(),
    MENU *menu,
    char *title_html,
    char *subtitle_html,
    char *subsubtitle_html,
    char *javascript_replace(),
    char *input_onload_string );

/* Process */
char *document_body_menu_onload_string(
    boolean menu_boolean );

char *document_body_tag(
    char *input_onload_string,
    document_body_menu_onload_string(),
    char *javascript_replace );

char *document_body_hide_preload_html(
    boolean menu_boolean );

char *document_body_horizontal_menu_html(
    document_body_hide_preload_html(),
    boolean menu_boolean,
    MENU *menu );

char *document_body_title_html(
    char *application_title_string );

char *document_body_html(
    char *document_body_tag(),
    char *document_body_horizontal_menu_html(),
    char *title_html,
    char *subtitle_html );
    char *subsubtitle_html );

/* Public */
char *document_body_set_onload_control_string(
    char *onload_control_string,
    char *append_onload_control_string );

char *document_body_close_html( void );
```

DOCUMENT_DETAIL

```
DOCUMENT_DETAIL *document_detail_new(
);

/* Process */
DOCUMENT_HEAD *document_head =
    document_head_new(
        application_name,
        title,
        document_head_menu_setup_string(),
        (char *)0 /* calendar_setup_string */
        document_head_javascript_include_string() );

DOCUMENT_BODY_DETAIL *
    document_body_detail =
        document_body_detail_new(
);
```

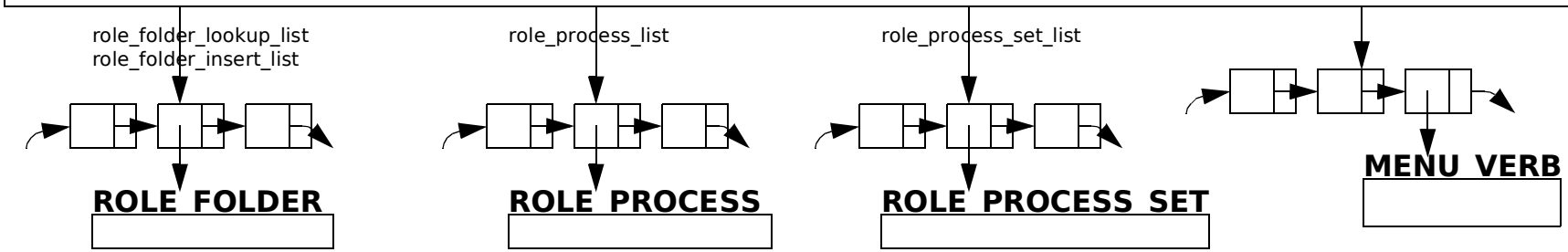
DOCUMENT_HEAD

DOCUMENT_BODY_DETAIL

```
/* Usage */
DOCUMENT_BODY_DETAIL *
    document_body_detail =
        document_body_detail_new(
 );

/* Process */
```

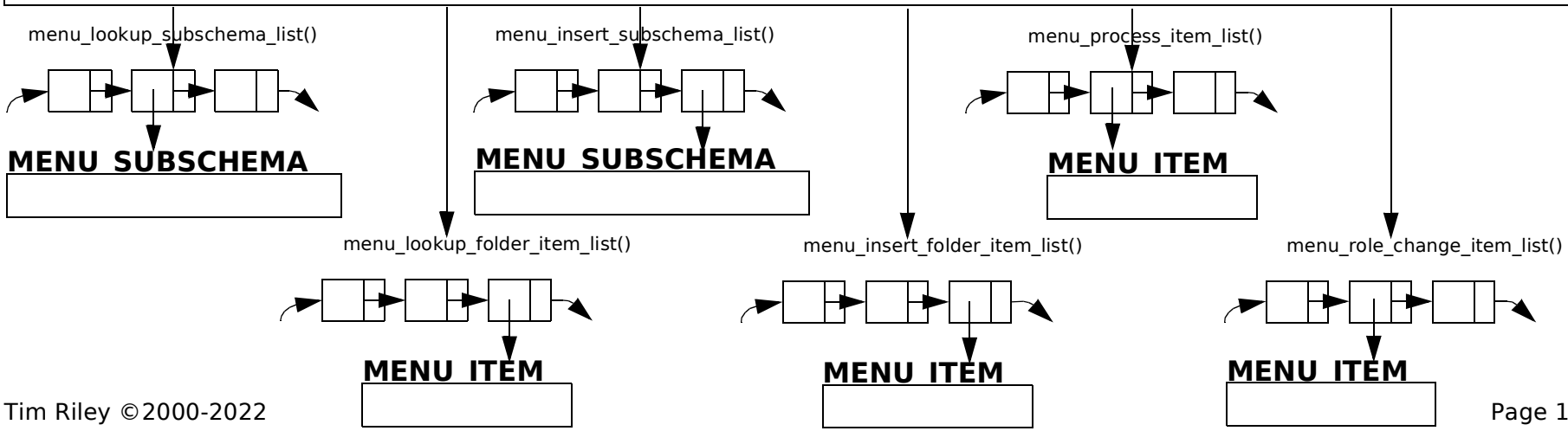
MENU

```
/* Usage */
MENU *menu_fetch(
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame,
    LIST *folder_menu_lookup_count_list() );

/* Process */
LIST *menu_verb_list = list_new();

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_lookup_subschema_fetch(
        role_folder_lookup_list(),
        folder_menu_lookup_count_list,
        application_name,
        login_name,
        session_key,
        role_name,
        target_frame ) );

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_insert_subschema_fetch(
        role_folder_insert_list(),
        application_name,
        login_name,
        session_key,
        role_name,
        target_frame ) );

LIST *role_process_list(
    role_name );

LIST *role_process_set_list(
    role_name );

LIST *menu_process_group_name_list(
    role_process_group_name_list(
        role_process_list() ),
    role_process_set_group_name_list(
        role_process_set_list() ) );

for group_name in
    'menu_process_group_name_list()'
{
    list_set(
        menu_verb_list,
        MENU_VERB *menu_verb_process_fetch(
            group_name /* verb */,
            role_process_list(),
            role_process_set_list(),
            application_name,
            login_name,
            session_key,
            role_name,
            target_frame ) );
}

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_role_change_fetch(
        role_name_list(
            role_list_fetch(
                login_name ) ),
        application_name,
        login_name,
        session_key,
        role_name ) );

list_set(
    menu_verb_list,
    MENU_VERB *menu_verb_role_display(
        role_name ) );

list_set(
    menu_verb_list,
    MENU_VERB*menu_verb_login_name_display(
        login_name ) );

/* Public */
boolean menu_boolean(
    char *current_frame,
    boolean frameset_menu_horizontal() );

char *menu_horizontal_html(
    char *document_body_hide_preload_html(),
    LIST *menu_verb_list );

char *menu_vertical_html(
    LIST *menu_verb_list );

char *menu_output_system_string(
    char *MENU_OUTPUT_EXECUTABLE,
    char *session_key,
    char *login_name,
    char *role_name,
    boolean frameset_menu_horizontal(),
    char *output_filename );
```



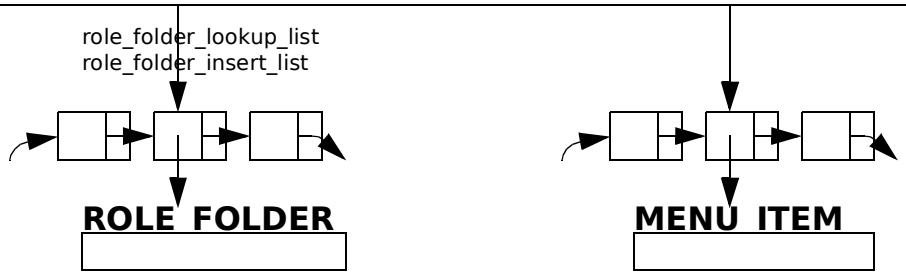
MENU_VERB

```
/* Usage */
MENU_VERB *menu_verb_lookup_subschema_fetch(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_insert_subschema_fetch(
    role_folder_insert_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_process_fetch(
    char *verb,
    LIST *role_process_list(),
    LIST *role_process_set_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_VERB *menu_verb_role_change_fetch(
    char *role_name /* verb */,
    LIST *role_name_list,
    char *application_name,
    char *login_name,
    char *session_key );

MENU_VERB *menu_verb_role_display(
    char *role_name /* verb */ );

MENU_VERB*menu_verb_login_name_display(
    char *login_name /* verb */ );

/* Process */
LIST *menu_lookup_subschema_list(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_lookup_folder_item_list(
    role_folder_subschema_missing_folder_name_list(
        role_folder_lookup_list() ),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_subschema_list(
    role_folder_insert_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_folder_item_list(
    role_folder_subschema_missing_folder_name_list(
        role_folder_insert_list() ),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_process_item_list(
    role_process_or_set_name_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_role_change_item_list(
    role_name_list,
    application_name,
    login_name,
    session_key,
    role_name );

char *menu_verb_tag( void );
```



MENU_SUBSCHEMA

```
/* Usage */
LIST *menu_lookup_subschema_list(
    role_folder_lookup_list(),
    folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_insert_subschema_list(
    role_folder_insert_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_SUBSCHEMA *menu_lookup_subschema_fetch(
    char *subschema_name,
    LIST *role_folder_lookup_name_list(),
    LIST *folder_menu_lookup_count_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_SUBSCHEMA *menu_insert_subschema_fetch(
    char *subschema_name,
    LIST *role_folder_insert_name_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

/* Input */
char *subschema_name;
char *application_name;
char *login_name;
char *session_key;
char *role_name;
char *target_frame;

/* Process */
LIST *menu_lookup_folder_item_list(
    role_folder_lookup_name_list(),
    folder_menu_lookup_count_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

LIST *menu_insert_folder_item_list(
    role_folder_insert_name_list(),
    application_name,
    login_name,
    session_key,
    role_name,
    target_frame );

char *menu_subschema_span_tag(
    char *subschema_name );

/* Public */
void menu_subschema_horizontal_output(
    FILE *output_stream,
    LIST *subschema_list );
```



MENU_ITEM

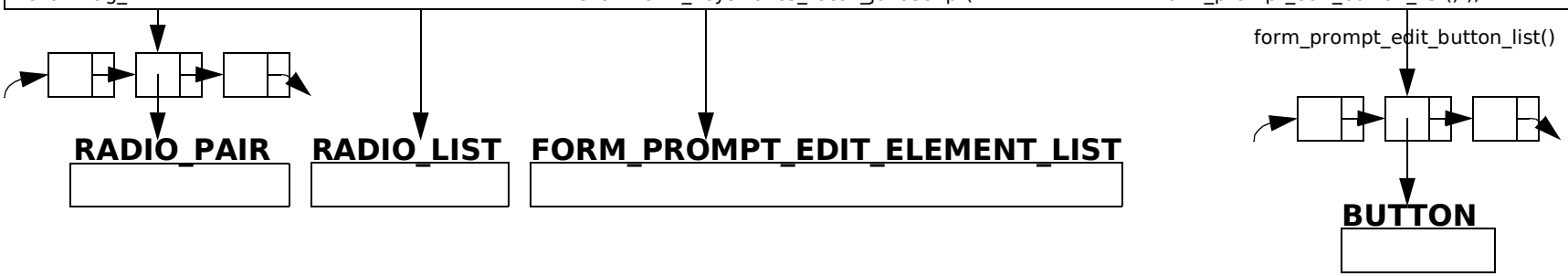
```
/* Usage */
LIST *menu_lookup_folder_item_list(
    LIST *role_folder_lookup_name_list,
    LIST *folder_menu_lookup_count_list,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_insert_folder_item_list(
    LIST *role_folder_insert_name_list,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_process_item_list(
    role_process_or_set_name_list(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

LIST *menu_role_change_item_list(
    LIST *role_name_list,
    char *application_name,
    char *login_name,
    char *session_key );

MENU_ITEM *menu_item_folder_new(
    char *folder_name,
    char *folder_menu_seek()->
        folder_menu_count_display,
    char *state,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_ITEM *menu_item_process_new(
    char *process_or_set_name,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

MENU_ITEM *menu_item_role_change_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *change_role_name );

/* Attributes */
char *folder_name;
char *folder_menu_count_display;
char *process_or_set_name;
char *application_name;
char *login_name;
char *session_key;
char *role_name;
char *target_frame;

/* Process */
char *state = “lookup” | ”insert”;

char *menu_item_folder_span_tag(
    char *folder_name,
    char *folder_menu_count_display );

char *menu_item_folder_action_tag(
    appaserver_library_http_prompt(),
    char *folder_name,
    char *state,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

char *menu_item_process_span_tag(
    char *process_or_set_name );

char *menu_item_process_action_tag(
    appaserver_library_http_prompt(),
    char *process_or_set_name,
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name,
    char *target_frame );

char *menu_item_role_name_span_tag(
    char *role_name );

char *menu_item_role_change_action_tag(
    appaserver_library_http_prompt(),
    char *application_name,
    char *login_name,
    char *session_key,
    char *role_name );

/* Public */
void menu_item_horizontal_output(
    FILE *output_stream,
    LIST *menu_item_list );
```

FORM

```
/* Public */
char *form_tag_html(
    char *name,
    char *action_string,
    char *target_frame );

char *form_close_html(
    void );

char *form_next_reference_number(
    int *form_current_reference_number );

char *form_element_list_html(
    char *form_tag_html(),
    LIST *element_list,
    char *form_close_html() );

char *form_cookie_key(
    char *form_name,
    char *folder_name );

char *form_cookie_multi_key(
    char *form_name,
    char *folder_name );

char *form_keystrokes_save_javascript(
    char *form_name,
    char *form_cookie_key(),
    LIST *element_list );

char *form_keystrokes_multi_save_javascript(
    char *form_name,
    char *form_cookie_multi_key(),
    LIST *element_list );

char *form_keystrokes_recall_javascript(
    char *form_name,
    char *form_cookie_key(),
    LIST *element_list );

char *form_keystrokes_multi_recall_javascript(
    char *form_name,
    char *form_cookie_multi_key(),
    LIST *element_list );

char *form_multi_select_all_javascript(
    LIST *element_list );

char *form_verify_notepad_widths_javascript(
    char *form_name,
    LIST *element_list );
```

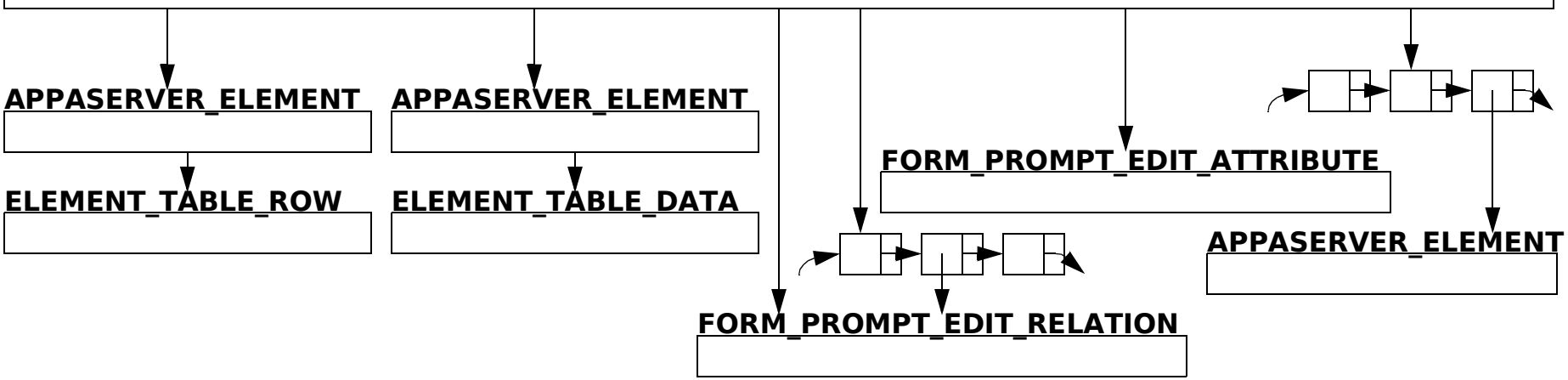
FORM_PROMPT_EDIT

```
/* Usage */
FORM_PROMPT_EDIT *form_prompt_edit_new(
    char *folder_name,
    char *prompt_edit_action_string(),
    boolean prompt_edit_omit_insert_button(),
    boolean prompt_edit_omit_delete_button(),
    LIST *folder->folder_attribute_append_isa_list,
    LIST *folder->relation_mto1_non_isa_list,
    LIST *folder->relation_join_one2m_list,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    char *security_entity_where(),
    boolean drillthru_participating(),
    boolean drillthru_skipped(),
    boolean drillthru_finished() );

/* Process */
LIST *form_prompt_edit_radio_pair_list(
    boolean prompt_edit_omit_insert_button(),
    boolean prompt_edit_omit_delete_button(),
    int list_length( relation_mto1_non_isa_list )
        /* relation_mto1_non_isa_list_length */ );

RADIO_LIST *radio_list =
    radio_list_new(
        FORM_RADIO_LIST_NAME,
        form_prompt_edit_radio_pair_list(),
        RADIO_LOOKUP_LABEL /* initial_label */ );

char *form_prompt_edit_target_frame(
    boolean drillthru_participating(),
    boolean drillthru_skipped(),
    boolean drillthru_finished(),
    char *FRAMESET_PROMPT_FRAME,
    char *FRAMESET_EDIT_FRAME );

char *tag_html =
    form_tag_html(
        FORM_PROMPT_EDIT_NAME /* form_name */,
        prompt_edit_action_string(),
        form_prompt_edit_target_frame() );

FORM_PROMPT_EDIT_ELEMENT_LIST *
    form_prompt_edit_element_list =
        form_prompt_edit_element_list_new(
            folder_attribute_append_isa_list,
            relation_mto1_non_isa_list,
            relation_join_one2m_list,
            drillthru_dictionary,
            login_name,
            security_entity_where );

char *form_multi_select_all_javascript(
    form_prompt_edit_element_list->element_list );

char *form_cookie_key(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    folder_name );

char *form_cookie_multi_key(
    FORM_PROMPT_EDIT /* form_name */,
    folder_name );

char *form_keystrokes_save_javascript(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    form_cookie_key(),
    form_prompt_edit_element_list->element_list );

char *form_keystrokes_multi_save_javascript(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    form_cookie_multi_key(),
    form_prompt_edit_element_list->element_list );

char *form_keystrokes_recall_javascript(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    form_cookie_key(),
    form_prompt_edit_element_list->element_list );

char *form_keystrokes_multi_recall_javascript(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    form_cookie_multi_key(),
    form_prompt_edit_element_list->element_list );

char *form_verify_notepad_widths_javascript(
    FORM_PROMPT_EDIT_NAME /* form_name */,
    form_prompt_edit_element_list->element_list );

LIST *form_prompt_edit_button_list(
    char *form_multi_select_all_javascript(),
    char *form_keystrokes_save_javascript(),
    char *form_keystrokes_multi_save_javascript(),
    char *form_keystrokes_recall_javascript(),
    char *form_keystrokes_multi_recall_javascript(),
    char *form_verify_notepad_widths_javascript() );

char *form_prompt_edit_html(
    char *tag_html,
    char *radio_list->html,
    char *form_prompt_edit_element_list->
        element_list_html,
    char *button_list_html(
        form_prompt_edit_button_list() ),
    char *form_close_html() );

free( form_prompt_edit_element_list->
        element_list_html );

free( button_list_html() );

button_list_free(
    form_prompt_edit_button_list() );
```



FORM_PROMPT_EDIT_ELEMENT_LIST

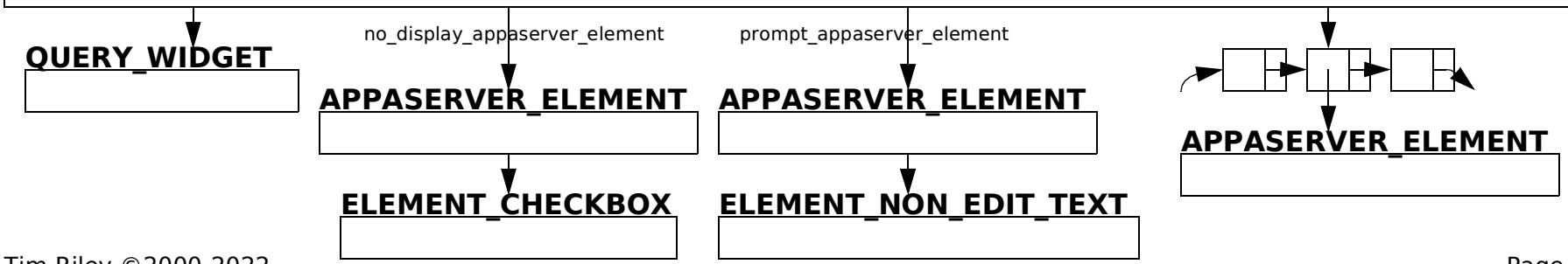
```
/* Usage */
FORM_PROMPT_EDIT_ELEMENT_LIST *
  form_prompt_edit_element_list_new(
     LIST *folder_attribute_append_isa_list,
     LIST *relation_mto1_non_isa_list,
     LIST *relation_join_one2m_list,
     DICTIONARY *drillthru_dictionary,
     char *login_name,
     char *security_entity_where );

/* Process */
LIST *element_list = list_new();

for FOLDER_ATTRIBUTE *folder_attribute in
  folder_attribute_append_isa_list
{
  list_set(
     element_list,
        appaserver_element_new(
           table_row, (char *)0 ) );

  list_set(
     element_list,
        appaserver_element_new(
           table_data (char *)0 ) );

if ( ( form_prompt_edit_relation =
       form_prompt_edit_relation_new(
          folder_attribute->attribute_name,
          folder_attribute->primary_key_index,
          relation_mto1_non_isa_list,
          drillthru_dictionary,
          login_name,
          security_entity_where,
          form_prompt_edit_relation_list ) ) )
{
  if ( !form_prompt_edit_relation_list )
  {
     LIST *form_prompt_edit_relation_list =
        list_new();
  }

  list_set(
     form_prompt_edit_relation_list,
     form_prompt_relation );

  list_set_list(
     element_list,
     form_prompt_edit_relation->element_list );

  continue;
}

if ( ( form_prompt_edit_attribute =
       form_prompt_edit_attribute_new(
          folder_attribute->attribute_name,
          folder_attribute->primary_key_index,
          folder_attribute->attribute->datatype_name,
          folder_attribute->attribute->width,
          folder_attribute->attribute->hint_message,
          form_prompt_edit_relation_list ) ) )
{
  list_set_list(
     element_list,
     form_prompt_edit_attribute->element_list );
}
}

LIST *form_prompt_edit_element_list_join_element_list(
     char *FORM_PROMPT_EDIT_NO_DISPLAY_PREFIX,
     LIST *relation_join_one2m_list );

list_set_list(
  element_list,
  form_prompt_edit_element_list_join_element_list() );

char *element_list_html(
  LIST *element_list );
```



FORM_PROMPT_EDIT_RELATION (1)

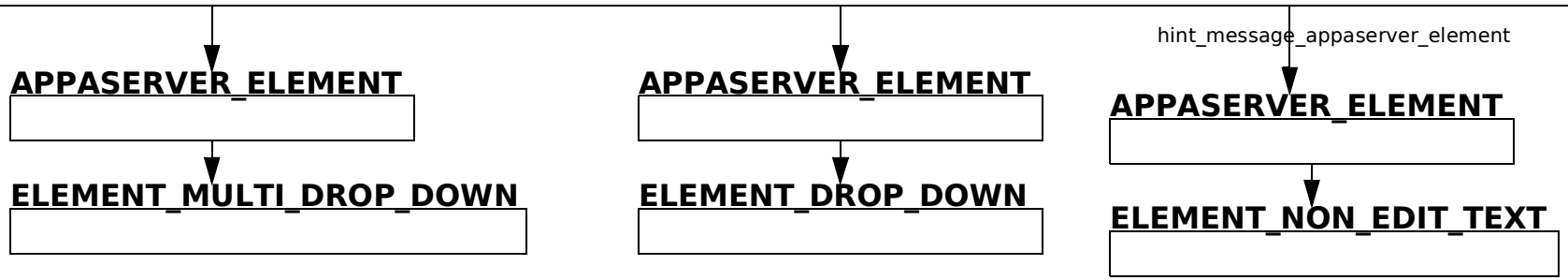
This is a query row having a mto1 relation drop-down.

```
/* Usage */
FORM_PROMPT_EDIT_RELATION *
   form_prompt_edit_relation_new(
      char *attribute_name,
      int primary_key_index,
      LIST *relation_mto1_non_isa_list,
      DICTIONARY *drillthru_dictionary,
      char *login_name,
      char *security_entity_where,
      LIST *form_prompt_edit_relation_list );

/* Process */
if ( form_prompt_edit_relation_attribute_name_exists(
   char *attribute_name,
   LIST *form_prompt_edit_relation_list ) )
{
   return (FORM_PROMPT_EDIT_RELATION *)0;
}

RELATION *relation =
   relation_consumes(
      attribute_name,
      relation_mto1_non_isa_list );

if ( !relation )
{
   return (FORM_PROMPT_EDIT_RELATION *)0;
}

LIST *element_list = list_new();

QUERY_WIDGET *query_widget =
   query_widget_new(
      relation->one_folder->folder_name
         /* widget_folder_name */,
      login_name,
      relation->
         one_folder->
         folder_attribute_list(),
      relation->
         one_folder->
         relation_mto1_non_isa_list,
      security_entity_where,
   drillthru_dictionary );

char *form_prompt_edit_relation_name(
   char *relation->one_folder->folder_name
      /* one_folder_name */,
   char *relation->related_attribute_name,
   LIST *relation->foreign_key_list );

char *form_prompt_edit_relation_no_display_name(
   char *FORM_PROMPT_EDIT_NO_DISPLAY_PREFIX,
   char *form_prompt_edit_relation_name() );

list_set(
   element_list,
   ( no_display_appaserver_element =
      appaserver_element_new(
         checkbox,
         form_prompt_edit_relation_no_display_name() ) ) );

ELEMENT_CHECKBOX *checkbox =
   element_checkbox_new(
      no_display_appaserver_element->element_name,
      (char *)0 /* element_name */,
      (char *)0 /* prompt_string */,
      (char *)0 /* on_click */,
      -1 /* tab_order */,
      (char *)0 /* image_source */,
      1 /* recall */ );

list_set(
      element_list,
         appaserver_element_new(
            table_data, (char *)0 ) );

char *form_prompt_edit_relation_prompt(
   char *relation->one_folder->folder_name
      /* one_folder_name */,
   char *relation->related_attribute_name,
   LIST *relation->foreign_key_list,
   int primary_key_index );

list_set(
   element_list,
   ( prompt_appaserver_element =
      appaserver_element_new(
         non_edit_text, (char *)0 ) ) );

ELEMENT_NON_EDIT_TEXT *non_edit_text =
   element_non_edit_text_new(
         (char *)0,
         form_prompt_edit_relation_prompt() );

list_set(
   element_list,
   appaserver_element_new(
         table_data, (char *)0 ) );
```



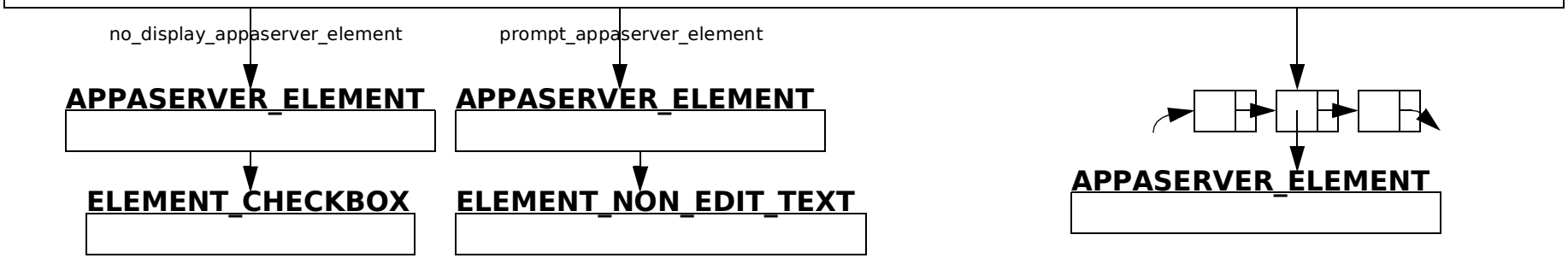
FORM_PROMPT_EDIT_RELATION (2)

```
/* Process */
char *form_prompt_edit_relation_element_name(
   char *FORM_PROMPT_EDIT_RELATION_PREFIX,
   char *form_prompt_edit_relation_name() );

if ( relation->one_folder->multi_select )
{
   char *form_prompt_edit_relation_original_name(
      char *FORM_PROMPT_EDIT_ORIGINAL_PREFIX,
      char *form_prompt_edit_relation_name() );

   list_set(
      element_list,
      ( drop_down_appaserver_element =
         appaserver_element_new(
            multi_drop_down, (char *)0 ) ) );

   ELEMENT_MULTI_DROP_DOWN *
      multi_drop_down =
         element_multi_drop_down_new(
            form_prompt_edit_relation_original_name(),
            form_prompt_edit_relation_element_name(),
            query_widget->delimited_list,
            relation->one_folder->no_initial_capital );
}
else
{
   list_set(
      element_list,
      ( drop_down_appaserver_element =
         appaserver_element_new(
            prompt_drop_down,
            form_prompt_edit_relation_element_name() ) ) );

   ELEMENT_PROMPT_DROP_DOWN *
      prompt_drop_down =
         element_prompt_drop_down_new(
            drop_down_appaserver_element->
               element_name,
            query_widget->delimited_list,
            relation->one_folder->no_initial_capital,
            1 /* output_null_option */,
            1 /* output_not_null_option */,
            1 /* output_select_option */,
            element_drop_down_display_size(),
            -1 /* tab_order */,
            0 /* not multi_select */,
            (char *)0 /* post_change_javascript */,
            1 /* recall */ );

   list_set(
      element_list,
         appaserver_element_new(
            table_data, (char *)0 ) );

   list_set(
      element_list,
         appaserver_element_new(
            table_data, (char *)0 ) );

   list_set(
      element_list,
         appaserver_element_new(
            table_data, (char *)0 ) );
}

if ( relation->hint_message )
{
   list_set(
      element_list,
      ( hint_message_appaserver_element =
         appaserver_element_new(
            non_edit_text, (char *)0 ) ) );

   ELEMENT_NON_EDIT_TEXT *non_edit_text =
      element_non_edit_text_new(
         (char *)0 /* prompt_name */,
         relation->hint_message );
}
```



FORM_PROMPT_EDIT_ATTRIBUTE (1)

```
/* Usage */
FORM_PROMPT_EDIT_ATTRIBUTE *
  form_prompt_edit_attribute_new(
     char *attribute_name,
     int primary_key_index,
     char *datatype_name,
     int attribute_width,
     char *hint_message,
     LIST *form_prompt_edit_relation_list );

/* Process */
if ( form_prompt_edit_relation_attribute_name_exists(
  attribute_name,
  form_prompt_edit_relation_list ) )
{
  return (FORM_PROMPT_EDIT_ATTRIBUTE *)0;
}

LIST *element_list = list_new();

char *form_prompt_edit_attribute_no_display_name(
  char *FORM_PROMPT_EDIT_NO_DISPLAY_PREFIX,
  char *attribute_name );

list_set(
  element_list,
  ( no_display_appaserver_element =
     appaserver_element_new(
        checkbox,
        form_prompt_edit_attribute_no_display_name() ) ) );

ELEMENT_CHECKBOX *checkbox =
  element_checkbox_new(
     no_display_appaserver_element->element_name,
     (char *)0 /* element_name */,
     (char *)0 /* prompt_string */,
     (char *)0 /* on_click */,
     -1 /* tab_order */,
     (char *)0 /* image_source */,
     1 /* recall */ );

list_set(
     element_list,
        appaserver_element_new(
           table_data, (char *)0 ) );

char *form_prompt_edit_attribute_prompt(
  char *attribute_name,
  int primary_key_index );

list_set(
  element_list,
  ( prompt_appaserver_element =
     appaserver_element_new(
        non_edit_text, (char *)0 ) ) );

ELEMENT_NON_EDIT_TEXT *non_edit_text =
  element_non_edit_text_new(
     (char *)0,
     form_prompt_edit_attribute_prompt() );

list_set(
     element_list,
        appaserver_element_new(
           table_data, (char *)0 ) );
```



FORM_PROMPT_EDIT_ATTRIBUTE (2)

```
/* Process */
char *form_prompt_edit_attribute_from_name(
  char *FORM_PROMPT_EDIT_FROM_PREFIX,
  char *attribute_name );

if ( attribute_is_yes_no( attribute_name ) )
{
  list_set(
    element_list,
    ( yes_no_appaserver_element =
      appaserver_element_new(
        yes_no,
        form_prompt_edit_attribute_from_name() ) ) );

  ELEMENT_YES_NO *yes_no =
    element_yes_no_new(
      (char *)0 /* attribute_name */,
      yes_no_appaserver_element->element_name,
      1 /* output_null_option */,
      1 /* output_not_null_option */,
      (char *)0 /* post_change_javascript */,
      -1 /* tab_order */,
      1 /* recall */ );

  list_set(
    element_list,
      appaserver_element_new(
        table_data, (char *)0 ) );

  list_set(
    element_list,
      appaserver_element_new(
        table_data, (char *)0 ) );

  list_set(
    element_list,
      appaserver_element_new(
        table_data, (char *)0 ) );
}
else
{
  char *form_prompt_edit_attribute_relational_name(
    char *FORM_PROMPT_EDIT_RELATIONAL_PREFIX,
    char *attribute_name );

  char *form_prompt_edit_attribute_to_name(
    char *FORM_PROMPT_EDIT_TO_PREFIX,
    char *attribute_name );

  FORM_PROMPT_EDIT_RELATIONAL *
    form_prompt_edit_relational =
      form_prompt_edit_relational_new(
        form_prompt_edit_attribute_relational_name(),
        form_prompt_edit_attribute_from_name(),
        form_prompt_edit_attribute_to_name(),
        datatype_name,
        attribute_width );

  list_set_list(
    element_list,
    form_prompt_edit_relational->
      element_list );
}

if ( hint_message )
{
  list_set(
    element_list,
      appaserver_element_new(
        table_data, (char *)0 ) );

  list_set(
    element_list,
    ( hint_message_appaserver_element =
      appaserver_element_new(
        non_edit_text, (char *)0 ) ) );

  ELEMENT_NON_EDIT_TEXT *non_edit_text =
    element_non_edit_text_new(
      (char *)0 /* prompt_name */,
      hint_message );
}
```

yes_no_appaserver_element

APPASERVER_ELEMENT

ELEMENT_YES_NO

This is a query row having a relational operator drop-down.

FORM_PROMPT_EDIT_RELATIONAL

hint_message_appaserver_element

APPASERVER_ELEMENT

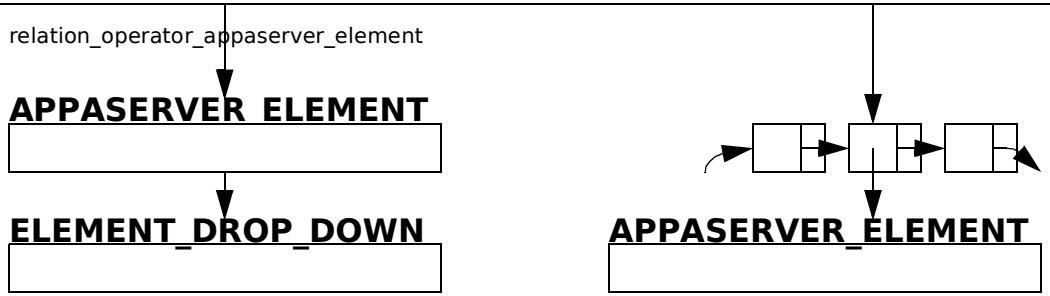
ELEMENT_NON_EDIT_TEXT

APPASERVER Data Structure

FORM_PROMPT_EDIT_RELATIONAL (1)

This is a query row having a relational operator drop-down.

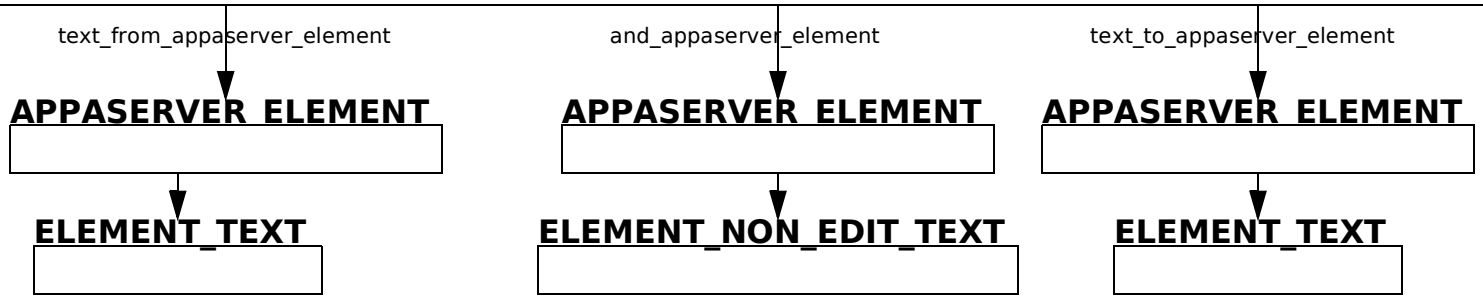
```
/* Usage */
FORM_PROMPT_EDIT_RELATIONAL *
  form_prompt_edit_relational_new(
      form_prompt_edit_attribute_relational_name(),
      form_prompt_edit_attribute_from_name(),
      form_prompt_edit_attribute_to_name(),
      char *datatype_name,
      int attribute_width );

/* Process */
LIST *element_list = list_new();

LIST *form_prompt_edit_relational_operation_list(
  char *datatype_name );

list_set(
  element_list,
  ( relation_operator_appaserver_element =
      appaserver_element_new(
          prompt_drop_down,
          form_prompt_edit_attribute_relational_name() ) ) );

ELEMENT_PROMPT_DROP_DOWN *
  prompt_drop_down =
      element_prompt_drop_down_new(
          relation_operator_appaserver_element->
              element_name,
          form_prompt_edit_relational_operation_list(),
          0 /* not no_initial_capital */,
          1 /* output_null_option */,
          1 /* output_not_null_option */,
          1 /* output_select_option */,
          1 /* element_drop_down_display_size */,
          -1 /* tab_order */,
          0 /* not multi_select */,
          (char *)0 /* post_change_javascript */,
          1 /* recall */ );

list_set(
  element_list,
  appaserver_element_new(
      table_data, (char *)0 ) );
```



APPASERVER Data Structure

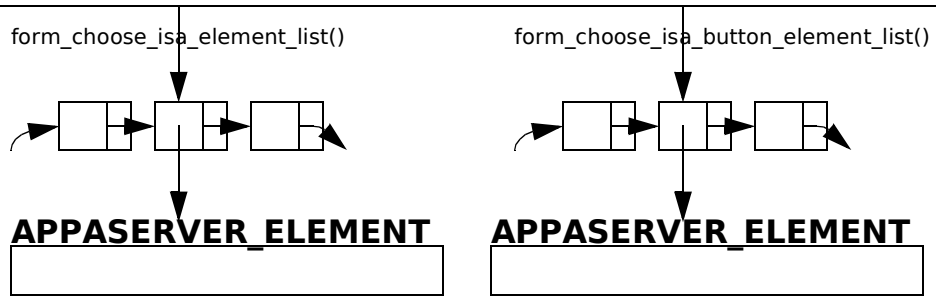
FORM_PROMPT_EDIT_RELATIONAL (2)

```
/* Process */
list_set(
  element_list,
  ( text_from_appaserver_element =
     appaserver_element_new(
        text,
        form_prompt_edit_attribute_from_name() ) ) );

ELEMENT_TEXT *text =
  element_text_new(
     text_from_appaserver_element->element_name,
     datatype_name,
     FORM_PROMPT_EDIT_FROM_ATTRIBUTE_WIDTH,
     0 /* not null_to_slash */,
     1 /* prevent_carrot */,
     (char *)0 /* on_change */,
     (char *)0 /* on_focus */,
     (char *)0 /* on_keyup */,
     -1 /* tab_order*/,
     1 /* recall */ );

list_set(
  element_list,
  appaserver_element_new(
        table_data, (char *)0 ) );

list_set(
  element_list,
  ( and_appaserver_element =
     appaserver_element_new(
        non_edit_text, (char *)0 ) ) );

ELEMENT_NON_EDIT_TEXT *non_edit_text =
  element_non_edit_text_new(
     (char *)0,
     "and" /* message */ );

list_set(
  element_list,
  appaserver_element_new(
        table_data, (char *)0 ) );

list_set(
  element_list,
  ( text_to_appaserver_element =
     appaserver_element_new(
        text,
        form_prompt_edit_attribute_to_name() ) ) );

ELEMENT_TEXT *text =
  element_text_new(
     text_to_appaserver_element->element_name,
     datatype_name,
     attribute_width,
     0 /* not null_to_slash */,
     1 /* prevent_carrot */,
     (char *)0 /* on_change */,
     (char *)0 /* on_focus */,
     (char *)0 /* on_keyup */,
     -1 /* tab_order*/,
     1 /* recall */ );
```



FORM_CHOOSE_ISA

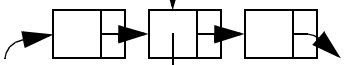
```
/* Usage */
FORM_CHOOSE_ISA *form_choose_isa_new(
    char *choose_isa_prompt_message(),
    LIST *primary_key_list,
    LIST *delimited_list,
    boolean no_initial_capital,
    char *choose_isa_action_string() );

/* Process */
char *tag_html =
    form_tag_html(
        "choose_isa" /* form_name */,
        choose_isa_action_string,
        FRAMESET_PROMPT_FRAME /* target_frame */ );

LIST *form_choose_isa_element_list(
    char *choose_isa_prompt_message,
    LIST *primary_key_list,
    LIST *delimited_list,
    boolean no_initial_capital );

LIST *form_choose_isa_button_element_list(
    void );

char *form_choose_isa_html(
    tag_html,
    form_choose_isa_element_list(),
    form_choose_isa_button_element_list(),
    char *form_close_html() );
```



FORM_CHOOSE_ROLE

```
/* Usage */
FORM_CHOOSE_ROLE *form_choose_role_new(
    LIST *role_name_list,
    char *choose_role_post_action_string(),
    char *choose_role_target_frame(),
    char *choose_role_form_name,
    char *choose_role_drop_down_element_name  );

/* Process */
char *tag_html =
    form_tag_html(
        choose_role_form_name,
        choose_role_post_action_string,
        choose_role_target_frame );

char *form_choose_role_drop_down_onchange_javascript(
    char *choose_role_form_name );

LIST *form_choose_role_element_list(
    LIST *role_name_list,
    char *form_choose_role_drop_down_onchange_javascript(),
    char *choose_role_drop_down_element_name );

char *form_choose_role_html(
    char *tag_html,
    LIST *form_choose_role_element_list(),
    char *form_close_html() );
```

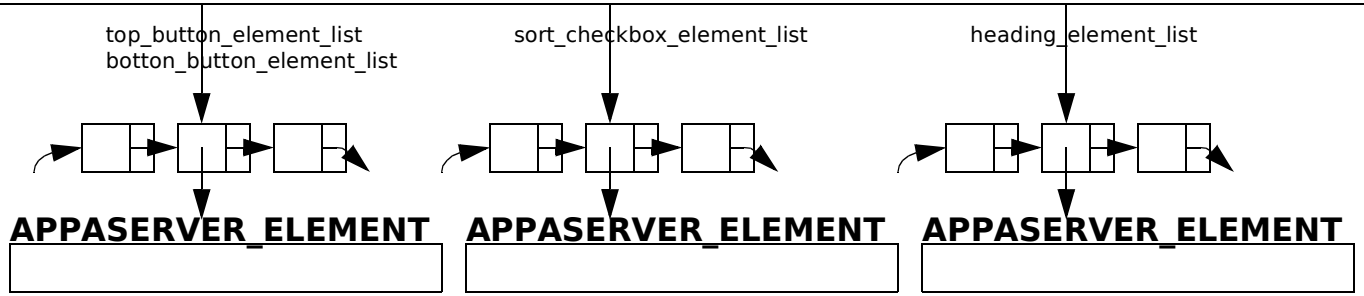
form_choose_role_element_list()



APPASERVER_ELEMENT

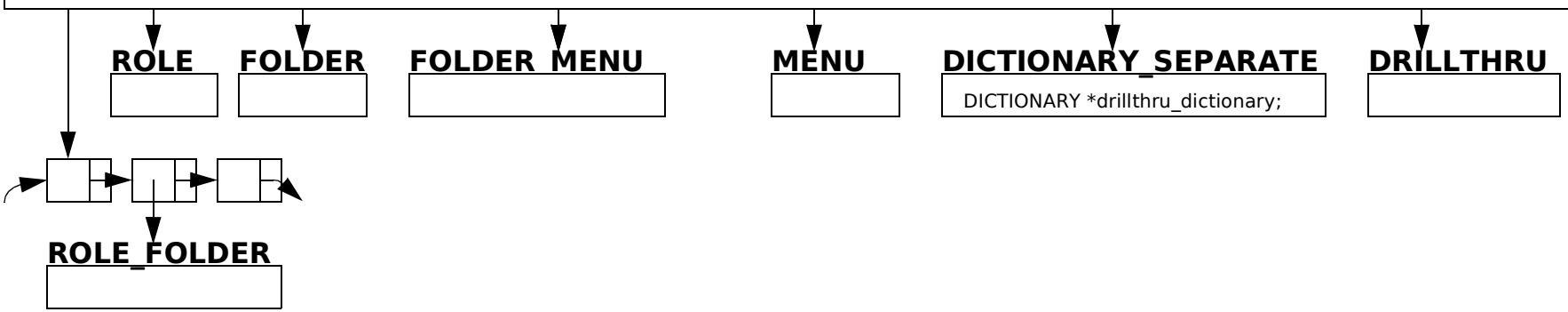
FORM_EDIT_TABLE

```
/* Usage */
FORM_EDIT_TABLE *form_edit_table_new(
    char *folder_name,
    char *post_change_javascript,
    int dictionary_list_length,
    char *edit_table_submit_action_string(),
    LIST *role_operation_list,
    LIST *edit_table_heading_name_list(),
    char *target_frame,
    DICTIONARY *query_dictionary,
    DICTIONARY *sort_dictioanry,
    DICTIONARY *drillthru_dictionary,
    DICTIOANRY *ignore_dictionary );

/* Process */
char *form_edit_table_tag(
    char *FORM_EDIT_TABLE_NAME,
    char *edit_table_submit_action_string(),
    char *target_frame,
    char *element_table_open_html() );

LIST *top_button_element_list =
    form_edit_table_button_element_list(
        char *post_change_javascript,
        0 /* dictionary_list_length */ );

char *top_button_element_list_html =
    element_list_html(
        top_button_element_list );

LIST *bottom_button_element_list =
    form_edit_table_button_element_list(
        char *post_change_javascript,
        dictionary_list_length );

char *bottom_button_element_list_html =
    element_list_html(
        bottom_button_element_list );

LIST *form_edit_table_sort_checkbox_element_list(
    char *folder_name,
    int list_length( role_operation_list ),
    LIST *edit_table_heading_name_list );

char *sort_checkbox_element_list_html =
    element_list_html(
        form_edit_table_sort_checkbox_element_list() );

LIST *form_edit_table_heading_element_list(
    LIST *role_operation_list,
    LIST *edit_table_heading_name_list );

char *heading_element_list_html =
    element_list_html(
        form_edit_table_heading_element_list() );

char *query_dictionary_hidden_html =
    dictionary_separate_hidden_html(
        DICTIONARY_SEPARATE_QUERY_PREFIX,
        query_dictionary );

char *sort_dictionary_hidden_html =
    dictionary_separate_hidden_html(
        DICTIONARY_SEPARATE_SORT_PREFIX,
        sort_dictionary );

char *drillthru_dictionary_hidden_html =
    dictionary_separate_hidden_html(
        DICTIONARY_SEPARATE_DRILLTHRU_PREFIX,
        drillthru_dictionary );

char *ignore_dictionary_hidden_html =
    dictionary_separate_hidden_html(
        DICTIONARY_SEPARATE_IGNORE_PREFIX,
        ignore_dictionary );

char *form_edit_table_html(
    char *form_edit_table_tag(),
    char *top_button_element_list_html,
    char *sort_checkbox_element_list_html,
    char *heading_element_list_html );

char *form_edit_table_trailer_html(
    char *bottom_button_element_list_html,
    char *element_table_close_html(),
    char *query_dictionary_hidden_html,
    char *sort_dictionary_hidden_html,
    char *drillthru_dictionary_hidden_html,
    char *ignore_dictionary_hidden_html,
    char *form_close_html() );
```



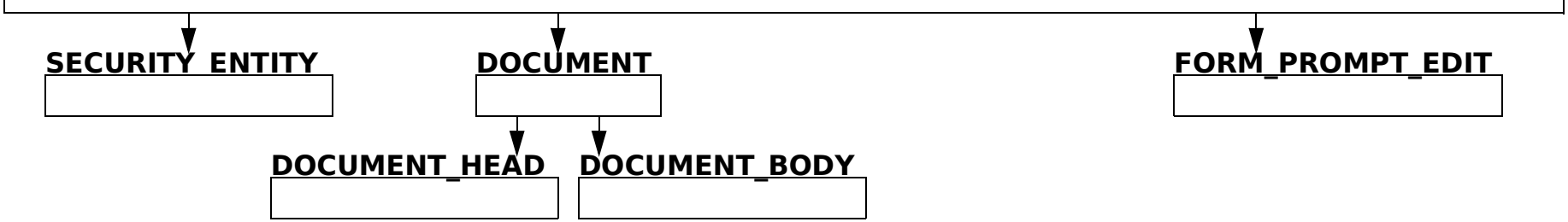
PROMPT_EDIT (1)

```
/* Usage */
PROMPT_EDIT *prompt_edit_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    char *state,
    boolean menu_boolean(
        FRAMESET_PROMPT_FRAME,
        boolean frameset_menu_horizontal(
            application_name,
            login_name ) ),
    char *appaserver_parameter->
        data_directory,
    POST_DICTIONARY *post_dictionary_string_new() );

/* Process */
LIST *role_folder_list(
    role_name,
    folder_name );

boolean prompt_edit_forbid(
    boolean role_folder_update( role_folder_list ),
    boolean role_folder_lookup( role_folder_list ),
    char *state,
    char *APPASERVER_UPDATE_STATE );

if ( prompt_edit_forbid() )
{
    return prompt_edit;
}

ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        /* Not fetching role_folder_list */
        (char *)0 /* role_name */,
        role_exclude_lookup_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets folder_attribute_primary_list */
        /* and primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

if ( menu_boolean() )
{
    FOLDER_MENU *folder_menu =
        folder_menu_fetch(
            application_name,
            session_key,
            data_directory,
            role_name );

    MENU *menu =
        menu_fetch(
            application_name,
            login_name,
            session_key,
            role_name,
            target_frame,
            folder_menu->lookup_count_list );
}

DICTIONARY_SEPARATE *dictionary_separate =
    dictionary_separate_folder_new(
        post_dictionary->original_post_dictionary,
        application_name,
        login_name,
        folder_attribute_date_name_list(
            folder->folder_attribute_append_isa_list ) );

DRILLTHRU *drillthru =
    drillthru_continue(
        dictionary_separate->
            drillthru_dictionary /* in/out */,
        folder_name );

if ( !drillthru_participating() || drillthru_finished() )
{
    folder->relation_join_one2m_list =
        relation_join_one2m_list(
            folder->relation_one2m_recursive_list,
            (DICTIONARY *)0 /* ignore_dictionary */ );
}
```



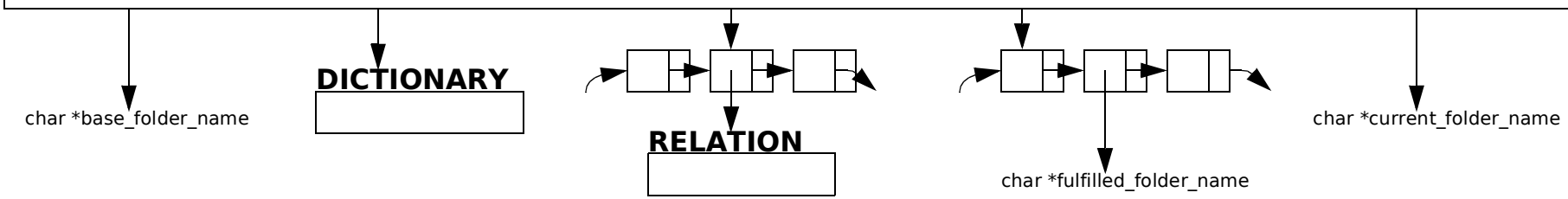
PROMPT_EDIT (2)

```
/* Process */
boolean prompt_edit_omit_insert_button(
    boolean drillthru_skipped(),
    boolean relation_exists_multi_select(
        folder->relation_mto1_non_isa_list ) );

boolean prompt_edit_omit_delete_button(
    list_length( folder->relation_mto1_isa_list ) );

char *prompt_edit_target_frame(
    char *target_frame,
    char *FRAMESET_EDIT_FRAME,
    boolean drillthru_skipped() );

folder->relation_mto1_non_isa_list =
    /* Returns relation_mto1_non_isa_list or */
    /* those with only a single foreign key. */
    prompt_edit_drillthru_skipped(
        LIST *relation_mto1_non_isa_list,
        drillthru_skipped() );

char *prompt_edit_title_html(
    char *state,
    char *folder_name );


char *prompt_edit_action_string(
    char *PROMPT_EDIT_POST_EXECUTABLE,
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *prompt_edit_target_frame(),
    char *state );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

FORM_PROMPT_EDIT *form_prompt_edit =
    form_prompt_edit_new(
        folder_name,
        prompt_edit_action_string(),
        prompt_edit_omit_insert_button(),
        prompt_edit_omit_delete_button(),
        folder->folder_attribute_append_isa_list,
        folder->relation_mto1_non_isa_list,
        folder->relation_join_one2m_list,
        dictionary_separate->drillthru_dictionary,
        login_name,
        security_entity_where(
            security_entity,
            folder->folder_attribute_list ),
        drillthru_participating(),
        drillthru_skipped(),
        drillthru_finished() );

int folder_attribute_date_name_list_length =
    list_length(
        folder_attribute_date_name_list(
            folder->folder_attribute_append_isa_list ) );

DOCUMENT *document =
    document_new(
        application_name,
        application_title_string(),
        char *prompt_edit_title_html(),
        (char *)0 /* subtitle_html */,
        (char *)0 /* subsubtitle_html */,
        (char *)0 /* javascript_replace */,
        menu_boolean(),
        menu,
        document_head_menu_setup_string(
            menu_boolean() ),
        document_head_calendar_setup_string(
            folder_attribute_date_name_list_length ),
        document_head_javascript_include_string(),
        (char *)0 /* input_onload_string */ );

char *document_form_html(
    document->html,
    document->document_head->html,
    document_head_close_html(),
    document->document_body->html,
    form_prompt_edit->html,
    document_body_close_html(),
    document_close_html() );

free( form_prompt_edit->html );

/* Public */
char *prompt_edit_output_system_string(
    char *PROMPT_EDIT_OUTPUT_EXECUTABLE,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    char *state,
    char *dictionary_separate_send_dictionary(),
    char *appaserver_error_filename() );
```



DRILLTHRU

```
/* Usage */
DRILLTHRU *drillthru_start(
    DICTIONARY *drillthru_dictionary /* out */,
    char *folder_name );

/* Process */
LIST *relation_mto1_drillthru_list(
    (LIST *)0 /* relation_list */,
    folder_name,
    (LIST *)0 /* fulfilled_folder_name_list */ );

boolean drillthru_participating =
    drillthru_start_participating(
        length( relation_mto1_drillthru_list ) );

if ( !drillthru_participating )
{
    return drillthru;
}

void drillthru_base_set(
    DICTIONARY *drillthru_dictionary,
    char *DRILLTHRU_BASE_KEY,
    char *folder_name );

/* Public */
char *drillthru_start_current_folder_name(
    LIST *relation_mto1_drillthru_list );

/* Usage */
DRILLTHRU *drillthru_continue(
    DICTIONARY *drillthru_dictionary /* in/out */,
    char *folder_name );

/* Process */
char *drillthru_base_folder_name(
    char *DRILLTHRU_BASE_KEY,
    DICTIONARY *drillthru_dictionary );

boolean drillthru_participating =
    drillthru_continue_participating(
        char *base_folder_name );

if ( !drillthru_participating )
{
    return drillthru;
}

boolean drillthru_skipped(
    int dictionary_length( drillthru_dictionary ) );

if ( drillthru_skipped() )
{
    return drillthru;
}

LIST *drillthru_fulfilled_folder_name_list(
    char *DRILLTHRU_FULFILLED_KEY,
    DICTIONARY *drillthru_dictionary );

LIST *relation_mto1_drillthru_list(
    (LIST *)0 /* relation_list */,
    drillthru_base_folder_name(),
    drillthru_fulfilled_folder_name_list() );

char *drillthru_current_folder_name(
    LIST *drillthru->relation_mto1_drillthru_list,
    char *drillthru_base_folder_name() );

boolean drillthru_finished(
    char *folder_name,
    drillthru_current_folder_name() );

if ( drillthru_finished() )
{
    return drillthru;
}

LIST *drillthru_current_fulfilled_folder_name_list(
    LIST *drillthru_fulfilled_folder_name_list() /* out */,
    char *drillthru_current_folder_name() );

void drillthru_dictionary_fulfilled_set(
    DICTIONARY *drillthru_dictionary /* out */,
    char *DRILLTHRU_FULFILLED_KEY,
    LIST *drillthru_current_fulfilled_folder_name_list() );

/* Private */
DRILLTHRU *drillthru_calloc( void );
```



APPASERVER Data Structure

APPASERVER_ELEMENT (1)

```
/* Usage */
APPASERVER_ELEMENT *appaserver_element_new(
    enum element_type element_type,
    char *element_name );

/* Attributes */
enum element_type element_type;
char *element_name;
char *heading_string;
char *element_key;

ELEMENT_TABLE_OPEN *table_open;
ELEMENT_TABLE_HEADING *table_heading;
ELEMENT_TABLE_ROW *table_row;
ELEMENT_TABLE_CLOSE *table_close;
ELEMENT_CHECKBOX *checkbox;
ELEMENT_PROMPT_DROP_DOWN *prompt_drop_down;
ELEMENT_DROP_DOWN *drop_down;
ELEMENT_BUTTON *button;
ELEMENT_NON_EDIT_TEXT *non_edit_text;
ELEMENT_HIDDEN *hidden;
ELEMENT_LINE_BREAK *line_break;
ELEMENT_TABLE_DATA *table_data;
ELEMENT_MULTI_DROP_DOWN *multi_drop_down;
ELEMENT_NOTEPAD *notepad;
ELEMENT_TEXT *text;
ELEMENT_PASSWORD *password;
ELEMENT_UPLOAD *upload;
ELEMENT_YES_NO *yes_no;

ELEMENT_RADIO *radio;
ELEMENT_PROMPT_DATA *prompt_data;
ELEMENT_REFERENCE_NUMBER *reference_number;
ELEMENT_JAVASCRIPT_FILENAME *javascript_filename;
ELEMENT_DATE *date;
ELEMENT_CURRENT_DATE *current_date;
ELEMENT_TIME *time;
ELEMENT_CURRENT_TIME *current_time;
ELEMENT_CURRENT_DATE_TIME *current_date_time;
ELEMENT_TIMESTAMP *timestamp;
ELEMENT_EMPTY_COLUMN *empty_column;

/* Public */
APPASERVER_ELEMENT *
    appaserver_element_name_seek(
        char *element_name,
        LIST *appaserver_element_list );

APPASERVER_ELEMENT *
    appaserver_element_key_seek(
        char *key_string,
        LIST *appaserver_element_list );

LIST *appaserver_element_heading_name_list(
    LIST *appaserver_element_list );

int appaserver_element_tab_order(
    int tab_order );

char *appaserver_element_row_name(
    char *element_name,
    int row_number );

char *appaserver_element_list_html(
    LIST *appaserver_element_list /* in/out */,
    char *environment_application_name(),
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_hidden_list_html(
    LIST *appaserver_element_list /* in/out */,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_background_color_html(
    char *background_color );

char *appaserver_element_tab_order_html(
    int tab_order );

char *appaserver_element_number_commas_string(
    char *attribute_name,
    char *value );

char *appaserver_element_key_string(
    LIST *attribute_name_list );

char *appaserver_element_value(
    char *appaserver_element_key_string(),
    DICTIONARY *row_dictionary );

/* Usage */
APPASERVER_ELEMENT *
    appaserver_element_operation_table_heading(
        char *operation->
            process->
            process_name,
        char *operation->
            delete_warning_javascript );

APPASERVER_ELEMENT *
    appaserver_element_table_heading(
        char *heading_name );
```

table_open
table_heading
table_row
table_close
checkbox
prompt_drop_down
drop_down

button
non_edit_text
hidden
line_break
table_data
multi_drop_down
notepad

text
password
upload
yes_no

radio
prompt_data

reference_number
javascript_filename
element_date
element_current_date
element_time
element_current_time
element_date_time

element_current_date_time
timestamp
empty_column

APPASERVER_ELEMENT (2)

```
/* Public */
boolean appaserver_element_recall_boolean(
    APPASERVER_ELEMENT *appaserver_element );

/* Private */
char *appaserver_element_heading_name(
    char *appaserver_element->name );

char *appaserver_element_html(
    APPASERVER_ELEMENT *appaserver_element /* in/out */,
    char *application_name,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_prompt_drop_down_html(
    ELEMENT_PROMPT_DROP_DOWN *prompt_drop_down /* in/out */,
    char *state );

char *appaserver_element_drop_down_html(
    ELEMENT_DROP_DOWN *drop_down /* in/out */,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_text_html(
    ELEMENT_TEXT *text /* in/out */,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_hidden_html(
    ELEMENT_HIDDEN *hidden /* in/out */,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_notepad_html(
    ELEMENT_NOTEPAD *notepad /* in/out */,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_checkbox_html(
    ELEMENT_CHECKBOX *checkbox /* in/out */,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *appaserver_element_upload_update_html(
    ELEMENT_UPLOAD *upload,
    char *background_color,
    DICTIONARY *row_dictionary,
    char *application_name );

char *appaserver_element_yes_no_html(
    ELEMENT_YES_NO *yes_no /* in/out */,
    char *background_color,
    char *state,
    int row_number,
    DICTIONARY *row_dictionary );

char *element_list_html(
    LIST *appaserver_element_list );
```

ELEMENT_MULTI_DROP_DOWN

```
/* Usage */
ELEMENT_MULTI_DROP_DOWN *
    element_multi_drop_down_new(
        char *original_name,
        char *element_name,
        LIST *delimited_list,
        boolean no_initial_capital );

/* Process */
APPASERVER_ELEMENT *original_drop_down =
    appaserver_element_new(
        prompt_drop_down,
        original_name );

ELEMENT_PROMPT_DROP_DOWN *prompt_drop_down =
    element_prompt_drop_down_new(
        original_drop_down->element_name,
        delimited_list,
        no_initial_capital,
        0 /* not output_null_option */,
        0 /* not output_not_null_option */,
        0 /* not output_select_option */,
        element_multi_display_size(),
        0 /* tab_order */,
        1 /* multi_select */,
        (char *0 /* post_change_javascript */,
        1 /* recall */ );

ELEMENT_TABLE_DATA *table_data =
    element_table_data_new(
        0 /* not align_right,
        1 /* column_span */ );

ELEMENT_BUTTON *move_right_button =
    element_button_new(
        element_multi_drop_down_move_right_label(),
        element_multi_drop_down_move_right_action_string() );

ELEMENT_LINE_BREAK *element_line_break =
    element_line_break_calloc();

ELEMENT_BUTTON *move_left_button =
    element_button_new(
        element_multi_drop_down_move_left_label(),
        element_multi_drop_down_move_left_action_string() );

APPASERVER_ELEMENT *empty_drop_down =
    appaserver_element_new(
        prompt_drop_down,
        element_name );

ELEMENT_PROMPT_DROP_DOWN *prompt_drop_down =
    element_empty_prompt_drop_down_new(
        empty_drop_down->element_name,
        element_multi_display_size(),
        1 /* multi_select */,
        0 /* not readonly */,
        1 /* recall */ );

/* Public */
char *element_multi_drop_down_html(
    ELEMENT_PROMPT_DROP_DOWN *original_drop_down,
    ELEMENT_TABLE_DATA *table_data,
    ELEMENT_BUTTON *move_right_button,
    ELEMENT_LINE_BREAK *element_line_break,
    ELEMENT_BUTTON *move_left_button,
    ELEMENT_PROMPT_DROP_DOWN *empty_drop_down );

/* Private */
int element_multi_display_size( void );

char *element_multi_drop_down_move_right_label( void );

char *element_multi_drop_down_move_left_label( void );

char *element_multi_drop_down_move_right_action_string(
    char *original_name,
    char *element_name,
    char ELEMENT_MULTI_MOVE_LEFT_RIGHT_DELIMITER );

char *element_multi_drop_down_move_left_action_string(
    char *original_name,
    char *element_name,
    char ELEMENT_MULTI_MOVE_LEFT_RIGHT_DELIMITER );
```

ELEMENT_DROP_DOWN

```
/* Usage */
ELEMENT_DROP_DOWN *element_drop_down_new(
    LIST *attribute_name_list,
    LIST *delimited_list,
    boolean no_initial_capital,
    boolean output_null_option,
    boolean output_not_null_option,
    boolean output_select_option,
    int element_multi|drop_down_display_size(),
    int appaserver_element_tab_order(),
    boolean multi_select,
    char *post_change_javascript,
    boolean readonly,
    boolean recall );

/* Public */
char *value =
    appaserver_element_value(
        appaserver_element_key_string(
            attribute_name_list ),
        row_dictionary );

char *element_drop_down_name(
    LIST *attribute_name_list,
    int row_number );

int element_drop_down_display_size(
    int list_length( delimited_list ) );

char *element_drop_down_html(
    char *element_drop_down_name(
            attribute_name_list,
            row_number ),
    char *value,
    LIST *delimited_list,
    LIST *element_drop_down_display_list(
            delimited_list,
            no_initial_capital ),
    boolean output_null_option,
    boolean output_not_null_option,
    boolean output_select_option,
    int element_drop_down_display_size(),
    int tab_order,
    char *javascript_replace(
        post_change_javascript,
        state,
        row_number ),
    char *background_color,
    int application->max_drop_down_size,
    int application->max_query_rows_for_drop_downs,
    int row_dictionary_list_length );

/* Private */
ELEMENT_DROP_DOWN *element_drop_down_calloc(
    void );

LIST *element_drop_down_display_list(
    LIST *delimited_list,
    boolean no_initial_capital );

char *element_drop_down_value_display(
    char *value,
    boolean no_initial_capital );

char *element_drop_down_option_value_html(
    char *value,
    char *element_drop_down_value_display() );

char *first_option_value_html =
    element_drop_down_option_value_html(
        char *element_drop_down_value(),
        char *list_offset_seek(
            display_list,
            list_seek_offset(
                char *element_drop_down_value(),
                LIST *delimtied_list) )
                    /* value_display */ );

char *element_drop_down_option_value_list_html(
    LIST *delimited_list,
    LIST *display_list,
    char *element_drop_down_value() );

char *element_drop_down_close_html(
    boolean output_null_option,
    boolean output_not_null_option );
```

ELEMENT_PROMPT_DROP_DOWN

```
/* Usage */
ELEMENT_PROMPT_DROP_DOWN *
    element_prompt_drop_down_new(
        char *element_name,
        LIST *delimited_list,
        boolean no_initial_capital,
        boolean output_null_option,
        boolean output_not_null_option,
        boolean output_select_option,
        int display_size,
        int appaserver_element_tab_order(),
        boolean multi_select,
        char *post_change_javascript,
        boolean recall );

ELEMENT_PROMPT_DROP_DOWN *
    element_empty_prompt_drop_down_new(
        char *element_name,
        int element_multi_display_size(),
        boolean multi_select,
        boolean recall );

/* Public */
char *element_prompt_drop_down_html(
    char *element_name,
    LIST *delimited_list,
    LIST *element_drop_down_display_list(
            delimited_list,
            no_initial_capital ),
    boolean output_null_option,
    boolean output_not_null_option,
    boolean output_select_option,
    int (display_size)
        ? display_size
        ? element_drop_down_display_size(),
    boolean multi_select,
    int tab_order,
    char *javascript_replace() );

char *element_empty_prompt_drop_down_html(
    char *element_name,
    int display_size,
    boolean multi_select );

/* Private */
ELEMENT_PROMPT_DROP_DOWN *
    element_prompt_drop_down_calloc(
        void );
```

ELEMENT_TEXT

```
/* Usage */
ELEMENT_TEXT *element_text_new(
    char *attribute_name,
    char *datatype_name,
    int attribute_width_max_length,
    boolean null_to_slash,
    boolean prevent_carrot,
    char *on_change,
    char *on_focus,
    char *on_keyup,
    int tab_order,
    boolean recall );

/* Public */
char *element_text_value(
    char *attribute_name,
    DICTIONARY *row_dictionary,
    boolean attribute_is_number(
        char *datatype_name ) );

char *element_text_javascript_replace_on_change(
    char *on_change,
    int row_number,
    char *state,
    boolean null_to_slash );

char *element_text_javascript_replace_on_focus(
    char *on_focus,
    int row_number,
    char *state );

char *element_text_prevent_carrot_on_keyup(
    char *on_keyup,
    boolean prevent_carrot );

boolean element_text_autocomplete_off(
    char *attribute_name );

char *element_text_html(
    char *appaserver_element_name(
        attribute_name,
        row_number )
            /* element_name */,
    char *(value)
            ? value
            : element_text_value(),
    int attribute_width_max_length,
    int ELEMENT_TEXT_MAX_DISPLAY_SIZE,
    element_text_javascript_replace_on_change(),
    element_text_javascript_replace_on_focus(),
    element_text_prevent_carrot_on_keyup(),
    boolean element_text_autocomplete_off(),
    int tab_order,
    char *background_color );

/* Private */
ELEMENT_TEXT *element_text_calloc(
    void );
```

ELEMENT_NON_EDIT_TEXT

```
/* Usage */
ELEMENT_NON_EDIT_TEXT *
    element_non_edit_text_new(
        /* Mutually exclusive */
        char *attribute_name,
        char *message );

/* Public */
char *element_non_edit_text_initial_capital(
    char *appaserver_element_value(
            attribute_name,
            row_dictionary ),
    char *message );

/* Private */
ELEMENT_NON_EDIT_TEXT *
    element_non_edit_text_calloc(
        void );
```

ELEMENT_PASSWORD

```
/* Usage */
ELEMENT_PASSWORD *element_password_calloc(
    void );

ELEMENT_PASSWORD *element_password_new(
    char *attribute_name,
    int attribute_width_max_length,
    int ELEMENT_TEXT_MAX_DISPLAY_SIZE,
    int tab_order );

/* Public */
char *element_password_compare_element_name(
    char *attribute_name,
    int row_number );

char *element_password_html(
    appaserver_element_name(
        attribute_name,
        row_number )
            /* element_name */,
    element_password_compare_element_name(),
    appaserver_element_value(
            attribute_name,
            row_dictionary ),
    int attribute_width_max_length,
    int ELEMENT_TEXT_MAX_DISPLAY_SIZE,
    int tab_order );
```

ELEMENT_CHECKBOX

```
/* Usage */
ELEMENT_CHECKBOX *element_checkbox_new(
    char *attribute_name /* mutually exclusive */,
    char *element_name /* mutually exclusive */,
    char *prompt_string,
    char *on_click,
    int tab_order,
    char *image_source,
    boolean recall );

/* Public */
boolean element_checkbox_checked(
    char *attribute_name,
    DICTIONARY *row_dictionary );

char *element_checkbox_html(
    (element_name)
        ? element_name
        : appaserver_element_name(
            attribute_name,
            row_number ),
    char *string_initial_capital(
        prompt_display,
        prompt_string ),
    boolean element_checkbox_checked(),
    javascript_replace(
        char *on_click,
        char *state,
        int row_number ),
    int tab_order,
    char *background_color,
    char *image_source );

char *element_checkbox_submit_javascript(
    char *on_click_javascript,
    char *form_name );

/* Private */
ELEMENT_CHECKBOX *element_checkbox_calloc(
    void );
```

ELEMENT_NOTEPAD

```
/* Usage */
ELEMENT_NOTEPAD *element_notepad_new(
    char *attribute_name,
    int attribute_size,
    int ELEMENT_NOTEPAD_COLUMNS,
    int ELEMENT_NOTEPAD_ROWS,
    boolean null_to_slash,
    char *post_change_javascript,
    int tab_order );

/* Public */
char *element_notepad_html(
    char *appaserver_element_name(
        attribute_name,
        row_number ),
    char *appaserver_element_value(
        attribute_name,
        row_dictionary ),
    int attribute_size,
    int columns,
    int rows,
    boolean null_to_slash,
    char *javascript_replace(
        char *post_change_javascript,
        char *state,
        int row_number ),
    int tab_order,
    char *background_color );

/* Private */
ELEMENT_NOTEPAD *element_notepad_calloc(
    void );

char *element_notepad_onchange(
    boolean null_to_slash,
    char *javascript_replace );
```

ELEMENT_HIDDEN

```
/* Usage */
ELEMENT_HIDDEN *element_hidden_calloc(
        void );

ELEMENT_HIDDEN *element_hidden_new(
    /* Mutually exclusive */
    char *attribute_name,
    LIST *attribute_name_list );

/* Public */
char *element_hidden_key_string(
    char *attribute_name,
    LIST *attribute_name_list );

char *element_hidden_name(
    element_hidden_key_string(),
    int row_number );

char *element_hidden_html(
    char *element_hidden_name(),
    char *appaserver_element_value(
        element_hidden_key_string(),
        row_dictionary ) );
```

ELEMENT_UPLOAD

```
/* Usage */
ELEMENT_UPLOAD *element_upload_new(
    char *attribute_name,
    int tab_order,
    boolean recall );

/* Public */
char *element_upload_insert_html(
    char *appaserver_element_name(
        attribute_name,
        row_number )
            /* element_name */,
    int tab_order );

char *element_upload_update_html(
    char *appaserver_element_value(
        attribute_name,
        row_dictionary ),
    char *background_color,
    char *application_name );

/* Private */
ELEMENT_UPLOAD *element_upload_calloc(
    void );

char *element_upload_update_hypertext_reference(
    char *application_name,
    char *value );

void element_upload_update_link(
    char *application_name,
    char *value,
    char *appaserver_parameter_upload_directory(),
    char *appaserver_parameter_document_root() );
```

ELEMENT_YES_NO

```
/* Usage */
ELEMENT_YES_NO *element_yes_no_new(
    char *attribute_name /* mutually exclusive */,
    char *element_name /* mutually exclusive */,
    boolean output_null_option,
    boolean output_not_null_option,
    char *post_change_javascript,
    int tab_order,
    boolean recall );

/* Public */
boolean element_yes_no_affirmative(
    char *attribute_name,
    DICTIONARY *row_dictionary );

char *element_yes_no_html(
    (element_name)
        ? element_name
        : appaserver_element_name(
            attribute_name,
            row_number ),
    boolean element_yes_no_affirmative(),
    boolean output_null_option,
    boolean output_not_null_option,
    javascript_replace(
        char *post_change_javascript,
        char *state,
        int row_number ),
    int tab_order,
    char *background_color );

/* Private */
ELEMENT_YES_NO *element_yes_no_calloc(
    void );
```

ELEMENT_TABLE_OPEN

```
/* Usage */
ELEMENT_TABLE_OPEN *element_table_open_calloc( void );

/* Public */
char *element_table_open_html( void );
```

ELEMENT_TABLE_HEADING

```
/* Usage */
ELEMENT_TABLE_HEADING *element_table_heading_new(
    char *heading_string );

/* Public */
ELEMENT_TABLE_HEADING *element_table_heading_calloc( void );

char *element_table_row_html( char *heading_string );
```

ELEMENT_TABLE_ROW

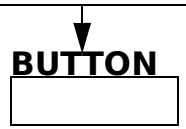
```
/* Usage */
ELEMENT_TABLE_ROW *element_table_row_calloc( void );

/* Public */
char *element_table_row_html( char *background_color );
```

ELEMENT_TABLE_DATA

```
/* Usage */
ELEMENT_TABLE_DATA *element_table_data_calloc( void );

ELEMENT_TABLE_DATA *element_table_data_new(
    boolean align_right,
    int column_span );

/* Public */
char *element_table_data_html(
    boolean align_right,
    int column_span );
```

ELEMENT_BUTTON

```
/* Usage */
ELEMENT_BUTTON *element_button_new(
    char *label,
    char *action_string );

/* Process */
BUTTON *button =
    button_new(
        label,
        action_string );

/* Public */
char *element_button_html(
    char *button->html );
```



ELEMENT_LINE_BREAK

```
/* Usage */
ELEMENT_LINE_BREAK *element_line_break_calloc(
    void );

/* Public */
char *element_line_break_html( void );
```

ELEMENT_TABLE_CLOSE

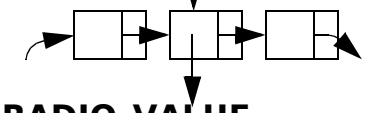
```
/* Usage */
ELEMENT_TABLE_CLOSE *element_table_close_calloc( void );

/* Public */
char *element_table_close_html( void );
```

BUTTON

```
/* Usage */
BUTTON *button_new( char *label, char *action_string );

/* Process */
char *button_html( char *label, char *action_string );

/* Usage */
BUTTON *button_submit(
    char *form_multi_select_all_javascript(),
    char *form_keystrokes_save_javascript(),
    char *form_keystrokes_multi_save_javascript(),
    char *form_verify_notepad_widths_javascript(),
    int form_number );

/* Process */
char *button_submit_html(
    char *form_post_change_multi_select_all_javascript(),
    char *form_keystrokes_save_javascript(),
    char *form_keystrokes_multi_save_javascript(),
    char *form_verify_notepad_widths_javascript(),
    int form_number );

/* Usage */
BUTTON *button_reset(
    char *javascript_replace(),
    int form_number );

/* Process */
char *button_reset_html(
    char *javascript_replace(),
    int form_number );

/* Usage */
BUTTON *button_back( void );

/* Process */
char *button_back_html( void );

/* Usage */
BUTTON *button_forward( void );

/* Process */
char *button_forward_html( void );

/* Usage */
BUTTON *button_back_to_drillthru( char *action_string );

/* Process */
char *button_back_to_drillthru_html( char *action_string );

/* Usage */
BUTTON *button_back_to_top( void );

/* Process */
char *button_back_to_top_html( void );

/* Usage */
BUTTON *button_recall(
    char *form_keystrokes_recall_javascript(),
    char *form_keystrokes_multi_recall_javascript() );

/* Process */
char *button_recall_html(
    char *form_keystrokes_recall_javascript(),
    char *form_keystrokes_multi_recall_javascript() );

/* Usage */
BUTTON *button_drillthru_skip( void );

/* Process */
char *button_drillthru_skip_html( void );

/* Usage */
BUTTON *button_help(
    char *application_name,
    char *html_help_file_anchor );

/* Process */
char *button_help_html(
    char *application_name,
    char *html_help_file_anchor );

/* Public */
char *button_list_html(
    LIST *button_list );

void button_list_free(
    LIST *button_list );

LIST *button_insert_pair_one2m_submit_list(
    LIST *pair_one2m_folder_list );
```

RADIO_LIST

```
/* Usage */
RADIO_LIST *radio_list_new(
    char *FORM_RADIO_LIST_NAME,
    LIST *radio_pair_list,
    char *initial_label );

/* Process */
LIST *radio_value_list = list_new();

for radio_pair in radio_pair_list
{
    list_set(
        radio_value_list,
        radio_value_new(
            form_radio_list_name,
            radio_pair,
            initial_label ) );
}

char *radio_list_html(
    LIST *radio_value_list );
```



RADIO_VALUE

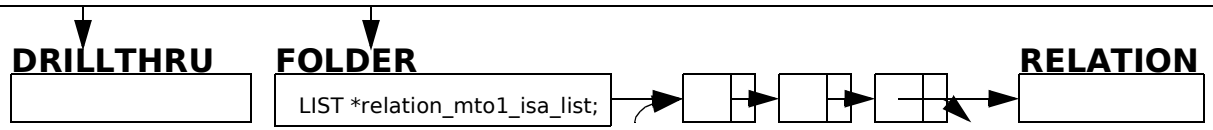
```
/* Usage */
RADIO_VALUE *radio_value_new(
    char *radio_list_name,
    RADIO_PAIR *radio_pair,
    char *initial_label );

/* Process */
char *radio_value_html(
    char *radio_list_name,
    char *radio_pair->radio_name,
    char *radio_pair->radio_label,
    char *initial_label );

/* Public */
void radio_value_free(
    RADIO_VALUE *radio_value );
```

RADIO_PAIR

```
/* Usage */
RADIO_PAIR *radio_pair_new(
    char *radio_name,
    char *radio_label );
```

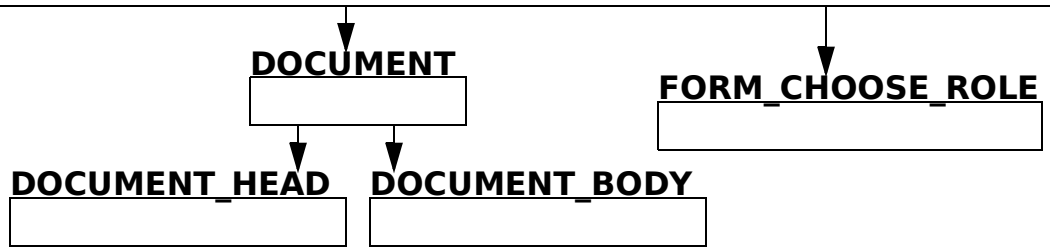
POST_CHOOSE_FOLDER

```
/* Usage */
POST_CHOOSE_FOLDER *
    post_choose_folder_new(
        /* See session_folder_integrity_exit() */
        char *application_name,
        char *login_name,
        char *session_key,
        char *role_name,
        char *folder_name,
        char *state );

/* Process */
DRILLTHRU *drillthru =
    drillthru_start(
        ( drillthru_dictionary =
            dictionary_small() ) /* out */,
        folder_name );

char *post_choose_folder_name(
    char *folder_name,
    char *drillthru_start_current_folder_name(
        drillthru->relation_mto1_drillthru_list ) );

boolean post_choose_folder_relation_isa_boolean(
    char *state,
    char *APPASERVER_INSERT_STATE );

FOLDER *folder =
    folder_fetch(
        post_choose_folder_name(),
        /* Not fetching role_folder_list */
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        0 /* not fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        post_choose_folder_relation_isa_boolean(),
        1 /* fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *folder->relation_pair_one2m_list(
        folder->relation_one2m_list );

char *post_choose_folder_form_name(
    list_length( folder->relation_pair_one2m_list )
        /* relation_pair_one2m_list_length */,
    folder->folder_form );

boolean post_choose_folder_isa_drop_down(
    list_length( folder->relation_mto1_isa_list )
        /* relation_mto1_isa_list_length */ );

boolean post_choose_folder_prompt_insert(
    post_choose_folder_form_name(),
    char *state );

boolean post_choose_folder_insert_table(
    post_choose_folder_form_name(),
    char *state );

boolean post_choose_folder_prompt_edit(
    post_choose_folder_form_name(),
    char *state,
    boolean drillthru->drillthru_participating );

boolean post_choose_folder_edit_table(
    post_choose_folder_form_name(),
    char *state );

char *post_choose_folder_target_frame(
    boolean post_choose_folder_insert_table(),
    boolean post_choose_folder_edit_table(),
    boolean drillthru->drillthru_participating,
    char *FRAMESET_PROMPT_FRAME,
    char *FRAMESET_EDIT_FRAME );

char *post_choose_folder_system_string(
    boolean post_choose_folder_isa_drop_down(),
    boolean post_choose_folder_prompt_insert(),
    boolean post_choose_folder_insert_table(),
    boolean post_choose_folder_prompt_edit()
    boolean post_choose_folder_edit_table(),
    application_name,
    login_name,
    session_key,
    char *folder->folder_name,
    role_name,
    post_choose_folder_target_frame(),
    state,
    drillthru_dictionary,
    list_first( folder->relation_mto1_isa_list )
        /* RELATION *first_one2m_isa_relation */ );
```



CHOOSE_ROLE

```
/* Usage */
CHOOSE_ROLE *choose_role_default_new(
    char *login_name );

/* Process */
char *choose_role_default_where(
    char *login_name );

char *choose_role_default_system_string(
    char *choose_role_default_where() );

char *choose_role_default_role_name(
    char *choose_role_default_system_string() );

/* Usage */
CHOOSE_ROLE *choose_role_new(
    char *application_name,
    char *login_name,
    char *session_key,
    boolean frameset_menu_horizontal() );

/* Process */
char *choose_role_title_string(
    char *login_name );

char *choose_role_post_action_string(
    char *application_name,
    char *login_name,
    char *session_key );

char *choose_role_target_frame(
    boolean frameset_menu_horizontal() );

char *choose_role_title_html(
    char *choose_role_title_string() );

DOCUMENT *document =
    document_new(
        application_name,
        choose_role_title_string(),
        choose_role_title_html(),
        (char *)0 /* subtitle_html */,
        (char *)0 /* subsubtitle_html */,
        (char *)0 /* javascript_replace */,
        0 /* menu_boolean */,
        (MENU *)0,
        (char *)0 /* menu_setup_string */,
        (char *)0 /* calendar_setup_string */,
        (char *)0 /* javascript_include_string */,
        (char *)0 /* input_onload_string */ );

FORM_CHOOSE_ROLE *form_choose_role =
    form_choose_role_new(
        appaserver_user_role_name_list( login_name ),
        choose_role_post_action_string(),
        choose_role_target_frame(),
        CHOOSE_ROLE_FORM_NAME,
        CHOOSE_ROLE_DROP_DOWN_ELEMENT_NAME );

char *document_form_html(
    document->html,
    document->document_head->html,
    document_head_close_html(),
    document->document_body->html,
    form_choose_role->html,
    document_body_close_html(),
    document_close_html() );

/* Public */
char *choose_role_output_system_string(
    char *CHOOSE_ROLE_OUTPUT_EXECUTABLE,
    char *session_key,
    char *login_name,
    boolean frameset_menu_horizontal(),
    char *output_filename );
```



CHOOSE_ROLE_POST

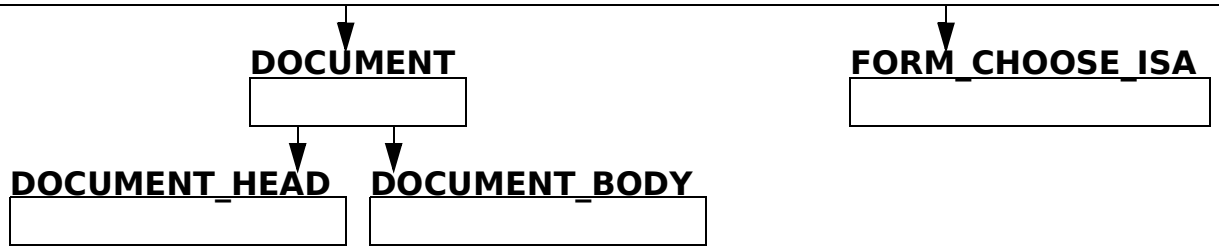
```
/* Usage */
CHOOSE_ROLE_POST *choose_role_post_new(
    char *sql_injection_escape_application_name,
    char *sql_injection_escape_session_key,
    char *sql_injection_escape_login_name );

/* Process */
```

CHOOSE_ISA

```
/* Usage */
CHOOSE_ISA *choose_isa_new(
    /* See session_folder_integrity_exit() */
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *one_isa_folder_name,
    boolean menu_boolean(
        FRAMESET_PROMPT_FRAME,
        frameset_menu_horizontal(
            application_name,
            login_name ) ) );

/* Process */
FOLDER *folder =
    folder_fetch(
        one_isa_folder_name,
        /* Not fetching role_folder_list */
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        /* Also sets folder_attribute_primary_list */
        /* and primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_attribute_exclude_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

LIST *delimited_list;

if ( folder->populate_drop_down_process )
{
    delimited_list =
        list_pipe_fetch(
            process_choose_isa_command_line(
                folder->
                    populate_drop_down_process->
                    command_line,
                security_entity_where(
                    security_entity,
                    folder->folder_attribute_list ),
                application_name,
                login_name,
                role_name,
                one_isa_folder_name  ) );
}
else
{
    QUERY_ISA_WIDGET *query_isa_widget =
        query_isa_widget_new(
            one_isa_folder_name,
            folder->primary_key_list,
            folder->folder_attribute_primary_list,
            login_name,
            security_entity_where(
                security_entity,
                folder->folder_attribute_list ) );

    delimited_list = query_isa_widget->delimited_list;
}

char *choose_isa_title_string(
    char *folder_name );

char *choose_isa_title_html(
    char *choose_isa_title_string() );

char *choose_isa_subtitle_html(
    char *folder_name,
    char *one_isa_folder_name );

char *choose_isa_prompt_message(
    LIST *folder->primary_key_list );

if ( menu_boolean() )
{
    FOLDER_MENU *folder_menu =
        folder_menu_fetch(
            application_name,
            session_key,
            appaserver_parameter_data_directory(),
            role_name );

    MENU *menu =
        menu_fetch(
            application_name,
            login_name,
            session_key,
            role_name,
            FRAMESET_PROMPT_FRAME,
            folder_menu->lookup_count_list );
}

char *choose_isa_action_string(
    char *CHOOSE_ISA_POST_EXECUTABLE,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *one_isa_folder_name );
```

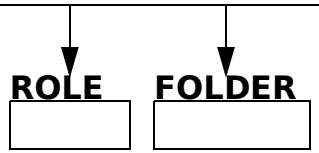
CHOOSE_ISA (2)

```
/* Process */
DOCUMENT *document =
    document_new(
        application_name,
        choose_isa_title_string(),
        choose_isa_title_html(),
        choose_isa_subtitle_html(),
        (char *)0 /* subsubtitle_html */,
        (char *)0 /* javascript_replace */,
        menu_boolean,
        menu,
        document_head_menu_setup_string( menu_boolean ),
        (char *)0 /* calendar_setup_string */,
        (char *)0 /* javascript_include_string */,
        (char *)0 /* input_onload_string */ );

FORM_CHOOSE_ISA *form_choose_isa =
    form_choose_isa_new(
        choose_isa_prompt_message(),
        folder->primary_key_list,
        delimited_list,
        folder->no_initial_capital,
        choose_isa_action_string() );

char *document_form_html(
    document->html,
    document->document_head->html,
    document_head_close_html(),
    document->document_body->html,
    form_choose_isa->html,
    document_body_close_html(),
    document_close_html() );

/* Public */
char *choose_isa_output_system_string(
    char *CHOOSE_ISA_OUTPUT_EXECUTABLE,
    char *session_key,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *one_isa_folder_name,
    char *appaserver_error_filename() );
```



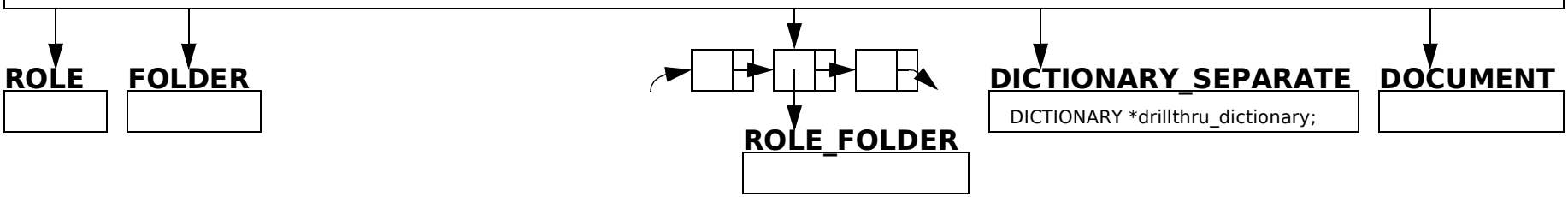
CHOOSE_ISA_POST

```
/* Usage */
CHOOSE_ISA_POST *choose_isa_post_new(
    /* See session_folder_integrity_exit() */
    char *application_name,
    char *session_key,
    char *login_name,
    char *folder_name,
    char *one_isa_folder_name,
    char *role_name );

/* Process */
ROLE *role =
    role_fetch(
        role_name );

FOLDER *folder =
    folder_fetch(
        one_isa_folder_name,
        /* Not fetching role_folder_list */
        (char *)0 /* role_name */,
        (LIST *)0 /* exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

char *choose_isa_post_system_string(
    char *application_name,
```



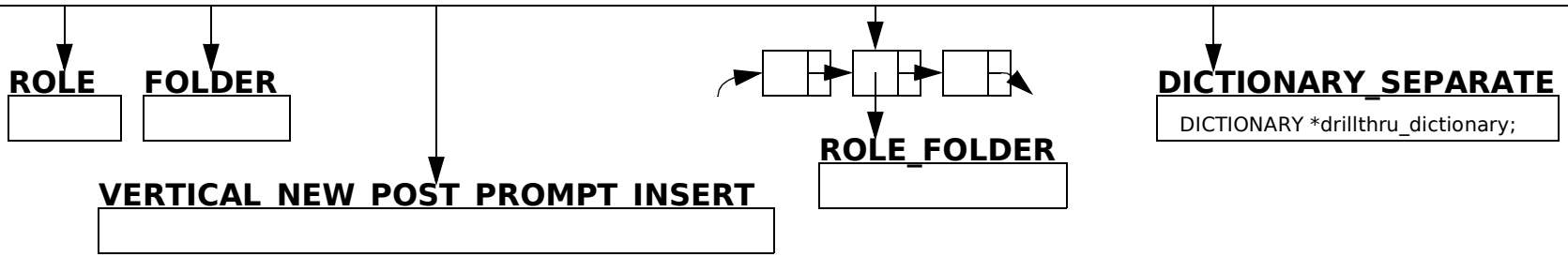
PROMPT_INSERT

```
/* Usage */
PROMPT_INSERT *
    prompt_insert_new(
        char *application_name,
        char *login_name,
        char *session_key,
        char *folder_name,
        char *role_name,
        char *state,
        char *target_frame,
        DICTIONARY_SEPARATE *dictionary_separate );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching_role_folder_list */,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

boolean prompt_insert_forbid(
    role_folder_insert(
        folder->role_folder_list ) );

/* Public */
char *prompt_insert_output_system_string(
    char *PROMPT_INSERT_OUTPUT_EXECUTABLE,
    char *login_name,
    char *sesion_key,
    char *folder_name,
    char *role_name,
    char *dictionary_separate_send_string(),
    char *appaserver_error_filename() );
```



PROMPT_INSERT_POST

```
/* Usage */
PROMPT_INSERT_POST *
    prompt_insert_post_new(
        char *application_name,
        char *login_name,
        char *session,
        char *folder_name,
        char *role_name,
        char *state,
        char *target_frame,
        DICTIONARY_SEPARATE *dictionary_separate );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

boolean prompt_insert_forbid(
    role_folder_insert(
        folder->role_folder_list ) );
```

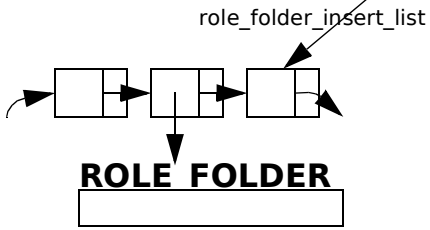


VERTICAL_NEW

```
/* Public */
void vertical_new_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *hidden_label,
    char *folder_name );

char *vertical_new_dictionary_folder_name(
    char *hidden_label,
    DICTIONARY *non_prefixed_dictionary );
```

VERTICAL_NEW_OUTPUT_PROMPT_INSERT

```
/* Public */
APPASERVER_ELEMENT *vertical_new_output_prompt_insert_element(
    char *one_folder_name,
    LIST *role_folder_insert_list,
    char *form_name,
    char *VERTICAL_NEW_ONE_PREFIX,
    char *VERTICAL_NEW_DISPLAY,
    char *element_checkbox_submit_javascript(
        on_click_recall_save_string,
        form_name ) );

/* Private */
char *vertical_new_output_prompt_insert_element_name(
    char *VERTICAL_NEW_ONE_PREFIX,
    char *one_folder_name );
```



APPASERVER_ELEMENT

VERTICAL_NEW_POST_PROMPT_INSERT

```
/* Usage */
VERTICAL_NEW_POST_PROMPT_INSERT *
    vertical_new_post_prompt_insert_new(
        DICTIONARY *non_prefixed_dictionary /* in/out */,
        char *many_folder_name,
        DICTIONARY *drillthru_dictionary,
        char *application_name,
        char *login_name,
        char *session_key,
        char *role_name,
        char *VERTICAL_NEW_ONE_PREFIX,
        char *VERTICAL_NEW_ONE_HIDDEN_LABEL,
        char *VERTICAL_NEW_MANY_HIDDEN_LABEL,
        char *FRAMESET_PROMPT_FRAME );

/* Process */
char *vertical_new_post_prompt_insert_one_folder_name(
    char *VERTICAL_NEW_ONE_PREFIX,
    DICTIONARY *non_prefixed_dictionary );

if ( !vertical_new_post_prompt_insert_one_folder_name() )
{
    return (VERTICAL_NEW_POST_PROMPT_INSERT *)0;
}

void vertical_new_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *VERTICAL_NEW_ONE_HIDDEN_LABEL,
    vertical_new_post_prompt_insert_one_folder_name() );

void vertical_new_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *VERTICAL_NEW_MANY_HIDDEN_LABEL,
    char *many_folder_name );

char *vertical_new_post_prompt_insert_system_string(
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *non_prefixed_dictionary,
    char *application_name,
    char *login_name,
    char *session_key,
    char *vertical_new_post_prompt_insert_one_folder_name(),
    char *role_name,
    char *FRAMESET_PROMPT_FRAME );
```

VERTICAL_NEW_OUTPUT_INSERT_TABLE

```
/* Usage */
VERTICAL_NEW_OUTPUT_INSERT_TABLE *
    vertical_new_output_insert_table_new(
        char *application_name,
        char *session_key,
        char *login_name,
        char *role_name,
        char *folder_name,
        char *document_root_directory,
        DICTIONARY *non_prefixed_dictionary,
        char *VERTICAL_NEW_ONE_HIDDEN_LABEL,
        char *FRAMESET_PROMPT_FRAME,
        char *appaserver_parameter->data_directory );

/* Process */
char *one_folder_name =
    vertical_new_dictionary_folder_name(
        VERTICAL_NEW_ONE_HIDDEN_LABEL,
        non_prefixed_dictionary );

if ( !one_folder_name )
{
    return (VERTICAL_NEW_OUTPUT_INSERT_TABLE *)0;
}

if ( strcmp( folder_name, one_folder_name ) != 0 )
{
    exit( 1 );
}

VERTICAL_NEW_BLANK_FILENAME *
    vertical_new_blank_filename =
        vertical_new_blank_filename_new(
            application_name,
            session_key,
            document_root_directory );

void vertical_new_output_insert_table_blank_prompt_frame(
    char *vertical_new_blank_filename->output_filename,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *FRAMESET_PROMPT_FRAME,
    char *data_directory );

char *vertical_new_output_insert_table_onload_control_string(
    char *vertical_new_blank_filename->prompt_filename,
    char *FRAMESET_PROMPT_FRAME );
```

VERTICAL_NEW_BLANK_FILENAME

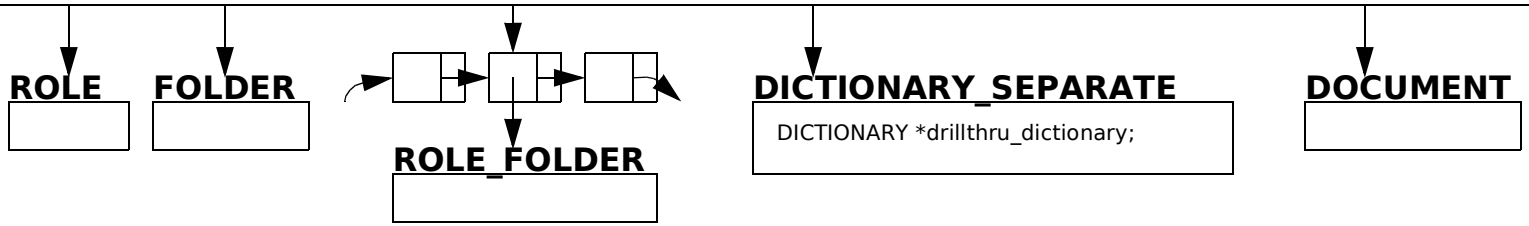
```
VERTICAL_NEW_BLANK_FILENAME *
    vertical_new_blank_filename_new(
        char *application_name,
        char *session_key,
        char *document_root_directory );

/* Process */
APPASERVER_LINK *appaserver_link =
    appaserver_link_new();

char *output_filename =
    appaserver_link->output->filename;

char *prompt_filename =
    appaserver_link->prompt->filename;
```

VERTICAL_NEW_POST_INSERT_TABLE

```
/* Usage */
VERTICAL_NEW_POST_INSERT_TABLE *vertical_new_post_insert_table_new(
```

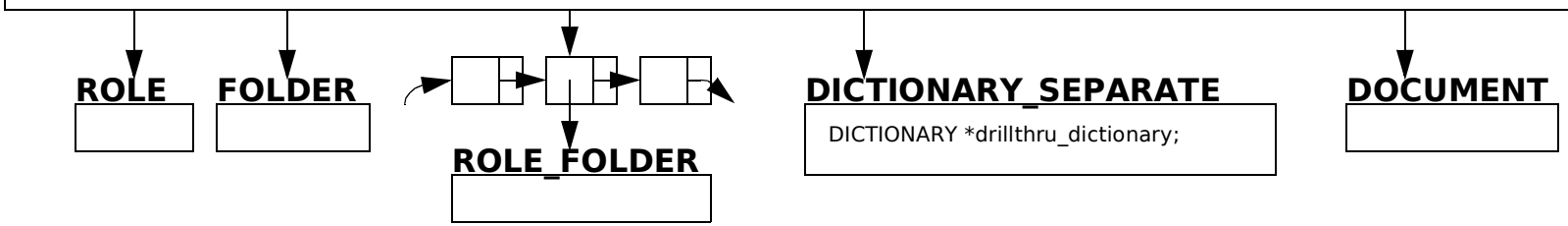
INSERT_TABLE

```
/* Usage */
INSERT_TABLE *insert_table_new(
    char *application_name,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *target_frame,
    char *message,
    POST_DICTIONARY *post_dictionary );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching_role_folder_list */,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

boolean insert_table_forbid(
    boolean role_folder_insert(
        folder->role_folder_list ) );

DICTIONARY_SEPARATE *dictionary_separate =
    dictionary_separate_folder_new(
        post_dictionary->original_post_dictionary,
        application_name,
        login_name,
        folder_attribute_date_name_list(
            folder->folder_attribute_append_isa_list ) );

/* Public */
char *insert_table_output_system_string(
    char *INSERT_TABLE_OUTPUT_EXECUTABLE,
    char *login_name,
    char *session_key,
    char *folder_name,
    char *role_name,
    char *dictionary_separate_send_string(),
    char *appaserver_error_filename() );
```



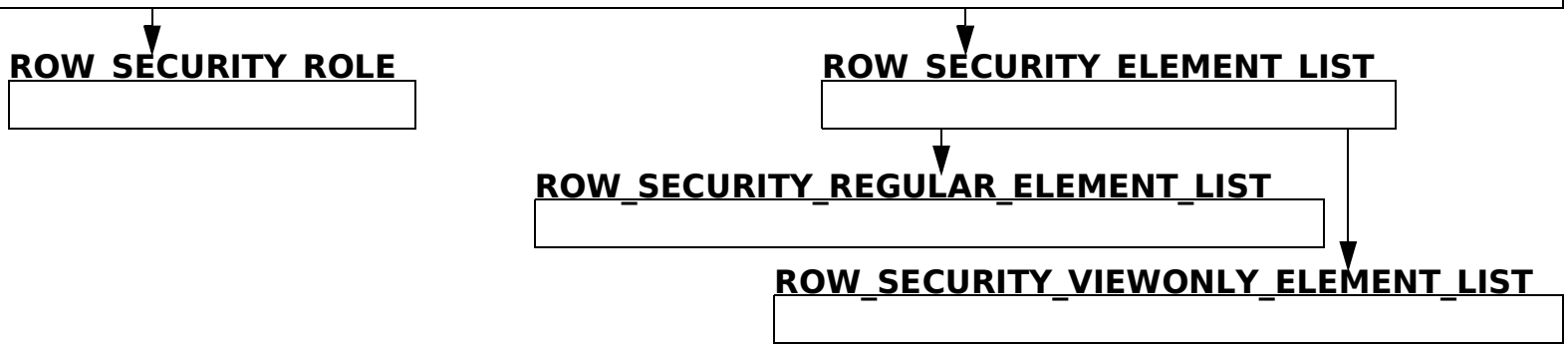
INSERT_TABLE_POST

```
/* Usage */
INSERT_TABLE_POST *
    insert_table_post_new(
        char *application_name,
        char *login_name,
        char *session_key,
        char *folder_name,
        char *role_name,
        char *target_frame,
        char *message,
        char *dictionary_string );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        role_exclude_insert_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        0 /* not fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

LIST *role_folder_list(
    role_name,
    folder_name );

boolean insert_table_forbid(
    role_folder_insert(
        role_folder_list ) );

DICTIONARY_SEPARATE *dictionary_separate =
    dictionary_separate_string_new(
        dictionary_string );
```



ROW_SECURITY

```
/* Usage */
ROW_SECURITY *row_security_new(
    char *folder_name,
    LIST *folder->folder_attribute_append_isa_list,
    LIST *folder->relation_mto1_non_isa_list,
    LIST *folder->relation_join_one2m_list,
    char *folder->post_change_javascript,
    DICTIONARY *drillthru_dictionary,
    boolean edit_table_primary_keys_non_edit(),
    LIST *folder->role_operation_list,
    LIST *ignore_select_attribute_name_list,
    char *edit_table_state(),
    LIST *role_exclude_update_attribute_name_list(),
    LIST *role_exclude_lookup_attribute_name_list(),
    boolean role->override_row_restrictions,
    char *login_name,
    char *security_entity_where() );

/* Process */
ROW_SECURITY_ROLE *row_security_role =
    /* Returns null if not participating */
    row_security_role_new(
        folder_name,
        folder_attribute_primary_key_list(
            folder_attribute_append_isa_list ),
            role_override_row_restrictions );

ROW_SECURITY_ELEMENT_LIST *
    row_security_element_list =
        row_security_element_list_new(
            folder_attribute_append_isa_list,
            relation_mto1_non_isa_list,
            relation_join_one2m_list,
            post_change_javascript,
            drillthru_dictionary,
            primary_keys_non_edit,
            role_operation_list,
            ignore_select_attribute_name_list,
            state,
            login_name,
            security_entity_where,
            exclude_update_attribute_name_list,
            exclude_lookup_attribute_name_list,
            row_security_role );

/* Private */
LIST *row_security_operation_element_list(
    LIST *role_operation_list,
    boolean viewonly );
```



ROW_SECURITY_DETAIL

```
/* Usage */
ROW_SECURITY_DETAIL *row_security_detail_new(
    application_name,
    where_attribute_data_list,
    folder,
    ignore_select_attribute_name_list );

/* Process */
```

ROW_SECURITY_ELEMENT_LIST

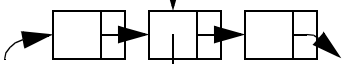
```
/* Usage */
ROW_SECURITY_ELEMENT_LIST *row_security_element_list_new(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_join_one2m_list,
    char *post_change_javascript,
    DICTIONARY *drillthru_dictionary,
    boolean primary_keys_non_edit,
    LIST *role_operation_list,
    LIST *ignore_select_attribute_name_list,
    char *state,
    char *login_name,
    char *security_entity_where(),
    LIST *role->exclude_update_attribute_name_list,
    LIST *role->exclude_lookup_attribute_name_list,
    /* Null if not participating */
    ROW_SECURITY_ROLE *row_security_role );

/* Process */
if ( strcmp( state, ELEMENT_UPDATE_STATE ) == 0 )
{
    ROW_SECURITY_ELEMENT_LIST_REGULAR *
        regular =
            row_security_element_list_regular_new(
                LIST *folder_attribute_append_isa_list,
                LIST *relation_mto1_non_isa_list,
                LIST *relation_join_one2m_list,
                char *post_change_javascript,
                DICTIONARY *drillthru_dictionary,
                boolean primary_keys_non_edit,
                LIST *role_operation_list,
                LIST *ignore_select_attribute_name_list,
                char *login_name,
                char *security_entity_where,
                LIST *role->exclude_update_attribute_name_list,
                LIST *role->exclude_lookup_attribute_name_list,
                ROW_SECURITY_ROLE *row_security_role );

    if ( row_security_role )
    {
        ROW_SECURITY_ELEMENT_LIST_VIEWONLY *
            viewonly =
                row_security_element_list_viewonly_new(
                    LIST *folder_attribute_append_isa_list,
                    LIST *relation_mto1_non_isa_list,
                    LIST *relation_join_one2m_list,
                    LIST *role_operation_list,
                    LIST *ignore_select_attribute_name_list,
                    char *security_entity_where,
                    LIST *role->exclude_lookup_attribute_name_list );
    }
}

if ( strcmp( state, ELEMENT_VIEWONLY_STATE ) == 0 )
{
        ROW_SECURITY_ELEMENT_LIST_VIEWONLY *
            viewonly =
                row_security_element_list_viewonly_new(
                    LIST *folder_attribute_append_isa_list,
                    (LIST *)0 /* relation_mto1_non_isa_list */,
                    LIST *relation_join_one2m_list,
                    LIST *role_operation_list,
                    LIST *ignore_select_attribute_name_list,
                    char *security_entity_where,
                    LIST *role->exclude_lookup_attribute_name_list );
}
```

ROW_SECURITY_ELEMENT_LIST_REGULAR

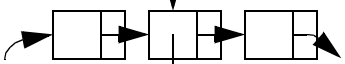
ROW_SECURITY_ELEMENT_LIST_VIEWONLY

ROW_SECURITY_ELEMENT_LIST_REGULAR

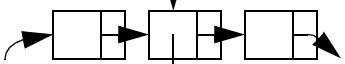
```
/* Usage */
ROW_SECURITY_ELEMENT_LIST_REGULAR *
    row_security_element_list_regular_new(
        LIST *folder_attribute_append_isa_list,
        LIST *relation_mto1_non_isa_list,
        LIST *relation_join_one2m_list,
        char *post_change_javascript,
        DICTIONARY *drillthru_dictionary,
        boolean primary_keys_non_edit,
        LIST *role_operation_list,
        LIST *ignore_select_attribute_name_list,
        char *login_name,
        char *security_entity_where(),
        LIST *exclude_update_attribute_name_list,
        LIST *exclude_lookup_attribute_name_list );

/* Process */
LIST *element_list =
    row_security_operation_element_list(
        role_operation_list,
        0 /* not viewonly */ );

LIST *attribute_name_list =
    folder_attribute_name_list(
        folder_attribute_append_isa_list );

for attribute_name in attribute_name_list
{
    if ( list_string_exists(
        attribute_name,
        ignore_select_attribute_name_list ) )
    {
        continue;
    }

    if ( list_string_exists(
        attribute_name,
        exclude_lookup_attribute_name_list ) )
    {
        continue;
    }

    FOLDER_ATTRIBUTE *folder_attribute =
        folder_attribute_seek(
            attribute_name,
            folder_attribute_append_isa_list );

    if ( primary_keys_non_edit
    && folder_attribute->primary_key_index )
    {
        goto skip_relation;
    }

    if ( list_string_exists(
        attribute_name,
        exclude_update_attribute_name_list ) )
    {
        goto skip_relation;
    }

    if ( ( row_security_relation =
            row_security_relation_new(
                attribute_name,
                relation_mto1_non_isa_list,
                post_change_javascript,
                drillthru_dictionary,
                login_name,
                security_entity_where,
                0 /* not viewonly */,
                row_security_relation_list ) ) )
    {
        if ( !row_security_relation_list )
        {
            LIST *row_security_relation_list = list_new();
        }

        list_set(
            row_security_relation_list,
            row_security_relation );

        list_set_list(
            element_list,
            row_security_relation->element_list );

        continue;
    }

skip_relation:

    if ( ( row_security_attribute =
            row_security_attribute_new(
                attribute_name,
                primary_keys_non_edit,
                folder_attribute->primary_key_index,
                folder_attribute->
                    attribute->
                    datatype_name,
                folder_attribute->
                    attribute->
                    width,
                exclude_update_attribute_name_list,
                post_change_javascript,
                row_security_relation_list ) )
    {
        list_set(
            row_security_attribute_list,
            row_security_attribute );

        list_set_list(
            element_list,
            row_security_attribute->element_list );
    }
}
```



ROW_SECURITY_RELATION



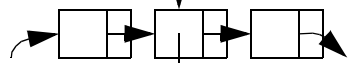
ROW_SECURITY_ATTRIBUTE



APPASERVER_ELEMENT

ROW_SECURITY_RELATION

```
/* Usage */
ROW_SECURITY_RELATION *
    row_security_relation_new(
        char *attribute_name,
        LIST *relation_mto1_non_isa_list,
        char *post_change_javascript,
        DICTIONARY *drillthru_dictionary,
        char *login_name,
        char *security_entity_where,
        boolean viewonly,
        LIST *row_security_relation_list );

/* Process */
if ( row_security_relation_attribute_name_exists(
    char *attribute_name,
    LIST *row_security_relation_list ) )
{
    return (ROW_SECURITY_RELATION *)0;
}

RELATION *relation =
    relation_consumes(
        attribute_name,
        relation_mto1_non_isa_list );

if ( !relation ) return (ROW_SECURITY_RELATION *)0;

LIST *row_security_relation_attribute_name_list(
    relation->foreign_key_list );

APPASERVER_ELEMENT *
    table_data_appaserver_element =
        appaserver_element_new(
            table_data,
            (char *)0 /* element_name */ );

char *row_security_relation_element_name(
    char *relation->one_folder->folder_name,
    char *related_attribute_name,
    LIST *foreign_key_list );

if ( viewonly )
{
    APPASERVER_ELEMENT *
        non_edit_text_appaserver_element =
            appaserver_element_new(
                non_edit_text,
                row_security_relation_element_name() );

    goto element_list_set;
}
else
{
    APPASERVER_ELEMENT *
        drop_down_appaserver_element =
            appaserver_element_new(
                drop_down,
                row_security_relation_element_name() );
}

QUERY_WIDGET *query_widget =
    query_widget_new(
        relation->one_folder->folder_name
            /* widget_folder_name */,
        login_name,
        relation->
            one_folder->
            folder_attribute_list(),
        relation->
            one_folder->
            relation_mto1_non_isa_list,
        security_entity_where,
        drillthru_dictionary );

ELEMENT_DROP_DOWN *
    drop_down_appaserver_element =
        element_drop_down_new(
            row_security_relation_element_name(),
            (LIST *)0 /* attribute_name_list */,
            query_widget->delimited_list,
            (LIST *)0 /* display_list */,
            relation->
                one_folder->
                no_initial_capital,
            1 /* output_null_option */,
            1 /* output_not_null_option */,
            1 /* output_select_option */,
            element_drop_down_display_size(),
            -1 /* tab_order */,
            0 /* not multi_select */,
            post_change_javascript,
            0 /* not readonly */,
            0 /* not recall */ );

element_list_set:

list_set(
    element_list,
    table_data_appaserver_element );

if ( viewonly )
{
    list_set(
        element_list,
        non_edit_text_appaserver_element );
}
else
{
    list_set(
        element_list,
        drop_down_appaserver_element );
}
```

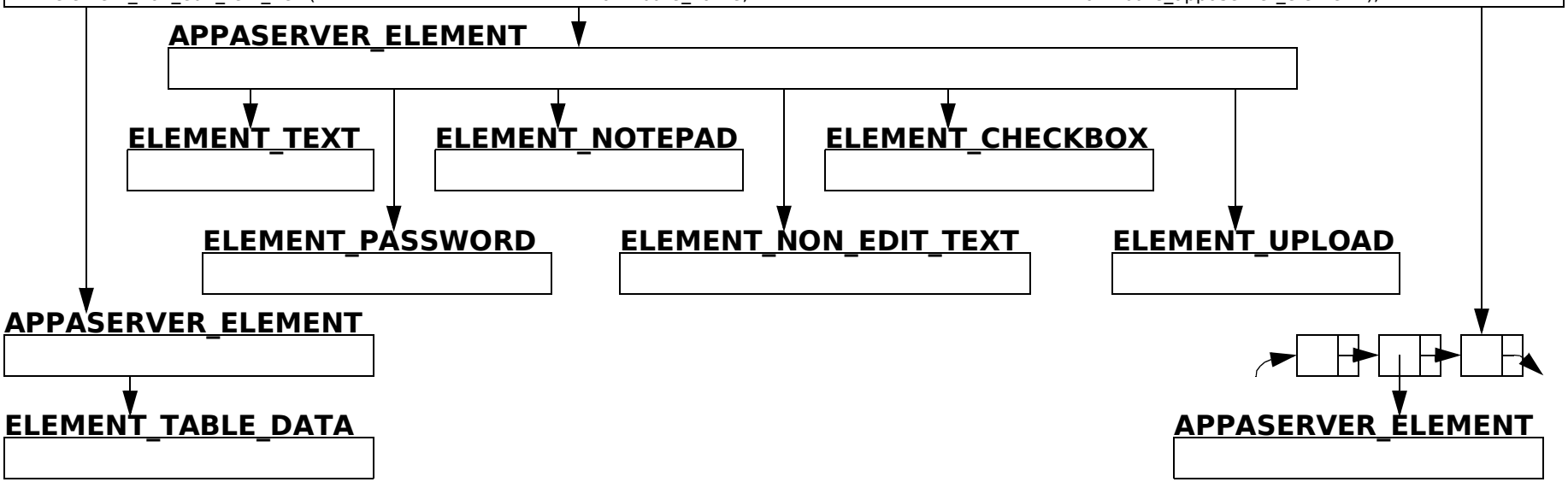


QUERY_WIDGET **APPASERVER_ELEMENT** **APPASERVER_ELEMENT** **APPASERVER_ELEMENT**

ELEMENT_TABLE_DATA **ELEMENT_DROP_DOWN** **ELEMENT_NON_EDIT_TEXT**

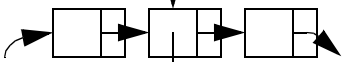
ROW_SECURITY_ATTRIBUTE

```
/* Usage */
ROW_SECURITY_ATTRIBUTE *
    row_security_attribute_new(
        char *attribute_name,
        boolean primary_keys_non_edit,
        int primary_key_index,
        char *datatype_name,
        int attribute_width,
        LIST *exclude_update_attribute_name_list,
        char *post_change_javascript,
        LIST *row_security_relation_list );

/* Process */
if ( row_security_relation_attribute_name_exists(
    attribute_name,
    row_security_relation_list ) )
{
    return (ROW_SECURITY_ATTRIBUTE *)0;
}

/* Process */
APPASERVER_ELEMENT *
    table_data_appaserver_element =
        appaserver_element_new(
            table_data,
            (char *)0 /* element_name */ );

ELEMENT_NON_EDIT_TEXT *non_edit_text =
    element_non_edit_text_new(
        attribute_name,
        (char *)0 /* message */ );

ELEMENT_TEXT *text =
    element_text_new(
        attribute_name,
        datatype_name,
        attribute_width /* attribute_width_max_length */,
        1 /* null_to_slash */,
        1 /* prevent_carrot */,
        post_change_javascript /* on_change */,
        (char *)0 /* on_focus */,
        (char *)0 /* on_keyup */,
        -1 /* tab_order*/,
        0 /* not recall */ );

ELEMENT_NOTEPAD *notepad =
    element_notepad_new(
        attribute_name,
        attribute_width /* attribute_size */,
        ELEMENT_NOTEPAD_COLUMNS,
        ELEMENT_NOTEPAD_ROWS,
        1 /* null_to_slash */,
        post_change_javascript,
        -1 /* tab_order */ );

ELEMENT_CHECKBOX *checkbox =
    element_checkbox_new(
        attribute_name,
        (char *)0 /* element_name */,
        (char *)0 /* prompt_string */,
        post_change_javascript /* on_click */,
        -1 /* tab_order */,
        (char *)0 /* image_source */,
        0 /* not recall */ );

ELEMENT_PASSWORD *password =
    element_password_new(
        attribute_name,
        attribute_width /* attribute_width_max_length */,
        ELEMENT_TEXT_MAX_DISPLAY_SIZE,
        -1 /* tab_order */ );

ELEMENT_UPLOAD *upload =
    element_upload_new(
        attribute_name,
        -1 /* tab_order */,
        0 /* not recall */ );

LIST *element_list = list_new();

list_set(
    element_list,
    table_data_appaserver_element );

list_set(
    element_list,
    attribute_appaserver_element );
```

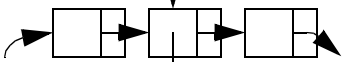


ROW_SECURITY_ELEMENT_LIST_VIEWONLY

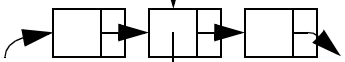
```
/* Usage */
ROW_SECURITY_ELEMENT_LIST_VIEWONLY *
    row_security_element_list_viewonly_new(
        LIST *folder_attribute_append_isa_list,
        LIST *relation_mto1_non_isa_list,
        LIST *relation_join_one2m_list,
        LIST *role_operation_list,
        LIST *ignore_select_attribute_name_list,
        LIST *exclude_lookup_attribute_name_list );

/* Process */
LIST *row_security_relation_list = list_new();
LIST *exclude_update_attribute_name_list = list_new();

LIST *element_list =
    row_security_operation_element_list(
        role_operation_list,
        1 /* viewonly */ );

LIST *attribute_name_list =
    folder_attribute_name_list(
        folder_attribute_append_isa_list );

for attribute_name in name_list
{
    if ( list_string_exists(
        attribute_name,
        ignore_select_attribute_name_list ) )
    {
        continue;
    }

    if ( list_string_exists(
        attribute_name,
        exclude_lookup_attribute_name_list ) )
    {
        continue;
    }

    FOLDER_ATTRIBUTE *folder_attribute =
        folder_attribute_seek(
            attribute_name,
            folder_attribute_append_isa_list );

    if ( ( row_security_relation =
            row_security_relation_new(
                attribute_name,
                relation_mto1_non_isa_list,
                post_change_javascript,
                drillthru_dictionary,
                login_name,
                security_entity_where,
                1 /* viewonly */,
                row_security_relation_list ) ) )
    {
        list_set(
            row_security_relation_list,
            row_security_relation );

        list_set_list(
            element_list,
            row_security_relation->element_list );
    }

    list_set( exclude_update_attribute_name_list,
            attribute_name );

    if ( ( row_security_attribute =
            row_security_attribute_new(
                attribute_name,
                primary_keys_non_edit,
                folder_attribute->primary_key_index,
                folder_attribute->
                    attribute->
                    datatype_name,
                folder_attribute->
                    attribute->
                    width,
                exclude_update_attribute_name_list,
                post_change_javascript,
                row_security_relation_list ) )
    {
        list_set(
            row_security_attribute_list,
            row_security_attribute );

        list_set_list(
            element_list,
            row_security_attribute->element_list );
    }
}
```



ROW_SECURITY_RELATION



ROW_SECURITY_ATTRIBUTE

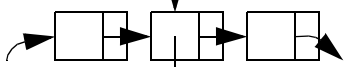


APPASERVER_ELEMENT

ROW_SECURITY_ROLE

```
/* Usage */
ROW_SECURITY_ROLE *row_security_role_new(
    char *folder_name,
    LIST *primary_key_list,
    boolean role_override_row_restrictions );

/* Attributes */
char *folder_name;
boolean role_override_row_restrictions;

/* Process */
if ( role_override_row_restrictions )
{
    return (ROW_SECURITY_ROLE *)0;
}

if ( !list_length( row_security_role_update_list() )
{
    return (ROW_SECURITY_ROLE *)0;
}

char *row_security_role_root_folder_name(
    char *folder_name,
    LIST *row_security_role_update_list() );

RELATION *row_security_role_relation(
    char *folder_name,
    LIST *row_security_role_update_list() );

boolean row_security_role_participating(
    char *row_security_role_root_folder_name(),
    RELATION *row_security_role_relation() );

if ( !row_security_role_participating() )
{
    return (ROW_SECURITY_ROLE *)0;
}

char *attribute_not_null =
    row_security_role_update_attribute_not_null(
        row_security_role_root_folder_name(),
        row_security_role_relation(),
        row_security_role_update_list() );

char *row_security_role_join(
    char *folder_name,
    LIST *primary_key_list,
    RELATION *row_security_role_relation() );

/* Public */
LIST *row_security_role_apply_element_list(
    LIST *regular_element_list,
    LIST *viewonly_element_list,
    DICTIONARY *row_dictionary,
    ROW_SECURITY_ROLE *row_security_role );

/* Private */
boolean row_security_role_viewonly(
    DICTIONARY *row_dictionary,
    row_security_role_update_attribute_not_null() );
```

row_security_role_update_list()



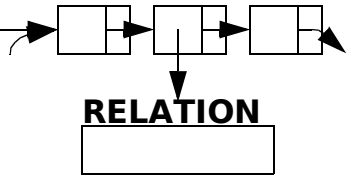
row_security_role_relation()

RELATION

```
FOLDER *many_folder;
LIST *foreign_key_list;
```

ROW_SECURITY_ROLE_UPDATE

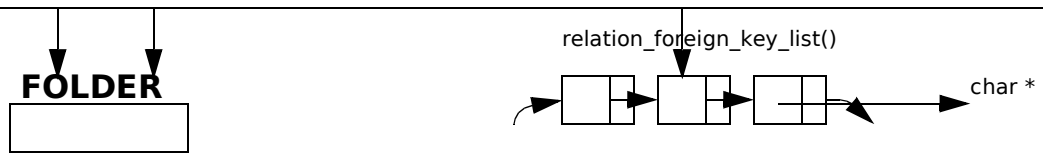
```
/* Usage */
LIST *row_security_role_update_list(
    void );

/* Attributes */
char *folder_name;
char *attribute_not_null;

/* Process */
char *row_security_role_update_system_string(
    void );

ROW_SECURITY_ROLE_UPDATE *
    row_security_role_update_parse(
        char *input );

LIST *relation_one2m_recursive_list =
    relation_one2m_recursive_list(
        (LIST *)0 /* relation_list */,
        folder_name,
        0 /* not fetch_process */ );

/* Public */
char *row_security_role_update_attribute_not_null(
        char *row_security_role_root_folder_name(),
        RELATION *row_security_role_relation(),
        LIST *row_security_role_update_list() );
```



RELATION

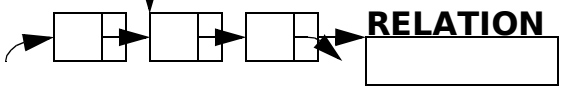
RELATION

```
/* Usage *
LIST *relation_mto1_non_isa_list(
    char *many_folder_name );

LIST *relation_mto1_isa_list(
    LIST *relation_list /* in/out */,
    char *many_folder_name,
    boolean fetch_process );

LIST *relation_one2m_list(
    char *one_folder_name );

LIST *relation_one2m_recursive_list(
    LIST *relation_list /* in/out */,
    char *one_folder_name );

LIST *relation_one2m_pair_list(
    char *one_folder_name );

LIST *relation_mto1_drillthru_list(
    LIST *relation_list /* in/out */,
    char *base_folder_name,
    LIST *fulfilled_folder_name_list );

/* Attributes */
char *many_folder_name;
char *one_folder_name;
char *related_attribute_name;
int pair_one2m_order;
boolean omit_one2m_detail;
boolean relation_type_isa;
boolean copy_common_attributes;
boolean automatic_preselection;
boolean drop_down_multi_select;
boolean join_one2m_each_row;
boolean ajax_fill_drop_down;
char *hint_message;
FOLDER *many_folder;
FOLDER *one_folder;
boolean consumes_taken;

/* Process */
char *relation_system_string(
    char *where,
    char *order );

LIST *relation_system_list(
    char *relation_system_string(),
    /* Setting both will set foreign_key_list */
    boolean fetch_folder,
    boolean fetch_attribute_list,
    boolean fetch_process );

RELATION *relation_parse(
    char *input,
    boolean fetch_folder,
    boolean fetch_folder_attribute_list,
    boolean fetch_process );

LIST *relation_foreign_key_list(
    folder_attribute_primary_key_list(
        one_folder->
            folder_attribute_list )
                /* primary_foreign_key_list */,
    related_attribute_name,
    foreign_attribute_name_list(
        foreign_attribute_list(
            many_folder->folder_name,
            one_folder->folder_name ) );

/* Public */
LIST *relation_pair_one2m_list(
    relation_one2m_list() );

LIST *relation_join_one2m_list(
    relation_one2m_recursive_list(),
    DICTIONARY *ignore_dictionary );

RELATION *relation_consumes(
    LIST *many_attribute_name,
    relation_mto1_non_isa_list() );

RELATION *relation_one2m_seek(
    char *folder_name,
    LIST *relation_one2m_list );

RELATION *relation_mto1_seek(
    char *folder_name,
    LIST *relation_mto1_list );

LIST *relation_mto1_folder_name_list(
    LIST *relation_mto1_list );

LIST *relation_one2m_folder_name_list(
    LIST *relation_one2m_list );

char *relation_list_display(
    LIST *relation_list );

void relation_set_one_folder_primary_delimited_list(
    LIST *relation_mto1_list );

boolean relation_exists_multi_select(
    LIST *relation_mto1_non_isa_list );

/* Private */
boolean relation_is_primary_key_subset(
    LIST *foreign_key_list,
    LIST *many_primary_key_list );

RELATION *relation_new(
    char *many_folder_name,
    char *one_folder_name );

char *relation_display(
    RELATION *relation );

LIST *relation_primary_delimited_list(
    FOLDER *one_folder );
```



RELATION_JOIN

```
/* Usage */
RELATION_JOIN *relation_join_new(
    DICTIONARY *row_dictionary /* in/out */,
    char *application_name,
    LIST *relation_join_one2m_list,
    char *one_folder_name,
    LIST *one_folder_primary_key_list  );

/* Process */
for RELATION *relation_join_one2m in relation_join_one2m_list
{
    RELATION_JOIN_FOLDER *relation_join_folder =
        relation_join_folder_new(
            row_dictionary,
            application_name,
            relation_join_one2m,
            one_folder_name,
            one_primary_key_list ) );

    dictionary_set(
        row_dictionary,
        relation_join_one2m->
            many_folder->
            folder_name,
        relation_join_folder->delimited_string );
}

/* Public */
void relation_join_free( relation_join );
```



RELATION_JOIN_FOLDER

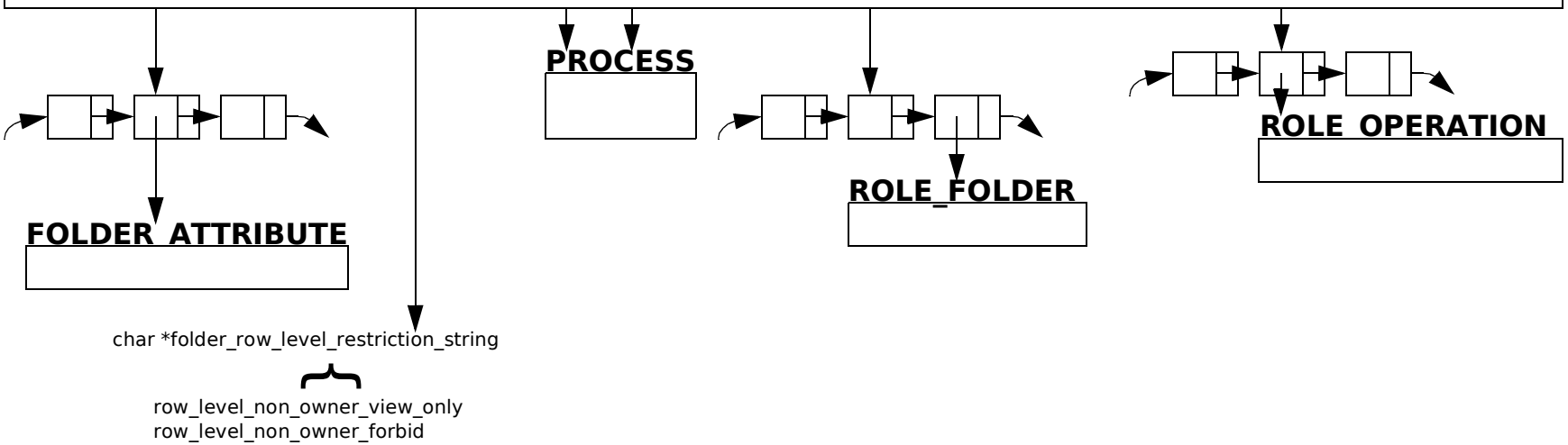
```
/* Usage */
RELATION_JOIN_FOLDER *relation_join_folder_new(
    DICTIONARY *row_dictionary /* in */,
    char *application_name,
    RELATION *relation_join_one2m,
    char *one_folder_name,
    LIST *one_primary_key_list  );

/* Process */
LIST *select_name_list =
    list_subtract(
        relation_join_one2m->
            many_folder->
            primary_key_list,
        primary_key_list )

RELATION_JOIN_FOLDER_WHERE *where =
    relation_join_folder_where_new(
        DICTIONARY *row_dictionary,
        RELATION *relation_join_one2m,
        char *one_folder_name,
        LIST *one_primary_key_list );

char *relation_join_folder_system_string(
    char *application_name,
    char *list_delimited(
        select_name_list,
        ',' ) /* select_name_list_string */,
    char *relation_join_one2m->
        many_folder->
        folder_name,
    char *relation_join_folder_where->string,
    char MULTI_ATTRIBUTE_DATA_LABEL_DELIMITER );

char *relation_join_folder_delimited_string(
    char *relation_join_folder_system_string() );
```

RELATION_JOIN_FOLDER_WHERE

```
RELATION_JOIN_FOLDER_WHERE *
    relation_join_folder_where_new(
        DICTIONARY *row_dictionary,
        RELATION *relation_join_one2m,
        char *one_folder_name,
        LIST *one_primary_key_list );

/* Process */
LIST *dictionary_data_list(
    one_primary_key_list,
    row_dictionary );

LIST *foreign_attribute_list(
    relation_join_one2m->many_folder->folder_name,
    one_folder_name );

LIST *foreign_attribute_name_list(
    foreign_attribute_list() );

LIST *relation_foreign_key_list(
    one_primary_key_list,
    relation_join_one2m->related_attribute_name,
    foreign_attribute_name_list() );

char *relation_join_folder_where_string(
    relation_foreign_key_list(),
    dictionary_data_list() );
```

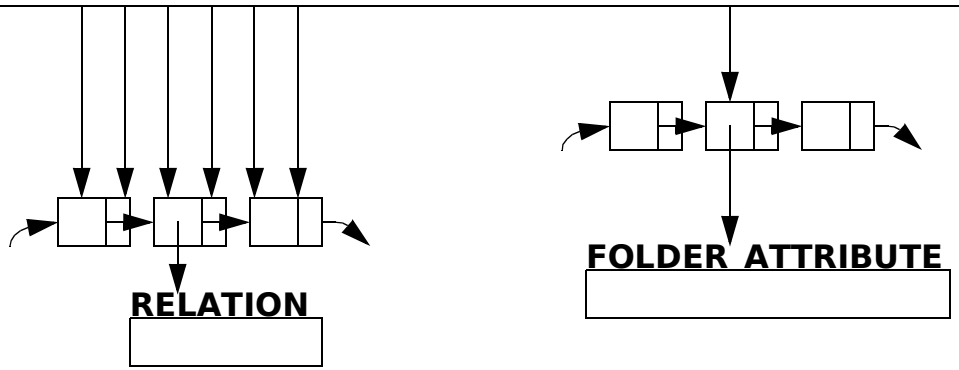
FOLDER (1)

```
/* Usage */
FOLDER *folder_fetch(
    security_sql_injection_escape(
        char *folder_name ),
    /* If fetching role_folder_list */
    security_sql_injection_escape(
        char *role_name ),
    role_exclude_..._attribute_name_list(),
    /* Also sets primary_attribute_list */
    /* and primary_key_list */
    boolean fetch_folder_attribute_list,
    boolean fetch_relation_mto1_non_isa_list,
    /* Maybe sets folder_attribute_append_isa_list */
    boolean fetch_relation_mto1_isa_list,
    boolean fetch_relation_one2m_list,
    boolean fetch_relation_one2m_recursive_list,
    boolean fetch_process,
    boolean fetch_role_folder_list,
    boolean fetch_row_level_restriction,
    boolean fetch_role_operation_list );

FOLDER *folder_quick_fetch(
    char *folder_name );

FOLDER *folder_mto1_fetch(
    char *folder_name );

/* Attributes */
char *folder_name;
char *folder_form;
int insert_rows_number;
boolean lookup_email_output;
char *notepad;
char *populate_drop_down_process_name;
char *post_change_process_name;
char *html_help_file_anchor;
char *post_change_javascript;
char *subschema;
boolean exclude_application_export;
boolean drillthru;
boolean no_initial_capital;
char *index_directory;
char *data_directory;
char *create_view_statement;
char *javascript_filename;

/* Process */
PROCESS *populate_drop_down_process;
PROCESS *post_change_process;

char *folder_primary_where(
    char *folder_name );

LIST *folder_attribute_list(
    folder_name,
    role_exclude_..._attribute_name_list() );

LIST *folder_attribute_primary_key_list(
        folder_attribute_list() );

LIST *role_folder_list(
    sql_injection_escape_role_name,
    sql_injection_escape_folder_name ) );

LIST *role_operation_list(
    role_name,
    folder_name );
```



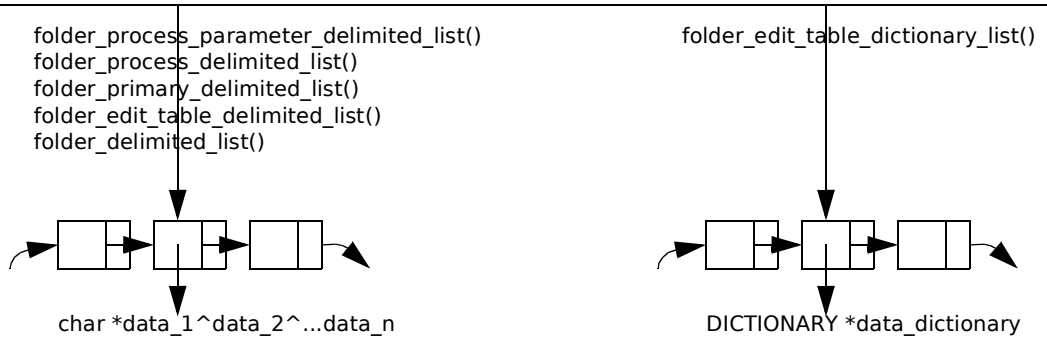
FOLDER (2)

```
LIST *relation_mto1_isa_list(
    (LIST *)0 /* relation_list */,
    char *folder_name,
    boolean fetch_process );

LIST *relation_mto1_non_isa_list(
    char *folder_name );

LIST *relation_one2m_list(
    folder_name );

LIST *relation_pair_one2m_list(
    relation_one2m_list() );

LIST *relation_one2m_recursive_list(
    (LIST *)0 /* one2m_recursive_list */,
    char *folder_name,
    boolean fetch_process );

LIST *relation_join_one2m_list(
    relation_one2m_recursive_list(),
    DICTIONARY *ignore_dictionary );

LIST *folder_attribute_append_isa_list(
    list_copy( folder_attribute_fetch_list() ),
    relation_mto1_isa_list() );
```



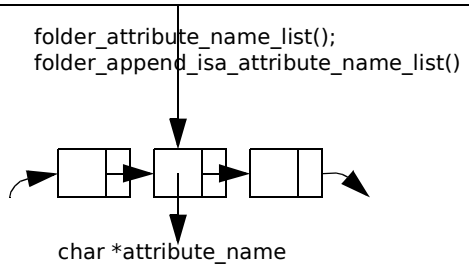
FOLDER (3)

```
/* Usage */
FOLDER *folder_process_delimited_fetch(
    char *folder_name,
    char *populate_drop_down_process_name,
    char *one2m_folder_name,
    char *state,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

ROLE *role =
    role_fetch(
        role_name );

FOLDER *folder =
    folder_fetch(
        char *folder_name,
        /* Not fetching role_folder_list */
        (char *)0 /* role_name */ ),
        (LIST *)0 /* role_exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_non_isa_list */,
        0 /* not fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        0 /* not fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid,
        role->override_row_restrictions );

LIST *folder_process_delimited_list(
    char *populate_drop_down_process_name,
    SECURITY_ENTITY *security_entity_new(),
    char *one2m_folder_name,
    char *state,
    DICTIONARY *drillthru_dictionary );

/* Public */
char *folder_table_name(
    char *environment_application_name(),
    char *folder_name );

LIST *folder_edit_table_delimited_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );

LIST *folder_delimited_list(
    char *folder_table_name(),
    LIST *attribute_name_list,
    char *where_clause );

LIST *folder_edit_table_dictionary_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );
```



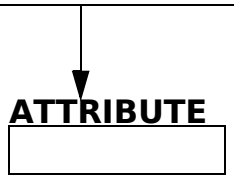
FOLDER (4)

```
/* Public */
long int folder_row_count(
    char *folder_table_name() );

char *folder_row_level_restriction_string(
    sql_injection_escape_folder_name );

boolean folder_non_owner_view_only(
    folder_row_level_restriction_string() );

boolean folder_non_owner_forbid(
    folder_row_level_restriction_string() );

boolean non_owner_view_only =
    folder_row_level_restrictions_non_owner_view_only(
        row_level_restriction_string );

boolean non_owner_forbid =
    folder_row_level_restrictions_non_owner_forbid(
        row_level_restriction_string );

LIST *folder_attribute_name_list(
    folder_attribute_list() );

LIST *folder_append_isa_attribute_name_list(
    folder_attribute_append_isa_list() );

LIST *attribute_float_list =
    folder_attribute_extract_float_list(
        folder_attribute_append_isa_list );

/* Unknown */
LIST *mto1_copy_common_related_folder_list;
LIST *copy_common_attribute_list;
LIST *append_isa_copy_folder_attribute_list;
LIST *copy_common_attribute_name_list;
LIST *append_isa_copy_attribute_name_list;
LIST *lookup_attribute_exclude_name_list;
```



FOLDER_ATTRIBUTE

```
/* Usage */
LIST *folder_attribute_fetch_list(
    char *security_sql_injection_escape(
        folder_name ),
    LIST *exclude_attribute_name_list );

/* Attributes */
char *folder_name;
char *attribute_name;
int primary_key_index;
int display_order;
boolean omit_insert;
boolean omit_insert_prompt;
boolean omit_update;
boolean additional_unique_index;
boolean additional_index;
boolean insert_required;
boolean lookup_required;

/* Process */
ATTRIBUTE *attribute =
    attribute_fetch(
        attribute_name );

char *folder_attribute_primary_where(
    char *folder_name,
    char *attribute_name );

char *folder_attribute_where(
    char *sql_injection_escape_folder_name,
    LIST *exclude_attribute_name_list );

char *folder_attribute_system_string(
    folder_attribute_where() );

LIST *folder_attribute_system_list(
    folder_attribute_system_string() );

FOLDER_ATTRIBUTE *folder_attribute_parse(
    char *input );

/* Public */
LIST *folder_attribute_append_isa_list(
    LIST *folder_attribute_list(),
    LIST *relation_mto1_isa_list );

LIST *folder_attribute_name_list(
    folder_attribute_list() |
    folder_attribute_append_isa_list(),
    char *folder_name );

LIST *folder_attribute_primary_list(
    folder_attribute_list() );

LIST *folder_attribute_primary_key_list(
    folder_attribute_list() );

LIST *folder_attribute_number_list(
    folder_attribute_list() );

LIST *folder_attribute_non_primary_key_list(
    folder_attribute_list() );

LIST *folder_attribute_date_name_list(
    folder_attribute_list() );

LIST *folder_attribute_time_name_list(
    folder_attribute_list() );

LIST *folder_attribute_date_time_name_list(
    folder_attribute_list() );

boolean folder_attribute_exists(
    char *folder_name,
    char *attribute_name );

FOLDER_ATTRIBUTE *folder_attribute_list_seek(
    char *attribute_name,
    LIST *folder_attribute_list );

char *folder_attribute_sort_attribute_name(
    LIST *folder_attribute_list );
```



ATTRIBUTE

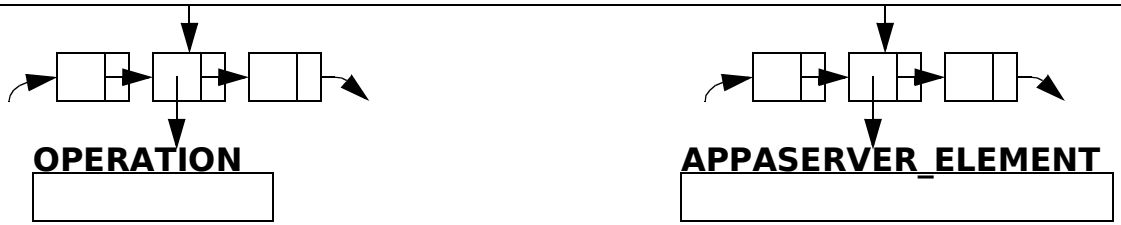
```
/* Usage */
ATTRIBUTE *attribute_fetch(
    char *attribute_name );

/* Attributes */
char *attribute_name;
char *datatype_name;
int width;
int float_decimal_places;
char *hint_message;
char *post_change_javascript;
char *on_focus_javascript_function;
boolean lookup_histogram_output;
boolean lookup_time_chart_output;

/* Public */
static LIST *attribute_list( void );

LIST *attribute_seek(
    char *attribute_name,
    LIST *attribute_list );

boolean attribute_exists(
    char *attribute_name,
    LIST * attribute_list );

LIST *attribute_extract_float_list(
    LIST *attribute_list );

char *attribute_full_attribute_name(
    char *folder_table_name(),
    char *attribute_name );

boolean attribute_is_date(
    char *datatype_name );

boolean attribute_is_time(
    char *datatype_name );

boolean attribute_is_number(
    char *datatype_name );

boolean attribute_is_date_time(
    char *datatype_name );

boolean attribute_is_text(
    char *datatype_name );

boolean attribute_is_notepad(
    char *datatype_name );

boolean attribute_is_password(
    char *datatype_name );

boolean attribute_is_upload(
    char *datatype_name );

boolean attribute_is_timestamp(
    char *datatype_name );

boolean attribute_is_yes_no(
    char *attribute_name );

boolean attribute_is_boolean(
    char *attribute_name );

int attribute_width(
    char *attribute_name );
```

FOREIGN_ATTRIBUTE

```
/* Usage */
LIST *foreign_attribute_list(
    char *many_folder_name,
    char *one_folder_name );

/* Input */
char *folder;
char *related_folder;
char *related_attribute;
char *foreign_attribute;
int foreign_key_index;

/* Process */
LIST *foreign_attribute_name_list(
    foreign_attribute_list() );
```

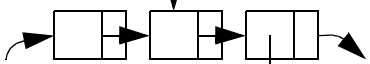
OPERATION_LIST

```
/* Usage */
OPERATION_LIST *operation_list_new(
    char *folder_name,
    char *role_name );

/* Process */
LIST *operation_list = list_new();

char *operation_list_select(
    void );

char *operation_list_where(
    char *folder_name,
    char *role_name );

char *operation_list_system_string(
    char *operation_list_select(),
    char *operation_list_where() );

list_set(
    operation_list,
    operation_parse(
        char *input ) );

/* Public */
LIST *operation_list_name_list(
    LIST *operation_list()->operation_list );

boolean operation_list_delete_boolean(
    LIST *operation_list()->operation_list );

LIST *operation_list_element_list(
    LIST *operation_list->operation_list(),
    int row_number,
    boolean viewonly );
```



OPERATION

```
/* Usage */
OPERATION *operation_parse(
    char *input );

/* Attributes */
char *operation_name;
boolean output;

/* Process */
OPERATION *operation_calloc(
    void );

PROCESS *process =
    process_fetch(
        operation_name,
        (char *)0 /* document_root_directory */,
        (char *)0 /* relative_source_directory */,
        1 /* check_executable_inside_filesystem */ );

boolean operation_delete_boolean(
    char *operation_name );

boolean operation_detail_boolean(
    char *operation_name );

char *operation_image_source(
    boolean operation_delete_boolean(),
    boolean operation_detail_boolean() );

char *operation_delete_warning_javascript(
    boolean operation_delete_boolean() );

APPASERVER_ELEMENT *operation_element(
    char *operation_name,
    char *operation_image_source(),
    char *operation_delete_warning_javascript() );

/* Public */
char *operation_html(
    APPASERVER_ELEMENT *operation_element(),
    int row_number );

int operation_row_total(
    DICTIONARY *row_dictionary,
    char *operation_name,
    int highest_index );

LIST *operation_row_list(
    DICTIONARY *row_dictionary,
    char *operation_name,
    LIST *primary_key_list,
    LIST *attribute_name_list,
    int operation_row_total() );
```



APPASERVER_ELEMENT

OPERATION_ROW

```
/* Usage */
LIST *operation_row_list(
    DICTIONARY *dictionary_separate->
        row_dictionary,
    char *operation_name,
    LIST *primary_key_list,
    LIST *attribute_name_list,
    int operation_row_total() );

/* Process */
OPERATION_ROW *operation_row_fetch(
    DICTIONARY *row_dictionary,
    char *operation_name,
    LIST *primary_key_list,
    LIST *attribute_name_list,
    int row_number );

LIST *operation_row_primary_data_list(
    DICTIONARY *row_dictionary,
    LIST *primary_key_list,
    int row_number );

DICTIONARY *operation_row_single_dictionary(
    DICTIONARY *row_dictionary,
    LIST *attribute_name_list,
    int row_number );

/* Public */
void operation_row_execute(
    process_operation_command_line(
        operation->process->command_line,
        environment_application_name(),
        operation_name,
        login_name,
        role_name,
        folder_name,
        getpid() /* parent_process_id */,
        session_key,
        operation_row_total(),
        operation_row_primary_data_list(),
        operation_single_row_dictionary() ) );

/* Private */
boolean operation_row_checked(
    DICTIONARY *dictionary_separate->
        row_dictionary,
    char *operation_name,
    int row_number );
```

OPERATION_SEMAPHORE

```
/* Usage */
OPERATION_SEMAPHORE *operation_semaphore_fetch(
    char *operation_name,
    char *appaserver_data_directory,
    pid_t parent_process_id,
    int operation_row_total() );

/* Process */
char *operation_semaphore_filename(
    char *operation_name,
    char *appaserver_data_directory,
    pid_t parent_process_id );

boolean operation_semaphore_group_first_time(
    operation_semaphore_filename() );

void operation_semaphore_initialize_file(
    operation_semaphore_filename() );

int operation_semaphore_row_current(
    operation_semaphore_filename() );

boolean operation_semaphore_group_last_time(
    operation_semaphore_row_current(),
    operation_row_total );

void operation_semaphore_increment(
    operation_semaphore_filename(),
    operation_semaphore_row_current() );

void operation_semaphore_remove_file(
    char *operation_semaphore_filename() );
```

SEMAPHORE

```
/* Usage */
SEMAPHORE *semaphore_new(
        key_t semaphore_key );

/* Process */
int semaphore_id(
        semaphore_key );

void semaphore_wait(
        semaphore_id() );

void semaphore_signal(
        semaphore_id() );
```

QUERY

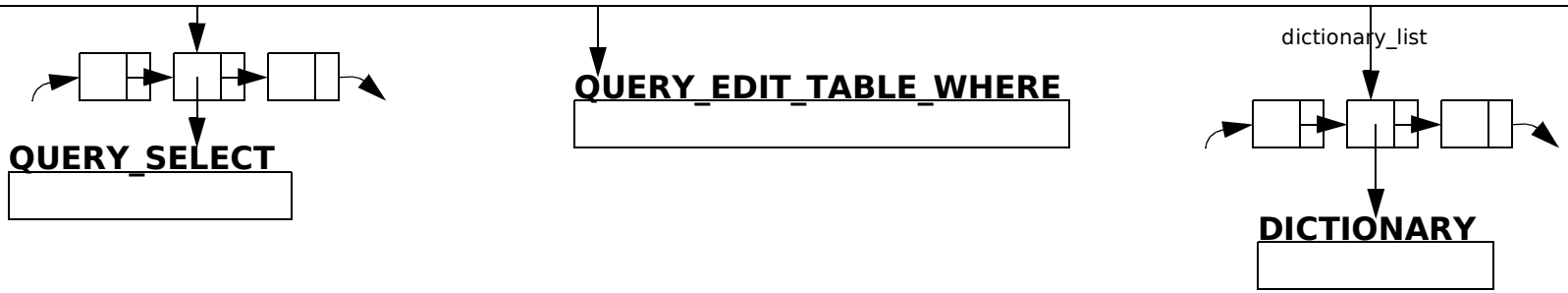
```
/* Public */
char *query_order_string(
    char *query_select_list_string(),
    DICTONARY *sort_dictionary );

LIST *query_delimited_list(
    char *query_select_list_string(),
    char *from_string,
    char *where->string,
    char *query_order_string() );

char *query_order_key_list_string(
    LIST *key_list );

char *query_system_string(
    char *select_string,
    char *from_string,
    char *where_string,
    char *order_string,
    int max_rows );

char *query_join_where(
    char *application_name,
    char *folder_name,
    LIST *primary_key_list,
    LIST *relation_mto1_isa_list,
    RELATION *row_security_role_relation );

LIST *query_primary_delimited_list(
    char *folder_table_name,
    LIST *primary_key_list,
    LIST *foreign_key_list,
    LIST *foreign_data_list );

/* Private */
char *query_join_relation_where(
    char *appliction_name,
    char *primary_folder_table_name,
    LIST *primary_key_list,
    LIST *relation_mto1_isa_list );

char *query_string_join_where(
    char *primary_folder_table_name,
    char *relation_folder_table_name,
    LIST *primary_key_list,
    LIST *foreign_key_list );

char *query_data_where(
    char *folder_name,
    LIST *where_attribute_name_list,
    LIST *where_attribute_data_list,
    LIST *folder_attribute_list );
```

QUERY_ISA WIDGET

```
/* Usage */
QUERY_ISA_WIDGET *query_isa_widget_new(
    char *one_isa_folder_name,
    LIST *folder_attribute_primary_key_list(),
    LIST *folder_attribute_primary_list(),
    char *login_name,
    char *security_entity_where() );

/* Process */
LIST *query_select_list(
    primary_key_list,
    folder_attribute_primary_list,
    query_date_convert_new(
        login_name ) );

char *query_select_list_string(
    LIST *query_select_list() );

QUERY_WIDGET_WHERE *where =
    query_widget_where_new(
        (LIST *)0 /* folder_attribute_list */,
        (LIST *)0 /* relation_mto1_non_isa_list */,
        security_entity_where,
        (DICTIONARY *)0 /* drillthru_dictionary */ );

char *query_order_string(
    query_select_list_string(),
    (DICTIONARY *)0 /* sort_dictionary */ );

LIST *delimited_list =
    query_delimited_list(
        query_select_list_string(),
        folder_table_name(
            environment_application_name(),
            one_isa_folder_name )
                /* from_string */,
        where->string,
        query_order_string() );
```

QUERY_EDIT_TABLE

```
/* Usage */
QUERY_EDIT_TABLE *query_edit_table_new(
    char *application_name,
    char *folder_name,
    char *login_name,
    char *security_entity_where(),
    LIST *relation_join_one2m_list,
    LIST *dictionary_separate->
        ignore_select_attribute_name_list,
    LIST *role->exclude_lookup_attribute_name_list,
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_non_isa_list,
    LIST *relation_mto1_isa_list,
    DICTIONARY *query_dictionary,
    DICTIONARY *sort_dictionary,
    char *row_security_role_folder_name,
    RELATION *row_security_role_relation,
    char *row_security_role_attribute_not_null );

/* Process */
LIST *query_edit_table_select_list(
    folder_attribute_append_isa_list,
    ignore_select_attribute_name_list,
    exclude_lookup_attribute_name_list,
    list_length( relation_mto1_isa_list ),
    QUERY_DATE_CONVERT *query_date_convert_new(
        login_name ),
    char *row_security_role_attribute_not_null );

char *query_select_list_string(
    query_edit_table_select_list() );

LIST *query_select_name_list(
    query_edit_table_select_list() );

char *query_edit_table_from_string(
    char *folder_name,
    LIST *relation_mto1_isa_list,
    char *row_security_role_folder_name );

QUERY_EDIT_TABLE_WHERE *where =
    query_edit_table_where_new(
        application_name,
        folder_name,
        folder_attribute_append_isa_list,
        relation_mto1_non_isa_list,
        list_length( relation_mto1_isa_list ),
        security_entity_where,
        query_dictionary,
        row_security_role_relation );

char *query_order_string(
    query_select_list_string,
    sort_dictionary );

char *query_system_string(
    query_select_list_string(),
    query_edit_table_from_string(),
    where->string,
    query_order_string(),
    QUERY_EDIT_TABLE_MAX_ROWS );

LIST *query_edit_table_dictionary_list(
    char *query_system_string(),
    LIST *query_select_name_list(),
    LIST *folder_attribute_primary_key_list(
        folder_attribute_append_isa_list )
            /* one_folder_primary_key_list */,
    char *application_name,
    LIST *relation_join_one2m_list,
    folder_name
            /* one_folder_name */  );
```

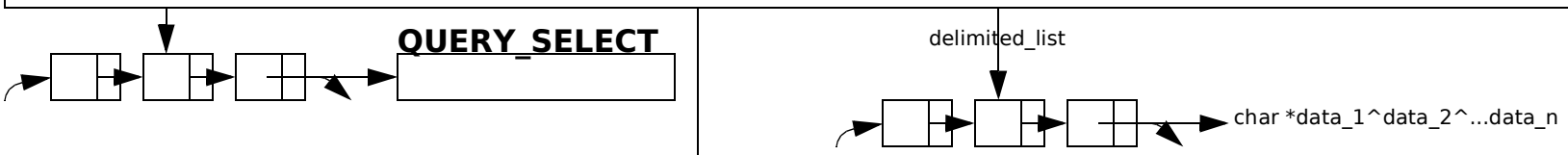


QUERY (DETAIL)

```
/* Usage */
QUERY *query_detail_new(
    LIST *where_attribute_data_list,
    char *security_sql_injection_escape(
        folder_name ),
    char *security_sql_injection_escape(
        role_name ),
    char *security_sql_injection_escape(
        login_name ),
    LIST *dictionary_separate->
        ignore_select_attribute_name_list,
    char *attribute_not_null_join,
    char *attribute_not_null_folder_name );

QUERY *query_search_replace_where_new(
    DICTIONARY *drillthru_dictionary,
    LIST *append_isa_attribute_list,
    LIST *relation_mto1_isa_list,
    boolean non_owner_forbid,
    SECURITY_ENTITY *security_entity );
```

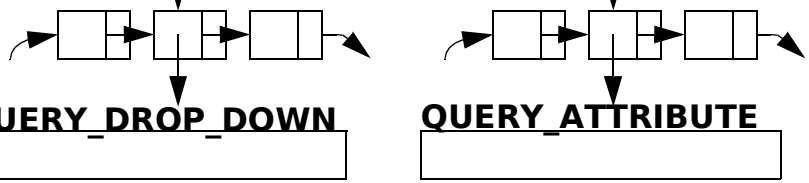
QUERY_WIDGET

```
/* Usage */
QUERY_WIDGET *query_widget_new(
    char *widget_folder_name,
    char *login_name,
    LIST *folder_attribute_list(),
    LIST *relation_mto1_non_isa_list,
    char *security_entity_where(),
    DICTIONARY *drillthru_dictionary );

/* Process */
LIST *query_widget_select_list(
    folder_attribute_primary_key_list(
        folder_attribute_list ),
    folder_attribute_list,
    query_date_convert_new(
        login_name ) );

char *query_select_list_string(
    query_widget_select_list() );

QUERY_WIDGET_WHERE *where =
    query_widget_where_new(
        folder_attribute_list,
        relation_mto1_non_isa_list,
        security_entity_where,
        drillthru_dictionary );

char *query_order_string(
    query_select_list_string(),
    (DICTIONARY *)0 /* sort_dictionary */ );

LIST *delimited_list =
    query_delimited_list(
        query_select_list_string(),
        folder_table_name(
            environment_application_name(),
            widget_folder_name )
                /* from_string */,
        where->string,
        query_order_string() );
```



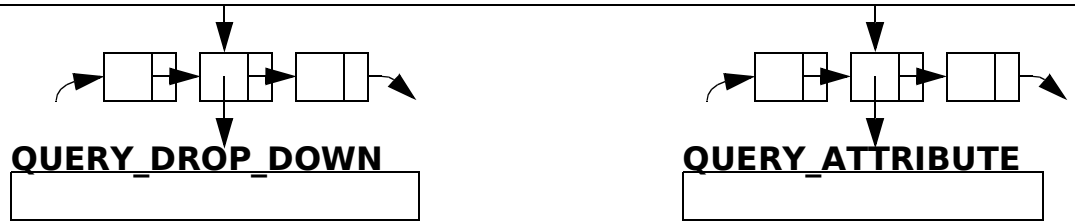
QUERY_WIDGET_WHERE

```
/* Usage */
QUERY_WIDGET_WHERE *
    query_widget_where_new(
        LIST *folder_attribute_list,
        LIST *relation_mto1_non_isa_list,
        char *security_entity_where(),
        DICTIONARY *drillthru_dictionary );

/* Process */
LIST *query_drop_down_list(
    (char *)0 /* many_folder_name */,
    folder_attribute_list,
    relation_mto1_non_isa_list,
    drillthru_dictionary );

char *query_drop_down_list_where(
    (char *)0 /* application_name */,
    query_drop_down_list(),
    0 /* relation_mto1_isa_list_length */ );

LIST *query_attribute_list =
    query_attribute_list(
        (char *)0 /* application_name */,
        folder_attribute_list,
        drillthru_dictionary,
        0 /* relation_mto1_isa_list_length */ );

char *query_attribute_list_where(
    query_attribute_list() );

char *query_widget_where_string(
    char *query_drop_down_list_where(),
    char *query_attribute_list_where(),
    char *security_entity_where );
```



QUERY_EDIT_TABLE_WHERE

```
/* Usage */
QUERY_EDIT_TABLE_WHERE *
    query_edit_table_where_new(
        char *application_name,
        char *folder_name,
        LIST *folder_attribute_append_isa_list,
        LIST *relation_mto1_non_isa_list,
        LIST *relation_mto1_isa_list,
        RELATION *row_security_role_relation,
        char *security_entity_where(),
        DICTIONARY *dictionary_separate->
            query_dictionary );

/* Process */
int relation_mto1_isa_list_length =
    list_length( relation_mto1_isa_list );

LIST *query_drop_down_list(
    folder_name /* many_folder_name */,
    folder_attribute_append_isa_list,
    relation_mto1_non_isa_list,
    query_dictionary );

char *query_drop_down_list_where(
    application_name,
    query_drop_down_list(),
    relation_mto1_isa_list_length );

LIST *query_attribute_list(
    application_name,
    folder_attribute_append_isa_list,
    query_dictionary,
    relation_mto1_isa_list_length );

char *query_attribute_list_where(
    query_attribute_list() );

char *query_join_where(
    application_name,
    folder_name,
    folder_attribute_primary_key_list(
        folder_attribute_append_isa_list ),
    related_mto1_isa_list,
    row_security_role_relation );

char *query_edit_table_where_string(
    query_drop_down_list_where(),
    query_attribute_list_where(),
    query_join_where(),
    security_entity_where );
```



QUERY_SELECT

```
/* Usage */
LIST *query_edit_table_select_list(
    LIST *folder_attribute_append_isa_list(),
    LIST *ignore_select_attribute_name_list,
    LIST *exclude_lookup_attribute_name_list,
    int relation_mto1_isa_length,
    QUERY_DATE_CONVERT *
        query_date_convert_new(
            login_name ),
    ROW_SECURITY_ROLE *row_security_role );

LIST *query_widget_select_list(
    LIST *folder_attribute_primary_key_list(),
    LIST *folder_attribute_list(),
    QUERY_DATE_CONVERT *
        query_date_convert_new(
            login_name) );

QUERY_SELECT *query_select_new(
    char *folder_name,
    char *attribute_name,
    boolean attribute_is_date(),
    boolean attribute_is_date_time(),
    QUERY_DATE_CONVERT *
        query_date_convert );

/* Process */
char *query_select_column_string(
    char *folder_name,
    char *attribute_name,
    boolean attribute_is_date,
    boolean attribute_is_date_time,
    enum date_convert_format
        query_date_convert->date_convert_format );

/* Public */
LIST *query_select_name_list(
    LIST *select_list );

char *query_select_list_string(
    LIST *select_list );
```

QUERY_DATE_CONVERT

```
/* Usage */
QUERY_DATE_CONVERT *query_date_convert_new(
    char *login_name );

/* Process */
enum date_convert_format
    query_date_convert_format(
        environment_application_name(),
        login_name );

/* Public */
char *query_date_convert_select_string(
    char *attribute_name,
    boolean attribute_is_date,
    boolean attribute_is_date_time,
    query_date_convert_format() );

enum date_convert_format
```

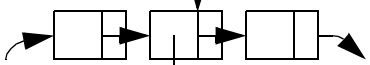
international,
american,
military,
date_convert_unknown

DATE_CONVERT

```
DATE_CONVERT *date_convert_international(
    char *application_name,
    char *login_name,
    char *date_string );

DATE_CONVERT *date_convert_string_international(
    enum date_convert_format source_format,
    char *date_string );

enum date_convert date_convert_user_date_format(
    char *application_name,
    char *login_name );
```

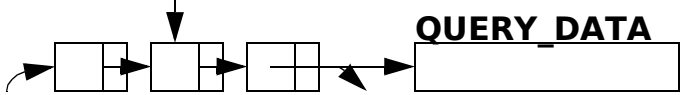
QUERY_DROP_DOWN

```
/* Usage */
LIST *query_drop_down_list(
    char *many_folder_name,
    LIST *folder_attribute_list,
    LIST *relation_mto1_non_isa_list,
    DICTIONARY *query|drillthru_dictionary );

/* Process */
for relation_mto1 in relation_mto1_non_isa_list
{
    int highest_index =
        dictionary_string_list_highest_index(
            relation_mto1->foreign_key_list,
            drillthru_dictionary );

    list_set(
        list,
        query_drop_down_new(
            many_folder_name,
            folder_attribute_list,
            relation_mto1->foreign_key_list,
            query|drillthru_dictionary,
            highest_index ) );
}

/* Usage */
QUERY_DROP_DOWN *query_drop_down_new(
    char *many_folder_name,
    LIST *folder_attribute_list,
    LIST *relation_mto1->foreign_key_list,
    DICTIONARY *query|drillthru_dictionary,
    int highest_index );

/* Input */
char *many_folder_name;

/* Process */
LIST *query_drop_down_row_list(
    foreign_key_list,
    folder_attribute_list,
    query|drillthru_dictionary /* dictionary */,
    highest_index );

/* Public */
char *query_drop_down_list_where(
    char *application_name,
    LIST *query_drop_down_list(),
    int relation_mto1_isa_list_length );
```



QUERY_DROP_DOWN_ROW

```
/* Usage */
LIST *query_drop_down_row_list(
    LIST *foreign_key_list,
    LIST *folder_attribute_list,
    DICTIONARY *query|drillthru_dictionary,
    int dictionary_string_list_highest_index() );

QUERY_DROP_DOWN_ROW*
    query_drop_down_row_new(
        LIST *foreign_key_list,
        LIST *folder_attribute_list,
        DICTIONARY *query|drillthru_dictionary
            /* dictionary */,
        int index );

/* Process */
char *query_drop_down_row_key(
    LIST *foreign_key_list,
    int index );

char *query_drop_down_row_data_list_string(
    query_drop_down_row_key(),
    DICTIONARY *query|drillthru_dictionary );

LIST *query_drop_down_row_data_string_list(
    char SQL_DELIMITER,
    query_drop_down_row_data_list_string() );

LIST *query_data_list(
    foreign_key_list,
    folder_attribute_list,
    query_drop_down_row_data_string_list() );

/* Public */
char *query_drop_down_row_list_where(
    char *folder_table_name(),
    LIST *query_drop_down_row_list(),
    int relation_mto1_isa_list_length );

/* Private *
char *query_drop_down_row_where(
    char *folder_table_name(),
    LIST *query_data_list,
    int relation_mto1_isa_list_length );

char *query_drop_down_row_data_where(
    char *folder_table_name,
    char *attribute_name,
```



QUERY_ATTRIBUTE

```
/* Usage */
LIST *query_attribute_list(
    char *application_name,
    LIST *folder_attribute_append_isa_list,
    DICTIONARY *query|drillthru_dictionary,
    int relation_mto1_isa_list_length );

QUERY_ATTRIBUTE *query_attribute_new(
    char *application_name,
    char *folder_name,
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary,
    int relation_mto1_isa_list_length );

/* Attributes */
char *application_name;
char *folder_name;
int relation_mto1_isa_list_length;

/* Process */
QUERY_RELATION *query_relation =
    query_relation_new(
        attribute_name,
        datatype_name,
        dictionary );

if ( !query_relation )
{
    return (QUERY_ATTRIBUTE *)0;
}

if ( query_relation->
    query_relation_enum == between )
{
    QUERY_BETWEEN_DATA *query_between_data =
        query_between_data_new(
            attribute_name,
            datatype_name,
            dictionary,
            application_name,
            folder_name,
            relation_mto1_isa_list_length );
}
else
{
    QUERY_DATA *query_data =
        query_data_fetch(
            folder_name,
            attribute_name,
            datatype_name,
            dictionary );
}

/* Public */
char *query_attribute_list_where(
    LIST *query_attribute_list() );

/* Private */
char *query_attribute_where_string(
    char *folder_table_name(
        application_name,
        folder_name ),
    int relation_mto1_isa_list_length,
    QUERY_RELATION *query_relation,
    QUERY_BETWEEN_DATA *query_between_data,
    QUERY_DATA *query_data );
```

QUERY_RELATION

```
/* Usage */
QUERY_RELATION *query_relation_new(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *query|drillthru_dictionary );

/* Process */
char *query_relation_key(
    char *attribute_name,
    char *QUERY_RELATION_STARTING_LABEL );

char *query_relation_yes_no_key(
    char *attribute_name );

char *query_relation_operator_fetch(
    char *query_relation_key(),
    char *query_relation_yes_no_key(),
    DICTIONARY *dictionary );

enum query_relation query_relation_enum(
    char *query_relation_operator_fetch(),
    char *datatype_name );

char *query_relation_character_string(
    enum query_relation_enum() );
```

```
                    equal                 greater_than_equal_to   query_or
                    not_equal             between                 is_null
enum query_relation { less_than           begins                  not_null
                    less_than_equal_to    contains
                    greater_than          not_contains
```

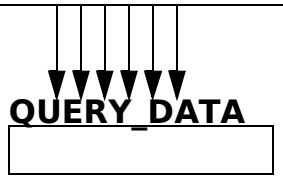
QUERY_DATA

QUERY_BETWEEN_DATA

QUERY_DATA

QUERY_BETWEEN_DATA

```
/* Usage */
QUERY_BETWEEN_DATA *query_between_data_new(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary_separate->
        query|drillthru_dictionary,
    char *application_name,
    char *folder_name,
    int relation_mto1_isa_list_length );

/* Process */
char *query_between_data_date_attribute_base(
    char *attribute_name,
    boolean
        (attribute_is_date( datatype_name ) ||
         attribute_is_time( datatype_name ) ) );

char *query_between_data_time_attribute_name(
    char *query_between_data_attribute_base() );

QUERY_DATA *query_between_data_from_date(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary );

QUERY_DATA *query_between_data_from_time(
    char *query_between_data_time_attribute_name(),
    DICTIONARY *dictionary );

if ( attribute_is_time( datatype_name )
&& query_between_data_from_date()
&& query_between_data_from_time() )
{
    /* Ignore the time attribute */
    return (QUERY_BETWEEN_DATA *)0;
}

QUERY_DATA *query_between_data_to_date(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary );

QUERY_DATA *query_between_data_to_time(
    char *query_between_data_time_attribute_name(),
    DICTIONARY *dictionary );

QUERY_DATA *query_between_data_from(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary );

QUERY_DATA *query_between_data_to(
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary );

char *query_between_data_date_time_where(
    QUERY_DATA *from_date,
    QUERY_DATA *from_time,
    QUERY_DATA *to_date,
    QUERY_DATA *to_time,
    char *folder_table_name(
        application_name,
        folder_name ),
    int relation_mto1_isa_list_length );

char *query_between_data_non_date_time_where(
    QUERY_DATA *data_from,
    QUERY_DATA *data_to,
    char *datatype_name,
    char *folder_table_name(
        application_name,
        folder_name ),
    int relation_mto1_isa_list_length );

char *query_between_data_where_string(
    query_between_data_date_time_where(),
    query_between_data_non_date_time_where() );
```



QUERY_DATA

```
/* Usage */
LIST *query_data_list(
    LIST *foreign_key_list,
    LIST *folder_attribute_list,
    LIST *query_drop_down_row_data_string_list() );

QUERY_DATA *query_data_fetch(
    char *folder_name,
    char *attribute_name,
    char *datatype_name,
    DICTIONARY *dictionary );

QUERY_DATA *query_data_new(
    char *folder_attribute->folder_name,
    char *folder_attribute->attribute_name,
    char *folder_attribute->attribute->datatype_name,
    char *data );

/* Attributes */
char *folder_name;
char *attribute_name;
char *datatype_name;

/* Process */
char *escaped_replaced_data =
    security_sql_injection_escape(
        security_replace_special_characters(
            query_data_date_international(
                char *string_trim_number_characters(
                    char *data,
                    char *datatype_name ),
                char *datatype_name ) ) );

/* Public */
char *query_data_list_where(
    char *application_name,
    int relation_mto1_isa_list_length,
    LIST *query_data_list() );

char *query_data_is_null_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ) );

char *query_data_not_null_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ) );

char *query_data_or_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    boolean attribute_is_number(
        query_data->datatype_name ),
    char *query_data->escaped_replaced_data );

char *query_data_begins_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    char *query_data->escaped_replaced_data );

char *query_data_contains_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    char *query_data->escaped_replaced_data );

char *query_data_not_contains_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    char *query_data->escaped_replaced_data );

char *query_data_not_equal_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    boolean attribute_is_number(
        query_data->datatype_name ),
    char *query_data->escaped_replaced_data );

char *query_data_number_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    char *query_relation->character_string,
    char *query_data->escaped_replaced_data );

char *query_data_string_where(
    char *attribute_full_attribute_name(
        folder_table_name(),
        query_data->attribute_name ),
    char *query_relation->character_string,
    char *query_data->escaped_replaced_data );
```

QUERY_OR_SEQUENCE

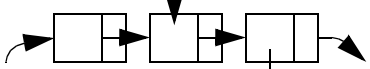
```
/* Usage */
QUERY_OR_SEQUENCE *query_or_sequence_new(
    LIST *attribute_name_list );

/* Attributes */
LIST *attribute_name_list;
LIST *data_list_list;

/* Public */
boolean query_or_sequence_set_data_list(
    LIST *data_list_list,
    LIST *data_list );

boolean query_or_sequence_set_data_list_string(
    LIST *data_list_list,
    char *list_string );

char *query_or_sequence_where(
    LIST *attribute_name_list,
    LIST *data_list_list );
```

ROLE

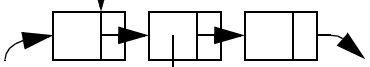
```
/* Usage */
ROLE *role_fetch(
    char *role_name,
    boolean fetch_role_attribute_exclude_list );

LIST *role_list_fetch(
    char *login_name );

/* Attributes */
char *role_name;
boolean folder_count;
boolean override_row_restrictions;

/* Process */
char *role_primary_where(
    char *role_name );

LIST *attribute_exclude_list =
    role_attribute_exclude_system_list();

/* Public */
LIST *role_exclude_lookup_attribute_name_list(
    attribute_exclude_list );

LIST *role_exclude_insert_attribute_name_list(
    attribute_exclude_list );

LIST *role_exclude_update_attribute_name_list(
    attribute_exclude_list );
```

ROLE_APPASERVER_USER

```
/* Usage */
ROLE_APPASERVER_USER *role_appaserver_user_fetch(
    char *session->login_name;
    char *security_sql_injection_escape(
        role_name ) );

LIST *role_appaserver_user_system_list(
    role_appaserver_user_system_string() );

/* Attributes */
char *login_name;
char *role_name;

/* Process */
char *role_appaserver_user_system_string(
    char *where );

char *role_appaserver_user_primary_where(
    char *login_name,
    char *role_name );
```



role_exclude_lookup_attribute_name_list()
role_exclude_insert_attribute_name_list()
role_exclude_update_attribute_name_list()



char *attribute_name

ROLE_ATTRIBUTE_EXCLUDE

```
/* Usage */
LIST *role_attribute_exclude_system_list(
    char *role_attribute_exclude_system_string(
        char *role_primary_where(
            char *role_name ) ) ) );

/* Attributes */
char *role_name;
char *attribute_name;
char *permission;

/* Process */
ROLE_ATTRIBUTE_EXCLUDE *
    role_attribute_exclude_parse(
        char *input );
```

insert
update
lookup

ROLE_FOLDER

```
/* Usage */
LIST *role_folder_list(
    char *role_name,
    char *folder_name );

LIST *role_folder_lookup_list(
    char *role_name );

LIST *role_folder_insert_list(
    char *role_name );

/* Attributes */
char *role_name;
char *folder_name;
char *permission;
char *subschema_name;

/* Public */
boolean role_folder_insert(
    role_folder_list() );

boolean role_folder_update(
    role_folder_list() );

boolean role_folder_lookup(
    role_folder_list() );

LIST *role_folder_subschema_name_list(
    role_folder_lookup|insert_list() );

LIST *role_folder_subschema_missing_folder_name_list(
    role_folder_lookup|insert_list() );

LIST *role_folder_subschema_folder_name_list(
    char *subschema_name,
    role_folder_lookup|insert_list() );

LIST *role_folder_name_list(
    LIST *role_folder_list() );
```

ROLE_PROCESS

```
/* Usage */
LIST *role_process_list(
    char *role_name );

/* Attributes */
char *role_name;
char *process_name;
char *process_group_name;

/* Public */
LIST *role_process_group_name_list(
    role_process_list() );

LIST *role_process_name_list(
    char *process_group_name,
    role_process_list() );

LIST *role_process_or_set_name_list(
        char *process_group_name,
        role_process_list(),
        role_process_set_list() );
```

ROLE_PROCESS_SET

```
/* Usage */
LIST *role_process_set_list(
    char *role_name );

/* Attributes */
char *role_name;
char *process_set_name;
char *process_group_name;

/* Generic */
LIST *role_process_set_group_name_list(
    role_process_set_list() );
```

UPDATE

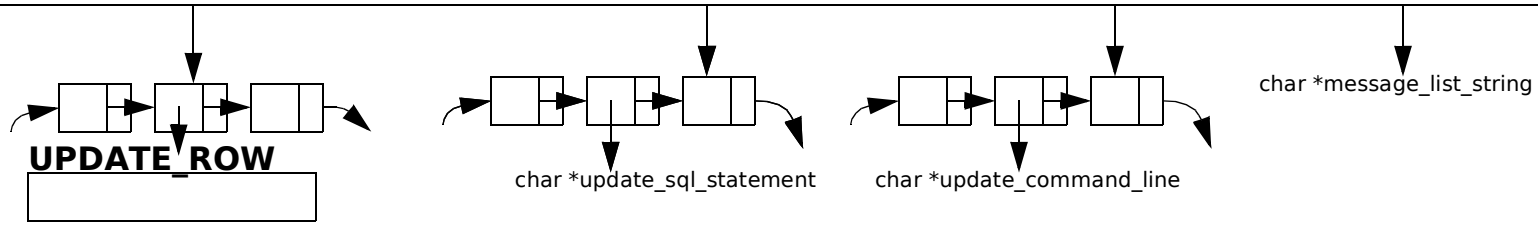
```
/* Usage */
UPDATE *update_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name );

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        role_exclude_update_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        0 /* not fetch_role_operation_list */ );

if ( !role_folder_update(
        folder->role_folder_list ) )
{
    return (UPDATE *)0;
}

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid ||
        folder->non_owner_view_only,
        role->override_row_restrictions );

UPDATE_ROW_LIST *update_row_list =
    update_row_list_new(
        post_dictionary,
        file_dictionary,
        login_name,
        folder->folder_name,
        folder->folder_attribute_list,
        folder->relation_mto1_isa_list,
        folder->relation_one2m_recursive_list,
        folder->post_change_process,
        security_entity );

/* Public */
char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        folder_name ),
    update_attribute_changed_list(),
    update_where_clause() );

char *update_command_line(
    char *command_line,
    char *login_name,
    LIST *update_attribute_list() );

/* Returns message_string */
char *update_execute(
    char *update_sql_statement() );

FILE *update_sql_pipe(
    char *environment_application_name() );

void update_sql_statement_list_execute(
    FILE *update_sql_pipe(),
    LIST *sql_statement_list );

/* Returns message_list_string */
char *update_command_line_list_execute(
    LIST *command_line_list );
```

post_dictionary
file_dictionary

DICTIONARY **FOLDER** **ROLE** **SECURITY_ENTITY** **UPDATE_ROW_LIST**

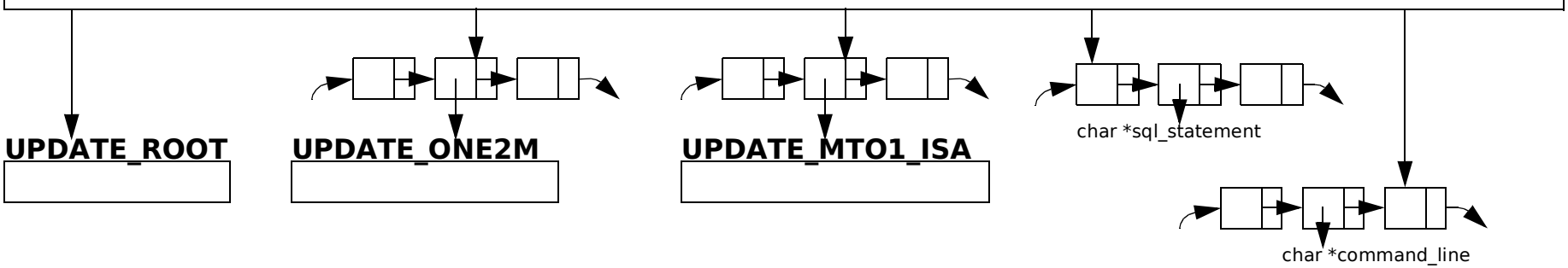
UPDATE_ROW_LIST

```
/* Usage */
UPDATE_ROW_LIST *update_row_list_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder->folder_name,
    LIST *folder->folder_attribute_list,
    LIST *folder->relation_mto1_isa_list,
    LIST *folder->relation_one2m_recursive_list,
    PROCESS *folder->post_change_process,
    SECURITY_ENTITY *security_entity );

/* Process */
LIST *list = list_new();

if ( dictionary_highest_row( post_dictionary )  == -1 )
    return (UPDATE_ROW_LIST *)0;

for (   row = 1;
        row <= dictionary_highest_row();
        row++ )
{
    list_set(
        list,
        update_row_new(
            post_dictionary,
            file_dictionary,
            login_name,
            folder_name,
            folder_attribute_list,
            relation_mto1_isa_list,
            relation_one2m_recursive_list,
            post_change_process,
            security_entity,
            row ) );
}

int update_row_list_cell_count(
    UPDATE_ROW_LIST *update_row_list );

/* Output */
char *message_list_string =
    char *update_row_list_execute(
        UPDATE_ROW_LIST *update_row_list )
{
    char *message_list_string = {0};
    char *message_string;

    for update_row in update_row_list->list
    {
        message_string = (char *)0;

        if ( update_row->update_root )
        {
            if ( ( message_string =
                    update_execute(
                        update_row->
                            update_root->
                            sql_statement,
                        (char *)0 /* application */ ) ) )
            {
                message_list_string =
                    string_append(
                        message_list_string,
                        message_string,
                        “<br>” );
            }
        }

        if ( message_string )
        {
            continue;
        }

        if ( list_length( update_row->
                        update_mto1_isa_list ) )
        {
            update_sql_statement_list_execute(
                update_sql_pipe(),
                update_mto1_isa_sql_statement_list(
                    update_row->
                        update_mto1_isa_list ) );
        }

        if ( update_row->
                update_root->
                changed_primary_key
        &&  list_length( update_row->
                        update_one2m_list ) )
        {
            update_sql_statement_list_execute(
                update_sql_pipe(),
                update_one2m_sql_statement_list(
                    update_row->
                        update_one2m_list ) );
        }

        if ( ( message_string =
                update_command_line_list_execute(
                    update_row->
                        command_line_list ) ) )
        {
            message_list_string =
                string_append(
                    message_list_string,
                    message_string,
                    “<br>” );
        }
    }
}

/* Debug */
LIST *update_row_list_sql_statement_list(
    UPDATE_ROW_LIST *update_row_list );

LIST *update_row_list_command_line_list(
    UPDATE_ROW_LIST *update_row_list );
```



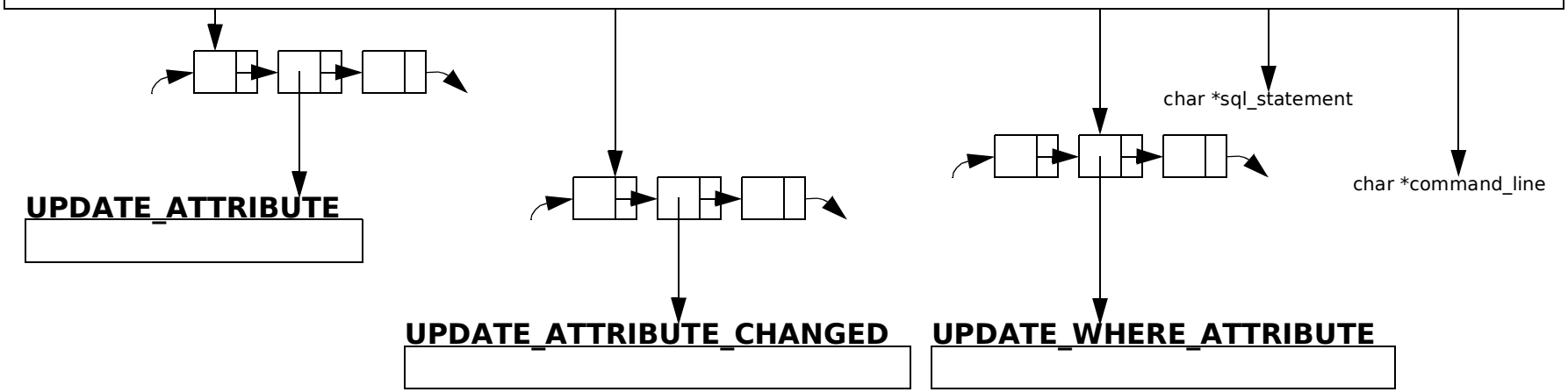
UPDATE_ROW

```
/* Usage */
UPDATE_ROW *update_row_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder_name,
    LIST *folder_attribute_list,
    LIST *relation_mto1_isa_list,
    LIST *relation_one2m_recursive_list,
    PROCESS *post_change_process,
    SECURITY_ENTITY *security_entity,
    int row );

/* Input */
int row;

/* Process */
UPDATE_ROOT *update_root =
    update_root_new(
        post_dictionary,
        file_dictionary,
        login_name,
        folder_name,
        folder_attribute_list,
        post_change_process,
        security_entity,
        int row );

LIST *update_mto1_isa_list(
    post_dictionary,
    file_dictionary,
    login_name,
    relation_mto1_isa_list,
    row );

LIST *update_one2m_list(
    post_dictionary,
    file_dictionary,
    login_name,
    relation_one2m_recursive_list,
    row );

int update_row_cell_count(
    UPDATE_ROOT *update_root,
    update_mto1_isa_list(),
    update_one2m_list() );

LIST *update_row_sql_statement_list(
    UPDATE_ROOT *update_root,
    LIST *update_mto1_isa_list(),
    LIST *update_one2m_list() );

LIST *update_row_command_line_list(
    UPDATE_ROOT *update_root,
    LIST *update_mto1_isa_list(),
    LIST *update_one2m_list() );
```



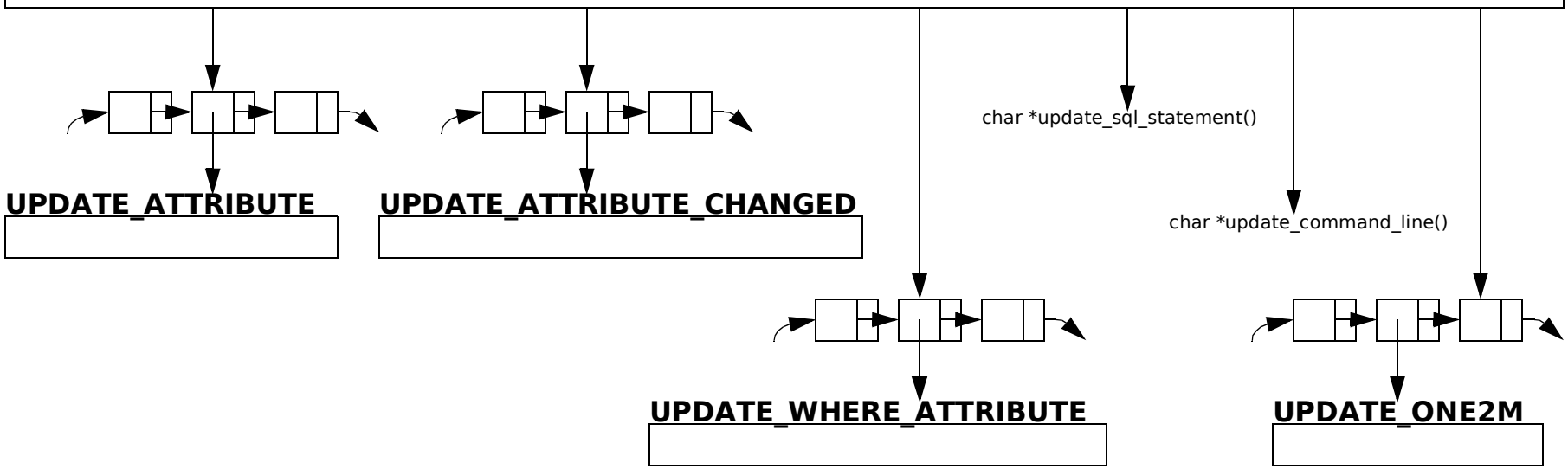
UPDATE_ROOT

```
/* Usage */
UPDATE_ROOT *update_root_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *folder_name,
    LIST *folder_attribute_list,
    PROCESS *post_change_process,
    SECURITY_ENTITY *security_entity,
    int row );

/* Process */
LIST *update_attribute_list(
    post_dictionary,
    file_dictionary,
    folder_attribute_list,
    row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

if ( !update_changed_attribute_list() ) return (UPDATE_ROOT *)0;

LIST *update_where_attribute_list(
    update_attribute_list() );

char *update_root_where_clause(
    char *update_where_clause(
        update_where_attribute_list(),
    SECURITY_ENTITY *security_entity,
    folder_attribute_list );

char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        folder_name ),
    update_attribute_changed_list(),
    update_root_where_clause() );

char *update_command_line(
    post_change_process->command_line,
    login_name,
    update_attribute_list() );

boolean update_attribute_changed_primary_key(
    update_attribute_changed_list() );
```



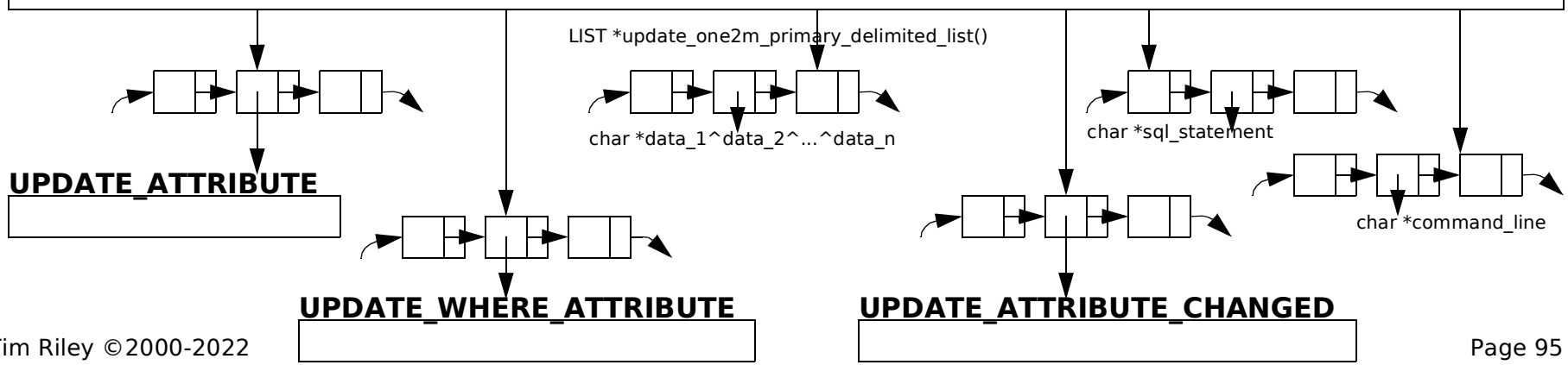
UPDATE_MTO1_ISA

```
/* Usage */
LIST *update_mto1_isa_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    LIST *relation_mto1_isa_list,.
    int row );

UPDATE_MTO1_ISA *update_mto1_isa_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    RELATION *relation_mto1_isa,
    int row  );

/* Process */
LIST *update_attribute_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    LIST *relation_mto1_isa->
        one_folder->
        folder_attribute_list,
    int row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

LIST *update_where_attribute_list(
    update_attribute_list() );

char *update_sql_statement(
    folder_table_name(
        environment_application_name(),
        relation_mto1_isa->
        one_folder->
        folder_name ),
    update_attribute_changed_list(),
    update_where_clause(
        update_where_attribute_list() )  );

char *update_command_line(
    relation_mto1_isa->
        one_folder->
        post_change_process->
        command_line,
    login_name,
    update_attribute_list() );

LIST *update_mto1_isa_one2m_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *relation_mto1_isa->
        one_folder->
        folder_name,
    int row );

/* Output */
LIST *update_mto1_isa_sql_statement_list(
    update_mto1_isa_list() );

LIST *update_mto1_isa_command_line_list(
    update_mto1_isa_list() );

LIST *update_mto1_isa_one2m_sql_statement_list(
    LIST *update_mto1_isa_one2m_list() );

LIST *update_mto1_isa_one2m_command_line_list(
    LIST *update_mto1_isa_one2m_list() );
```



UPDATE_ONE2M

```
/* Usage */
LIST *update_one2m_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    LIST *relation_one2m_recursive_list,
    int row  );

UPDATE_ONE2M *update_one2m_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    char *login_name,
    RELATION *relation_one2m,
    int row );

/* Process */
LIST *update_attribute_list(
    post_dictionary,
    file_dictionary,
    relation_one2m->many_folder->folder_attribute_list,
    row );

LIST *update_attribute_changed_list(
    update_attribute_list() );

LIST *update_where_attribute_list(
    update_attribute_list() );

LIST *update_one2m_primary_delimited_list(
    char *folder_table_name(
        environment_application_name(),
        relation_one2m->many_folder->folder_name ),
    LIST *relation_one2m->many_folder->primary_key_list,
    char *update_where_clause(
        update_where_attribute_list() );

LIST *update_one2m_sql_statement_list(
    char *folder_table_name(),
    LIST *update_attribute_changed_list(),
    LIST *relation_one2m->many_folder->primary_key_list,
    LIST *update_one2m_primary_delimited_list()  );

LIST *update_one2m_command_line_list(
    char *relation_one2m->many_folder->post_change_process->command_line,
    char *login_name,
    LIST *relation_one2m->many_folder->primary_key_list,
    LIST *update_one2m_primary_delimited_list(),
    LIST *update_attribute_changed_list() );

char *update_one2m_sql_statement(
    char *folder_table_name(),
    LIST *update_attribute_changed_list(),
    LIST *primary_key_list,
    LIST *list_string_list(
        list_get(
            update_one2m_primary_delimited_list() ),
        SQL_DELIMITER ) /* primary_data_list */ );

char *update_one2m_command_line(
    char *command_line,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    LIST *update_attribute_changed_list() );

/* Output */
LIST *update_one2m_list_sql_statement_list(
    LIST *update_one2m_list() );

LIST *update_one2m_list_command_line_list(
    LIST *update_one2m_list() );
```



UPDATE_ATTRIBUTE

```
/* Usage */
LIST *update_attribute_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    LIST *folder_attribute_list,
    int row );

UPDATE_ATTRIBUTE *update_attribute_new(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER_ATTRIBUTE *folder_attribute,
    int row );

/* Process */
char *file_data =
    dictionary_row_get(
        folder_attribute->attribute_name,
        row,
        file_dictionary );

char *sql_injection_escape_file_data =
    security_sql_injection_escape(
        file_data );

UPDATE_ATTRIBUTE_CHANGED *
    update_attribute_changed =
        update_attribute_changed_new(
            post_dictionary,
            folder_attribute->attribute_name,
            folder_attribute->
                attribute->
                datatype_name,
            folder_attribute->
                attribute->
                primary_key_index,
            file_data,
            row );

/* Public */
LIST *update_attribute_changed_list(
    LIST *update_attribute_list() );
```

UPDATE_ATTRIBUTE_CHANGED

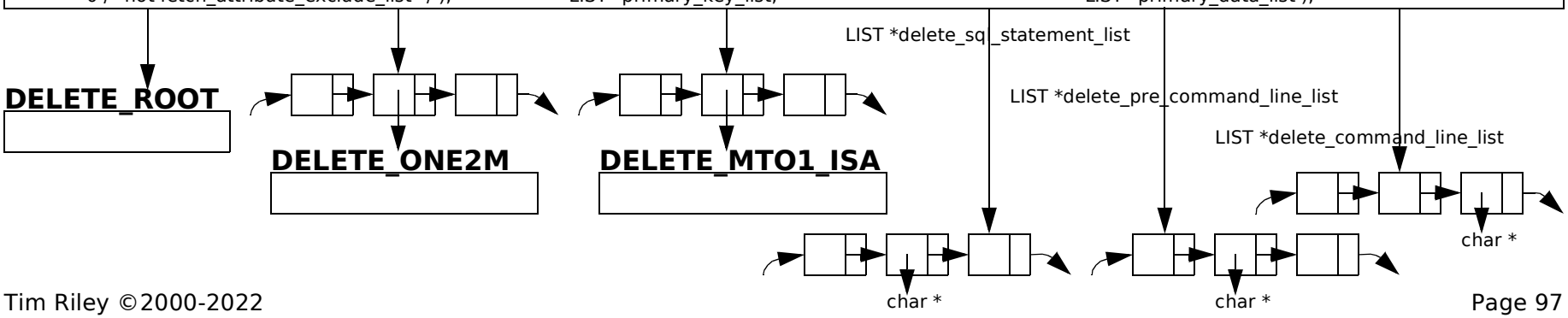
```
/* Usage */
UPDATE_ATTRIBUTE_CHANGED *
    update_attribute_changed_new(
        DICTIONARY *post_dictionary,
        char *attribute_name,
        char *datatype_name,
        int primary_key_index,
        char *file_data,
        int row );

/* Process */
char *post_data =
    dictionary_row_get(
        attribute_name,
        row,
        post_dictionary );

boolean attribute_changed =
    ( strcasecmp( file_data, post_data ) != 0 );

if ( !attribute_changed )
{
    return (UPDATE_ATTRIBUTE_CHANGED *)0;
}

char *sql_injection_escape_post_data =
    security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                post_data,
                datatype_name ) ) );

/* Public */
boolean update_attribute_changed_primary_key(
    LIST *update_attribute_changed_list() );

UPDATE_CHANGED_ATTRIBUTE *
    update_attribute_changed_seek(
        char *attribute_name,
        LIST *update_attribute_changed_list() );
```

UPDATE_WHERE_ATTRIBUTE

```
/* Usage */
LIST *update_where_attribute_list(
        LIST *update_attribute_list );

UPDATE_WHERE_ATTRIBUTE *
    update_where_attribute_new(
        char *primary_attribute_name,
        char *datatype_name,
        char *sql_injection_escape_file_data );

/* Public */
char *update_where_clause(
    LIST *update_where_attribute_list() );
```

DELETE

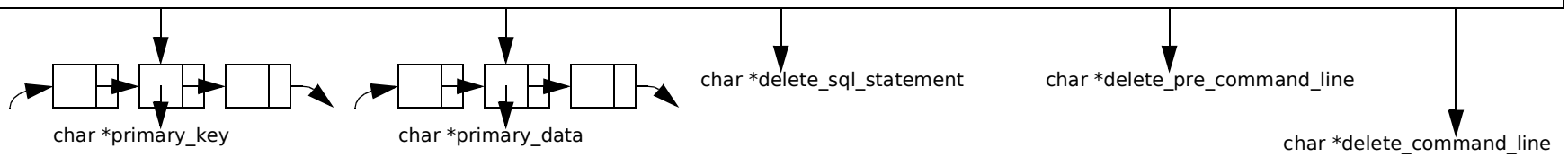
```
/* Usage */
DELETE *delete_new(
    char *application_name,
    char *session_key,
    char *login_name,
    char *folder_name,
    char *role_name,
    LIST *primary_data_list,
    boolean dont_delete_mto1_isa );

/* Process */
FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name /* fetching role_folder_list */,
        (LIST *)0 /* role_exclude_attribute_name_list */,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        /* Also sets folder_attribute_append_isa_list */
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        0 /* not fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */,
        1 /* fetch_role_operation_list */ );

if ( !role_operation_delete(
        folder->role_operation_list ) )
{
    return (DELETE *)0;
}

ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_attribute_exclude_list */ );

SECURITY_ENTITY *security_entity =
    security_entity_new(
        login_name,
        folder->non_owner_forbid ||
        folder->non_owner_view_only,
        role->override_row_restrictions );

DELETE_ROOT *delete_root =
    delete_root_new(
        application_name,
        folder_name,
        login_name,
        folder->primary_key_list,
        primary_data_list,
        folder->post_change_process,
        security_entity_where() );

LIST *delete_one2m_list(
    application_name,
    folder->relation_one2m_recursive_list,
    primary_data_list
        /* foreign_data_list */ );

if ( !dont_delete_mto1_isa )
{
    LIST *delete_mto1_isa_list(
        application_name,
        folder->relation_mto1_isa_list,
        primary_data_list );
}

/* Public */
char *delete_sql_statement(
    char *folder_table_name(
        application_name,
        folder_name ),
    LIST *primary_key_list,
    LIST *primary_data_list,
    char *security_entity_where );

char *delete_command_line(
    char *post_change_process->command_line,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    char *state );

/* Output */
LIST *delete_sql_statement_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_pre_command_line_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_command_line_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

LIST *delete_distinct_folder_name_list(
    DELETE_ROOT *delete_root,
    LIST *delete_one2m_list(),
    LIST *delete_mto1_isa_list() );

char *delete_message_html(
    char *primary_data_list,
    int list_length( delete_sql_statement_list() ),
    LIST *delete_distinct_folder_name_list() );

char *delete_primary_data_list_display(
    LIST *primary_data_list );
```



DELETE_ROOT

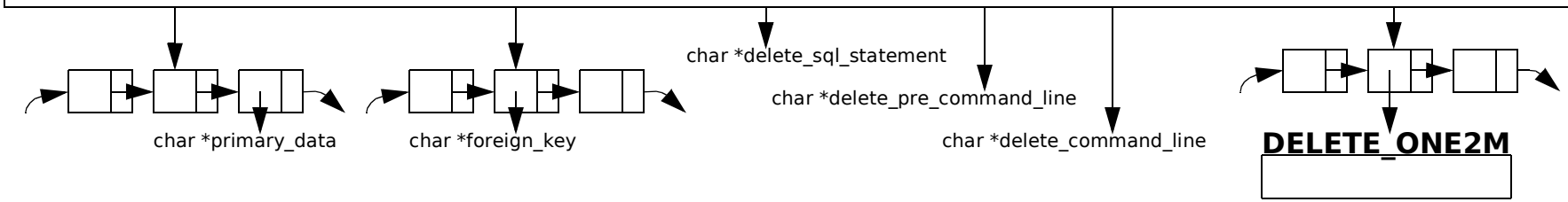
```
/* Usage */
DELETE_ROOT *delete_root_new(
    char *application_name,
    char *folder_name,
    char *login_name,
    LIST *primary_key_list,
    LIST *primary_data_list,
    PROCESS *post_change_process,
    char *security_entity_where );

/* Process */
```

```
char *delete_sql_statement(
    folder_table_name(),
    primary_key_list,
    primary_data_list,
    security_entity_where );

char *delete_pre_command_line =
    delete_command_line(
        post_change_process->command_line,
        login_name,
        primary_key_list,
        primary_data_list,
        “predelete” /* state */ );

char *delete_command_line =
    delete_command_line(
        post_change_process->command_line,
        login_name,
        primary_key_list,
        primary_data_list,
        “delete” /* state */ );
```



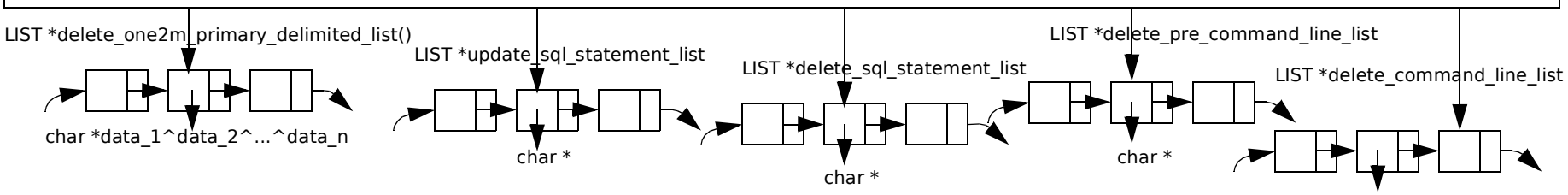
DELETE_MTO1_ISA

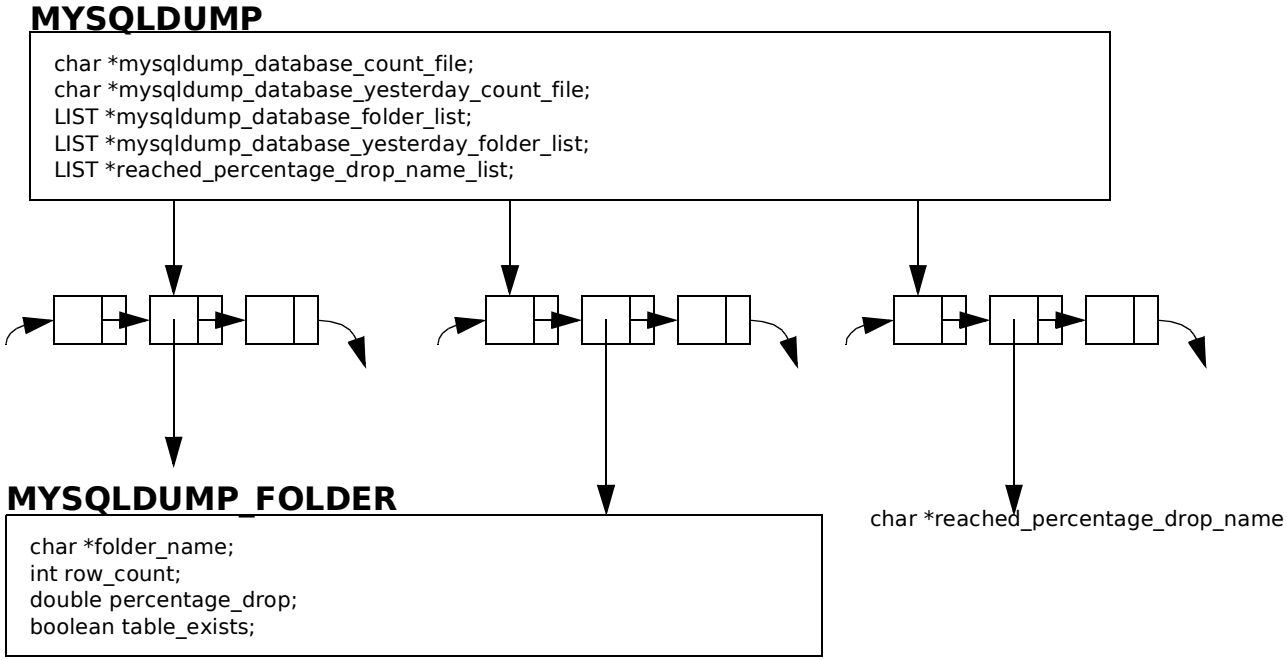
```
/* Usage */
LIST *delete_mto1_isa_list(
    char *application_name,
    char *login_name,
    LIST *folder->relation_mto1_isa_list,
    LIST *primary_data_list );

DELETE_MTO1_ISA *delete_mto1_isa_new(
    char *application_name,
    char *login_name,
    RELATION *relation_mto1_isa,
    LIST *primary_data_list );

/* Process */
char *delete_sql_statement(
    folder_table_name(
        application_name,
        relation_mto1_isa->
            one_folder->
            folder_name ),
    relation_mto1_isa->foreign_key_list,
    primary_data_list );

char *delete_pre_command_line =
    delete_command_line(
        relation_mto1_isa->
            one_folder->
            post_change_process->
            command_line,
        login_name,
        relation_mto1_isa->foreign_key_list,
        primary_data_list,
        "predelete" );

char *delete_command_line =
    delete_command_line(
        relation_mto1_isa->
            one_folder->
            post_change_process->
            command_line,
        login_name,
        relation_mto1_isa->foreign_key_list,
        primary_data_list,
        "delete" );

LIST *delete_mto1_isa_one2m_list(
    char *application_name,
    char *login_name,
    char *relation_mto1_isa->
        one_folder->
        folder_name,
    LIST *primary_data_list );

/* Output */
LIST *delete_mto1_isa_sql_statement_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_pre_command_line_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_command_line_list(
    LIST *delete_mto1_isa_list() );

LIST *delete_mto1_isa_one2m_sql_statement_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_one2m_pre_command_line_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_one2m_command_line_list(
    LIST *delete_mto1_isa_one2m_list() );

LIST *delete_mto1_isa_distinct_folder_name_list(
    LIST *delete_mto1_isa_list() );

LIST *
    delete_mto1_isa_one2m_distinct_folder_name_list(
        LIST *delete_mto1_isa_one2m_list() );
```



DELETE_ONE2M

```
/* Usage */
LIST *delete_one2m_list(
    char *application_name,
    char *login_name,
    LIST *folder->relation_one2m_recursive_list,
    LIST *foreign_data_list );

DELETE_ONE2M *delete_one2m_new(
    char *application_name,
    char *login_name,
    RELATION *relation_one2m,
    LIST *foreign_data_list );

/* Process */
LIST *delete_one2m_primary_delimited_list(
    char *folder_table_name(
        application_name,
        relation_one2m->
            many_folder->
            folder_name ),
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    LIST *relation_one2m->foreign_key_list,
    LIST *foreign_data_list );

if ( !relation_one2m->is_primary_key_subset )
{
    LIST *delete_one2m_update_sql_statement_list(
        char *folder_table_name(
            application_name,
            relation_one2m->
                many_folder->
                folder_name ),
        LIST *relation_one2m->foreign_key_list,
        LIST *relation_one2m->
            many_folder->
            primary_key_list,
        LIST *delete_one2m_primary_delimited_list() );
}
else
{
    LIST *delete_one2m_sql_statement_list(
        char *folder_table_name(
            application_name,
            relation_one2m->
                many_folder->
                folder_name ),
        LIST *relation_one2m->
            many_folder->
            primary_key_list,
        LIST *delete_one2m_primary_delimited_list() );
}

LIST *delete_pre_command_line_list =
    delete_one2m_command_line_list(
        char *relation_one2m->
            many_folder->
            post_change_process->
            command_line,
        char *login_name,
        LIST *relation_one2m->foreign_key_list,
        LIST *delete_one2m_primary_delimited_list(),
        “predelete” /* state */ );

LIST *delete_one2m_command_line_list(
    char *relation_one2m->
        many_folder->
        post_change_process->
        command_line,
    char *login_name,
    LIST *relation_one2m->foreign_key_list,
    LIST *delete_one2m_primary_delimited_list(),
    “delete” /* state */ );

/* Private */
char *delete_one2m_sql_statement(
    char *folder_table_name(),
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    list_get( delete_one2m_primary_delimited_list() )
        /* primary_data_list */ );

char *delete_one2m_update_sql_statement(
    char *folder_table_name(),
    LIST *relation_one2m->foreign_key_list,
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    list_get( delete_one2m_primary_delimited_list() )
        /* primary_data_list */ );

char *delete_one2m_update_set_null_statement(
    LIST *relation_one2m->foreign_key_list );

char *delete_command_line(
        post_change_process->command_line,
        login_name,
        relation_one2m->foreign_key_list,
        list_get( delete_one2m_primary_delimited_list() )
            /* primary_data_list */,
        state );

/* Output */
LIST *delete_one2m_list_sql_statement_list(
    LIST *delete_one2m_list() );

LIST *delete_pre_one2m_list_command_line_list(
    LIST *delete_one2m_list() );

LIST *delete_one2m_list_command_line_list(
    LIST *delete_one2m_list() );

LIST *delete_one2m_list_distinct_folder_name_list(
    LIST *delete_one2m_list() );
```



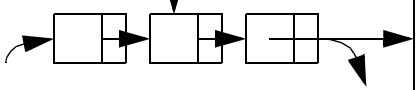

MYSQLDUMP
char *mysqldump_database_count_file;
char *mysqldump_database_yesterday_count_file;
LIST *mysqldump_database_folder_list;
LIST *mysqldump_database_yesterday_folder_list;
LIST *reached_percentage_drop_name_list;
MYSQLDUMP_FOLDER
char *folder_name;
int row_count;
double percentage_drop;
boolean table_exists;
char *reached_percentage_drop_name

STRING

```
/* Public */
boolean string_strcpy(
    char *destination,
    char *source,
    int buffer_size );

boolean string_strcmp(
    char *string1,
    char *string2 );

char *string_escape_character(
    char *destination,
    char *data,
    char character );

char *string_escape_character_array(
    char *destination,
    char *data,
    char *character_array );

/* Position is 1 based */
int string_character_position(
    char *data,
    char character );

char *string_trim_character(
    char *data,
    char character );

char *string_trim_character_array(
    char *data,
    char *character_array );

char *string_trim_number_characters(
    char *number,
    char *datatype_name )
{
    return string_trim_character_array(
        number,
        ",$" );
}

char *string_decode_html_post(
    char *destination,
    char *source );

char *string_trim( char *string );

char *string_extract_lt_gt_delimited(
    char *destination,
    char *source );

int string_index( char *source );

char string_last_character( char *string );

char *string_up( char *string );

char *string_low( char *string );
```

PAIR_ONE2M

```
char *many_folder_name;
char *one_folder_name;

PAIR_ONE2M *pair_one2m_prompt_form_new(
    char *one_folder_name,
    char *keystrokes_save_function );

LIST *pair_one2m_prompt_form_folder_list(
    char *keystrokes_save_function,
    LIST *relation_one2m_pair_relation_list() );

char *pair_one2m_prompt_element_name(
    char *PAIR_ONE2M_PREFIX,
    char *folder_label );

boolean pair_one2m_participating(
    DICTIONARY *pair_one2m_dictionary );

PAIR_ONE2M *pair_one2m_post_new(
    DICTIONARY *pair_one2m_dictionary );

boolean pair_one2m_duplicate(
    char *MYSQL_DUPLICATE_ERROR_MESSAGE_KEY,
    char *message );

void pair_one2m_duplicate_set(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_DUPLICATE_KEY );

boolean pair_one2m_inserted_duplicate(
    char *PAIR_ONE2M_DUPLICATE_KEY,
    DICTIONARY *pair_one2m_dictionary );

LIST *pair_one2m_fulfilled_folder_name_list(
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    DICTIONARY *pair_one2m_dictionary );

char *pair_one2m_folder_name(
    char *folder_label,
    DICTIONARY *pair_one2m_dictionary );

void pair_one2m_folder_set(
    DICTIONARY *pair_one2m_dictionary,
    char *folder_label,
    char *folder_name );

char *pair_one2m_next_folder_name(
    LIST *pair_one2m_fulfilled_folder_name_list(),
    LIST *relation_one2m_pair_relation_list(),
    char *one_folder_name,
    char *many_folder_name );

DICTIONARY *pair_one2m_fulfilled_dictionary(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    LIST *pair_one2m_fulfilled_folder_name_list );

/* Insert form */
PAIR_ONE2M *pair_one2m_insert_form_new(
    DICTIONARY *pair_one2m_dictionary );
```

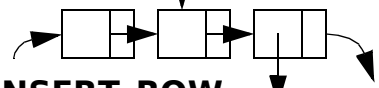


PAIR_ONE2M_FOLDER

```
/* Usage */
pair_one2m_folder_new(
    char *many_folder_name );

char *many_folder_name;

char *pair_folder_element_copy_function(
    prompt_many_element_name,
    many_folder_name,
    pair_one2m_prompt_form_element_name(),
    folder_name );

char *pair_folder_button_string(
    many_folder_name,
    keystrokes_save_function,
    pair_folder_element_copy_function() );
```

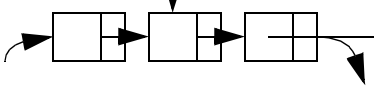
INSERT

```
/* Usage */
INSERT *insert_new(
    char *application_name,
    char *folder_name,
    LIST *primary_key_list,
    LIST *relation_mto1_non_isa_list(),
    LIST *folder_attribute_name_list(),
    LIST *folder_attribute_list(),
    DICTIONARY *row_dictionary,
    char *post_change_process->command_line );

/* Process */
LIST *insert_row_list = list_new();

for (    row = 0;
         row <= dictionary_highest_row(
             row_dictionary );
         row++ )
{
    list_set(
         insert_row_list,
         insert_row_new(
             folder_table_name(
                 application_name,
                 folder_name ),
             primary_key_list,
             relation_mto1_non_isa_list,
             folder_attribute_name_list,
             folder_attribute_list,
             row_dictionary,
             row,
             command_line ) );
}

/* Public */
LIST *insert_sql_statement_list(
    insert_row_list );

LIST *insert_post_change_command_line_list(
    insert_row_list );

char *insert_statement(
    char *folder_table_name(),
    LIST *insert_data_name_list(),
    LIST *insert_data_extract_list() );
```



INSERT_ROW

```
/* Usage */
INSERT_ROW *insert_row_new(
    char *folder_table_name(),
    LIST *primary_key_list,
    LIST *relation_mto1_non_isa_list,
    LIST *folder_attribute_name_list,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    char *post_change_command_line,
    int row );

/* Process */
LIST *insert_data_list = list_new();

list_set_list(
    insert_data_list,
    insert_data_relation_extract_list(
        list_get( relation_mto1_non_isa_list ),
        folder_attribute_list,
        row_dictionary,
        row ) );

list_set(
    insert_data_list,
    insert_data_attribute_extract(
        list_get( folder_attribute_name_list ),
        folder_attribute_list,
        row_dictionary,
        row ) );

boolean insert_row_primary_key_okay(
    LIST *insert_data_name_list(),
    LIST *primary_key_list );

char *insert_row_sql_statement(
    char *folder_table_name,
    LIST *insert_data_name_list(
        insert_data_list ),
    LIST *insert_data_extract_list(
        insert_data_list ) );

char *insert_row_command_line(
    char *post_change_command_line,
    LIST *insert_data_list );
```



INSERT_DATA

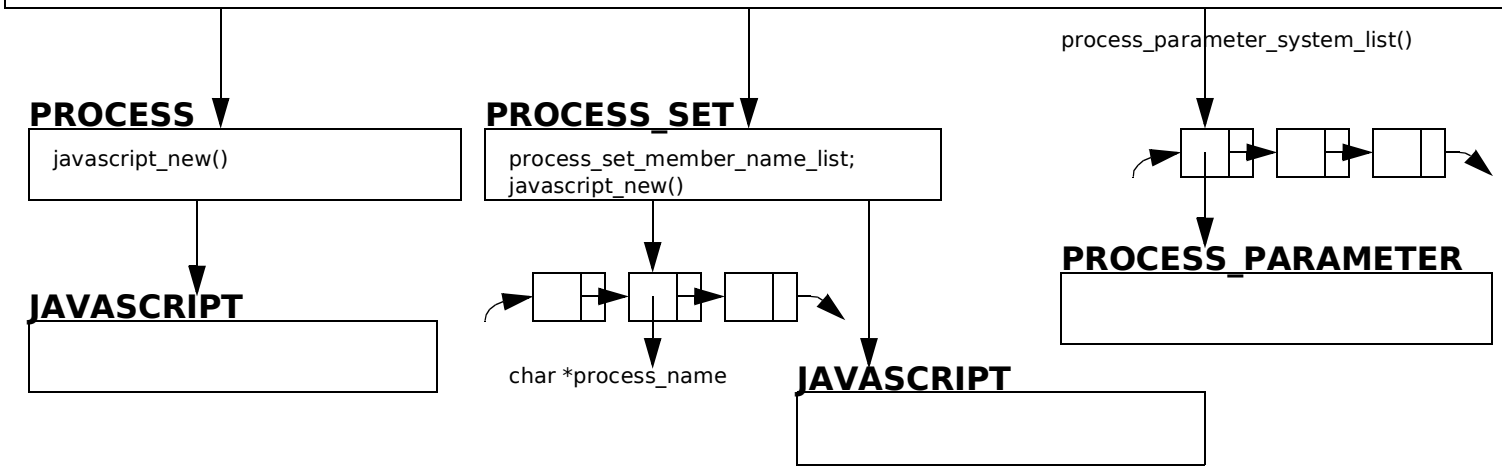
```
/* Usage */
INSERT_DATA *insert_data_new(
    char *attribute_name,
    char *security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                char *data,
                char *datatype_name ) ) ) );

/* Input */
char *attribute_name;
char *escaped_replaced_data;

/* Public */
LIST *insert_data_relation_extract_list(
    RELATION *relation_mto1,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    int row );

INSERT_DATA *insert_data_attribute_extract(
    char *attribute_name,
    LIST *folder_attribute_list,
    DICTIONARY *row_dictionary,
    int row );

LIST *insert_data_name_list(
    LIST *insert_data_list );

LIST *insert_data_extract_list(
    LIST *insert_data_list );
```

PROCESS_PROMPT_OUTPUT

```
/* Usage */
PROCESS_PROMPT_OUTPUT *
    process_prompt_output_fetch(
        char *process_or_set_name,
        char *role_name,
        char *login_name,
        boolean is_preprompt,
        appaserver_parameter->
            document_root,
        application_relative_source_directory(),
        DICTIONARY *drillthru_dictionary );

/* Input */
char *process_or_set_name;
char *role_name;
char *login_name;
boolean is_preprompt;
char *document_root_directory;
relative_source_directory;
DICTIONARY *drillthru_dictionary;
```

```
/* Process */
char *process_name =
    process_name_fetch(
        process_or_set_name );

char *process_set_name =
    process_set_name_fetch(
        process_or_set_name );

PROCESS *process =
    process_fetch(
        process_name,
        document_root_directory,
        relative_source_directory,
        0 /* not check_executable_inside */ );

PROCESS_SET *process_set =
    process_set_fetch(
        process_set_name,
        role_name,
        1 /* fetch_member_process_name_list */ );
```

```
LIST *process_parameter_list =
    process_parameter_system_list(
        process_set_parameter_system_string(
            process_set_parameter_where(
                process_set_name,
                is_preprompt ) ),
        role_name,
        login_name,
        drillthru_dictionary );

LIST *process_parameter_list =
    process_parameter_system_list(
        process_parameter_system_string(
            process_parameter_where(
                process_name,
                is_preprompt ) ),
        role_name,
        login_name,
        drillthru_dictionary );
```



APPASERVER Data Structure

PROCESS_PROMPT_SUBMIT

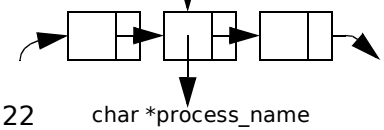
```
/* Usage */
PROCESS_PROMPT_SUBMIT *
    process_prompt_submit_fetch(
        char *process_name,
        char *role_name,
        char *login_name,
        DICTIONARY *working_post_dictionary );

/* Input */
char *process_name;
char *role_name;
char *login_name;
DICTIONARY *working_post_dictionary;

/* Process */
PROCESS *process =
    process_fetch(
        process_name,
        appaserver_parameter->
            document_root,
        application_relative_source_directory(),
        1 /* check_executable_inside_filesystem */ );

char *process_prompt_submit_command_line(
    char *command_line,
    char *process_name,
    char *role_name,
    char *login_name,
    DICTIONARY *working_post_dictionary );
```

PROCESS

PROCESS_SET

```
/* Usage */
PROCESS_SET *process_set_fetch(
    char *process_set_name,
    char *role_name,
    char *application_parameter_file->
        document_root,
    char *application_relative_source_directory(),
    boolean fetch_process_set_member_name_list );

/* Attributes */
char *notepad;
char *html_help_file_anchor;
char *post_change_javascript;
char *prompt_display_text;
char *process_group;
char *preprompt_help_text;
boolean prompt_display_bottom;
char *javascript_filename;

PROCESS_SET *process_set_parse(
    char *input,
    char *role_name,
    char *document_root_directory,
    char *application_relative_source_directory,
    boolean fetch_process_set_member_list );

char *process_set_primary_where(
    char *process_set_name );

LIST *process_set_member_name_list(
    process_set_primary_where(),
    role_name );

JAVASCRIPT *javascript =
    javascript_new(
        javascript_filename,
        document_root_directory,
        application_relative_source_directory );

/* Generic */
char *process_set_process_where(
    process_set_member_name_list() );
```

process_set_member_name_list()



char *process_name

PROCESS

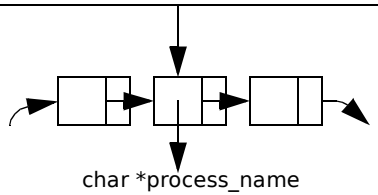
```
/* Usage */
PROCESS *process_fetch(
    char *process_name,
    char *application_parameter_file->
        document_root,
    char *application_relative_source_directory(),
    boolean check_executable_inside_filesystem );

/* Attributes */
char *command_line;
char *notepad;
char *html_help_file_anchor;
int execution_count;
char *post_change_javascript;
char *process_set_display;
char *process_group;
char *preprompt_help_text;
char *javascript_filename;

char *process_primary_where(
    char *process_name );

char *process_system_string(
    char *where );

PROCESS *process_parse(
    char *input,
    char *document_root_directory,
    char *relative_source_directory,
    boolean check_executable_inside_filesystem );

JAVASCRIPT *javascript =
    javascript_new(
        javascript_filename,
        document_root_directory,
        relative_source_directory );
```

JAVASCRIPT

```
/* Usage */
JAVASCRIPT *javascript =
    javascript_new(
        char *javascript_filename,
        appaserver_parameter->
            document_root,
        application_relative_source_directory() );

/* Process */
char *javascript_html(
    javascript_filename,
    document_root_directory,
    application_relative_source_directory );

char *javascript_source(
    javascript_html(),
    document_root_directory );

/* Public */
char *javascript_replace(
    char *post_change_javascript,
    char *state,
    int row_number );

char *javascript_post_change_append(
    char *original_post_change_javascript,
    char *post_change_javascript,
    boolean place_first );

void javascript_output_cat(
    FILE *output_stream,
    javascript_source() );

void javascript_output_link(
    FILE *output_stream,
    javascript_html() );

/* Private */
void javascript_replace_state(
    char *post_change_javascript /* in/out */,
    char *state );

void javascript_replace_row(
    char *post_change_javascript /* in/out */,
    int row_number );
```

PROCESS public

```
boolean process_prompt_exists_preprompt(
    char *process_or_set_name );

boolean process_executable_ok(
    char *executable );

char *process_name_fetch(
    char *process_or_set_name );

char *process_set_name_fetch(
    char *process_or_set_name );

char *process_dictionary_process_member(
    char *process_set_name,
    DICTIONARY *parsed_decoded_post_dictionary,
    char *from_starting_label );

LIST *process_role_process_name_list(
    char *role_primary_where(
        char *role_name ) );

LIST *process_delimited_list(
    char *command_line );

void process_execution_count_increment(
    char *process_name );

/* Private */
boolean process_interpreted_executable_ok(
    char *which_string );
```



PROCESS command_line

```
/* Public */
void process_replace_where_command_line(
    char *command_line,
    LIST *folder_attribute_list,
    LIST *relation_mto1_non_isa_list,
    char *security_entity_where(),
    DICTIONARY *drillthru_dictionary );

char *process_parameter_command_line(
    char *command_line,
    char *process_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

char *process_choose_isa_command_line(
    char *command_line,
    char *application_name,
    char *security_entity_where,
    char *login_name,
    char *role_name,
    char *one_isa_folder_name );

char *process_operation_command_line(
    char *command_line,
    char *environment_application_name(),
    char *operation_name,
    char *login_name,
    char *role_name,
    char *folder_name,
    pid_t parent_process_id,
    char *session_key,
    int operation_row_total(),
    LIST *operation_row_primary_data_list(),
    DICTIONARY *operation_single_row_dictionary() );

char *process_widget_command_line(
    char *command_line,
    char *application_name,
    char *security_entity_where,
    char *one_folder_name,
    char *state,
    DICTIONARY *drillthru_dictionary );

char *process_update_row_command_line(
    char *command_line,
    DICTIONARY *drillthru_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *state,
    int row,
    char *process_name,
    char *one_folder_name,
    LIST *primary_data_list );

void process_replace_one_folder_command_line(
    char *command_line,
    char *one_folder_name );

void process_replace_state_command_line(
    char *command_line,
    char *state );
```

PROCESS_PARAMETER

```
/*  Usage */
LIST *process_parameter_system_list(
    char *process_[set_]parameter_system_string(),
    char *role_name,
    char *login_name,
    DICTIONARY *drillthru_dictionary );

/* Input */
char *role_name;
char *login_name;
DICTIONARY *drillthru_dictionary;

/* Attributes */
char *process_or_set_name;
char *folder_name;
char *attribute_name;
char *drop_down_prompt_name;
char *prompt_name;

/* Process */
int display_order;
boolean drop_down_multi_select;
boolean preprompt;
char *populate_drop_down_process_name;
char *populate_helper_process_name;

char *process_parameter_system_string(
    char *where );

char *process_set_parameter_system_string(
    char *where );

PROCESS_PARAMETER *process_parameter_parse(
    char *input,
    char *role_name,
    char *login_name,
    DICTIONARY *drillthru_dictionary );

PROCESS_PARAMETER *process_parameter_new(
    char *process_or_set_name,
    char *folder_name,
    char *attribute_name,
    char *drop_down_prompt_name,
    char *prompt_name );

LIST *process_parameter_process_delimited_list(
    char *populate_drop_down_process_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

LIST *process_parameter_folder_delimited_list(
    char *widget_folder_name,
    char *login_name,
    char *role_name,
    DICTIONARY *drillthru_dictionary );

ATTRIBUTE *attribute =
    attribute_fetch(
        attribute_name );

DROP_DOWN_PROMPT *drop_down_prompt =
    drop_down_prompt_fetch(
        drop_down_prompt_name );

PROMPT *prompt =
    prompt_fetch(
        prompt_name );

/* Generic */
char *process_parameter_where(
    char *where,
    boolean is_preprompt );
```

ATTRIBUTE

```
char *attribute_name;
```

DROP_DOWN_PROMPT

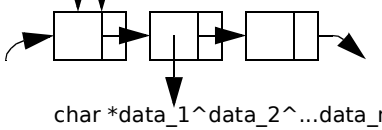
```
/* Usage */
DROP_DOWN_PROMPT *drop_down_prompt_fetch(
    char *drop_down_prompt_name );

char *drop_down_prompt_name;

/* Process */
char *hint_message;
char *optional_display;

LIST *drop_down_prompt_data_list =
    drop_down_prompt_data_system_list(
        drop_down_prompt_data_system_string(
            drop_down_prompt_primary_where(
                this->drop_down_prompt_name ) ) );
```

PROMPT

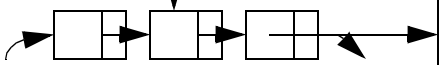
```
/* Usage */
PROMPT *prompt_fetch(
        char *prompt_name );

char *prompt_name;

/* Process */
int input_width;
char *hint_message;
boolean upload_filename;
boolean date;
```

FOLDER

```
char *folder_name;
LIST *primary_delimited_list
LIST *primary_delimited_display_list
```



char *data_1^data_2^...data_n



DROP_DOWN_PROMPT_DATA

```
char *drop_down_prompt_name;
char *drop_down_prompt_data;
int display_order;
```

PROCESS_GENERIC (1)

```
/* Usage */
PROCESS_GENERIC *process_generic_fetch(
    char *process_set_name,
    char *process_name,
    char *login_name,
    char *output_medium_string,
    DICTIONARY *post_dictionary );

/* Input */
char *process_set_name;
char *login_name;
char *output_medium_string;
DICTIONARY *post_dictionary;

/* Process */
char *process_name =
    process_generic_process_name(
        process_set_name,
        post_dictionary );

char *value_folder_name =
    process_generic_value_folder_name(
        process_name,
        process_set_name );

PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        value_folder_name );

PROCESS_GENERIC_PARAMETER *parameter =
    process_generic_parameter_parse(
        output_medium_string,
        post_dictionary,
        value_folder->datatype->aggregation_sum,
        value_folder->primary_key_list,
        value_folder->date_attribute_name,
        value_folder->time_attribute_name );

char *process_generic_select(
    value_folder->primary_key_list,
    parameter->value_attribute_name );

char *process_generic_date_where(
    value_folder->date_attribute_name,
    parameter->begin_date,
    parameter->end_date );

PROCESS_GENERIC_WHERE *process_generic_where =
    process_generic_where_new(
        value_folder_name,
        value_folder->foreign_folder->foreign_key_list
            /* where_attribute_name_list */,
        value_folder->foreign_folder->foreign_attribute_data_list
            /* where_attribute_data_list */,
        value_folder->foreign_folder->folder->folder_attribute_primary_list,
        process_generic_date_where() );

char *process_generic_system_string(
    process_generic_select(),
    value_folder_name,
    process_generic_where->where_clause,
    parameter->aggregate_level,
    parameter->aggregate_statistic,
    parameter->accumulate,
    parameter->date_piece,
    parameter->time_piece,
    parameter->value_piece,
    parameter->end_date,
    value_folder->datatype->unit /* datatype_unit */,
    value_folder->foreign_folder->unit /* foreign_folder_unit */,
    parameter->units_converted );
```

PROCESS_GENERIC_VALUE_FOLDER

PROCESS_GENERIC_PARAMETER

PROCESS_GENERIC_WHERE

PROCESS_GENERIC (2)

```
char *process_generic_heading(
    value_folder->primary_key_list,
    value_folder->value_attribute_name,
    value_folder->datatype->unit /* datatype_unit */,
    value_folder->foreign_folder->unit /* foreign_folder_unit */,
    parameter->units_converted,
    parameter->aggregate_level,
    parameter->aggregate_statistic,
    parameter->accumulate );

char *process_generic_subtitle(
    value_folder_name,
    parameter->begin_date,
    parameter->end_date,
    parameter->aggregate_level,
    parameter->aggregate_statistic
    additional_message );

/* Public */
char *process_generic_units_label(
    units,
    units_converted,
    enum aggregate_statistic );

enum date_convert_format process_generic_date_convert_format(
    login_name );

boolean process_generic_no_cycle_color(
    role_name );
```

PROCESS_GENERIC_PARAMETER

```
/* Attributes */
char *output_medium_string;
enum process_generic_medium output_medium;
char *begin_date;
char *end_date;
enum aggregate_level aggregate_level;
enum aggregate_statistic aggregate_statistic;
boolean accumulate;
char *email_address;
char *units_converted;
pidt process_id;
int date_piece;
int time_piece;
int value_piece;

/* Usage */
PROCESS_GENERIC_PARAMETER *process_generic_parameter_parse(
    process_generic->output_medium_string,
    process_generic_>post_dictionary,
    process_generic->value_folder->datatype->aggregation_sum,
    value_folder_primary_key_list,
    value_folder_date_attribute_name,
    value_folder_time_attribute_name );
```

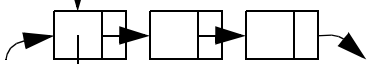
{ text_file, output_medium_stdout, table, spreadsheet, gracechart, output_medium_unknown

PROCESS_GENERIC_WHERE

```
/* Usage */
PROCESS_GENERIC_WHERE *process_generic_where_new(
    char *value_folder_name,
    LIST *where_attribute_name_list,
    LIST *where_attribute_data_list,
    LIST *folder_attribute_primary_list,
    char *date_where );

/* Process */
char *data_where =
    query_data_where(
        value_folder_name,
        where_attribute_name_list,
        where_attribute_data_list,
        folder_attribute_primary_list );

char *process_generic_where_clause(
        data_where,
        date_where );
```

PROCESS_GENERIC_VALUE_FOLDER

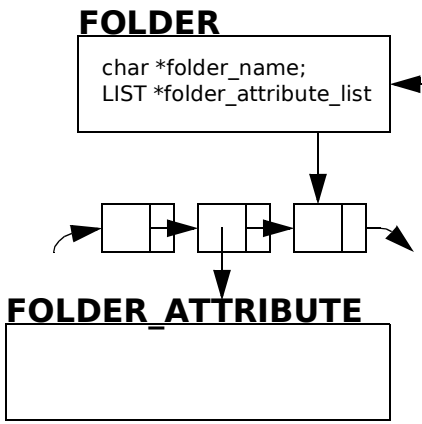
```
/* Usage */
PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        process_generic->value_folder_name,
        process_generic->post_dictionary );

/* Input */
char *value_folder_name;
DICTIONARY *post_dictionary;

/* Process */
PROCESS_GENERIC_VALUE_FOLDER *value_folder_parse(
    input );

char *datatype_folder_name;
char *foreign_folder_name;
char *datatype_attribute_name;
char *date_attribute_name;
char *time_attribute_name;
char *value_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_unit;
boolean foreign_exists_unit;

LIST *folder_attribute_list =
    folder_attribute_fetch_list(
        value_folder_name,
        (LIST *)0 /* exclude_attribute_name_list */ );

PROCESS_GENERIC_DATATYPE *datatype =
    process_generic_datatype_fetch(
        datatype_folder_name,
        datatype_attribute_name,
        exists_aggregation_sum,
        exists_bar_graph,
        exists_scale_graph_zero,
        datatype_exists_units,
        post_dictionary );

PROCESS_GENERIC_FOREIGN_FOLDER *foreign_folder =
    process_generic_foreign_folder_fetch(
        foreign_folder_name,
        foreign_exists_unit,
        post_dictionary );

LIST *process_generic_datatype_compare_list(
    process_generic->post_dictionary,
    exists_aggregation_sum,
    exists_bar_graph,
    exists_scale_graph_zero,
    datatype_exists_unit );
```



char *

PROCESS_GENERIC_DATATYPE

PROCESS_GENERIC_FOREIGN_FOLDER



PROCESS_GENERIC_FOREIGN_FOLDER

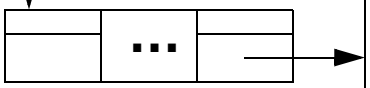
```
/* Usage */
PROCESS_GENERIC_FOREIGN_FOLDER *process_generic_foreign_folder_fetch(
    process_generic->value_folder->foreign_folder_name,
    process_generic->post_dictionary,
    process_generic->value_folder->foreign_exists_unit );

/* Process */
FOLDER *folder =
    folder_fetch(
        foreign_folder_name,
            /* Not fetching role_folder_list */
            (char *)0 /* role_name,
            (LIST *)0 /* exclude_attribute_name_list */,
            /* Also sets folder_attribute_primary_list */
            /* and primary_key_list. */
            1 /* fetch_folder_attribute_list */,
            0 /* not fetch_relation_mto1_non_isa_list */,
            0 /* not fetch_relation_mto1_isa_list */,
            0 /* not fetch_relation_one2m_list */,
            0 /* not fetch_relation_one2m_recursive_list */,
            0 /* not fetch_process */,
            0 /* not fetch_role_folder_list */,
            0 /* not fetch_row_level_restriction */,
            0 /* not fetch_role_operation_list */ );

LIST *foreign_key_list = folder->primary_key_list;

LIST *foreign_attribute_data_list =
    dictionary_key_list_fetch(
        foreign_key_list,
        post_dictionary );

char *unit = (foreign_exists_unit) ? list_last( foreign_attribute_data_list ) : (char *)0;
```

PROCESS_GENERIC_DATATYPE

```
/* Usage */
PROCESS_GENERIC_DATATYPE *process_generic_datatype_fetch(
    process_generic_value_folder->datatype_folder_name,
    process_generic_value_folder->datatype_attribute_name,
    process_generic_value_folder->exists_aggregation_sum,
    process_generic_value_folder->exists_bar_graph,
    process_generic_value_folder->exists_scale_graph_zero,
    process_generic_value_folder->datatype_exists_units,
    process_generic_value_folder->post_dictionary );

/* Input */
char *datatype_folder_name;
char *datatype_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_units;
DICTIONARY *post_dictionary;

/* Process */
char *datatype_name;
boolean aggregation_sum;
boolean bar_graph;
boolean scale_graph_zero;
char *unit;
```

```
char *process_generic_datatype_select(
    datatype_attribute_name,
    exists_aggregation_sum,
    exists_bar_graph,
    exists_scale_graph_zero,
    datatype_exists_unit );

char *datatype_name =
    dictionary_get(
        datatype_attribute_name,
        post_dictionary );

char *process_generic_datatype_where(
    datatype_attribute_name,
    datatype_name );

char *process_generic_datatype_system_string(
    process_generic_datatype_select(),
    process_generic_datatype->datatype_folder_name,
    process_generic_datatype_where() );

PROCESS_GENERIC_DATATYPE *process_generic_datatype_parse(
    string_pipe_fetch(
        process_generic_datatype_system_string() )
            /* input_buffer */ );
```

key = date or
key = date^time



PROCESS_GENERIC_VALUE

```
char *date;
char *time;
char *unit;
double value;
boolean is_null;
double accumulate
```

```
PROCESS_GENERIC_VALUE *process_generic_value_parse(
    double *accumulate_value,
    char *input,
    char *time_attribute,
    boolean unit_value_folder,
    boolean accumulate_boolean,
    enum aggregate_level );
```

DICTIONARY_SEPARATE (1)

```
/* Usage */
DICTIONARY_SEPARATE *dictionary_separate_folder_new(
    DICTIONARY *post_dictionary->original_post_dictionary,
    char *application_name,
    char *login_name,
    LIST *folder_attribute_date_name_list() );

DICTIONARY_SEPARATE *dictionary_separate_row_new(
    DICTIONARY *post_dictionary_row_new()->row_dictionary
        /* original_row_dictionary */,
    char *application_name,
    char *login_name,
    LIST *folder_attribute_name_list(
        folder->folder_attribute_list ),
    LIST *folder_attribute_date_name_list(
        folder->folder_attribute_list ),
    LIST *operation_name_list );

DICTIONARY_SEPARATE *dictionary_separate_string_new(
    DICTIONARY *post_dictionary_string_new()->dictionary
        /* original_post_dictionary */ );

/* Process */
DICTIONARY *dictionary_separate_working_post(
    DICTIONARY *original_post_dictionary,
    char *application_name,
    LIST *folder_attribute_date_name_list,
    char *login_name );

DICTIONARY *dictionary_separate_sort(
    char *DICTIONARY_SEPARATE_SORT_PREFIX,
    DICTIONARY *dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_query(
    char *DICTIONARY_SEPARATE_QUERY_PREFIX,
    DICTIONARY *dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_drillthru(
    char *DICTIONARY_SEPARATE_DRILLTHRU_PREFIX,
    DICTIONARY *dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_ignore(
    char *DICTIONARY_SEPARATE_IGNORE_PREFIX,
    DICTIONARY *dictionary_separate_working_post() );

LIST *dictionary_separate_ignore_select_attribute_name_list(
    DICTIONARY *dictionary_separate_ignore() );

DICTIONARY *dictionary_separate_pair_one2m(
    char *DICTIONARY_SEPARATE_PAIR_ONE2M_PREFIX,
    DICTIONARY *dictionary_separate_working_post() );

DICTIONARY *dictionary_separate_non_prefixed(
    DICTIONARY *dictionary_separate_working_post() );

void dictionary_separate_date_convert(
    DICTIONARY *working_post_dictionary,
    char *application_name,
    char *login_name,
    LIST *folder_attribute_date_name_list );
```

DICTIONARY_SEPARATE (2)

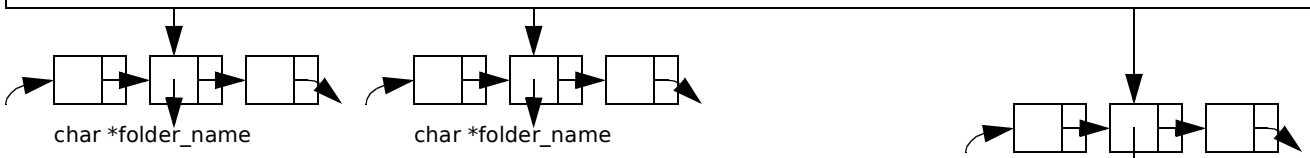
```
/* Public */
void dictionary_separate_string_date_convert(
    DICTIONARY *dictionary,
    char *application_namem,
    char *date_string,
    char *key,
    char *login_name );

DICTIONARY *dictionary_separate_row_dictionary_row_zero(
    DICTIONARY *working_post_dictionary,
    LIST *attribute_name_list );

DICTIONARY *dictionary_separate_row_dictionary_multi_row(
    dictionary_separate_non_prefixed(),
    folder_attribute_name_list,
    operation_name_list );

void dictionary_separate_parse_multi_attribute_keys(
    DICTIONARY *working_post_dictionary,
    char *prefix );

void dictionary_separate_trim_multi_drop_down_index(
    DICTIONARY *working_post_dictionary,
    char multi_select_move_left_right_index_delimiter );

void dictionary_separate_trim_double_bracked_data(
    DICTIONARY *working_post_dictionary );

DICTIONARY *dictionary_separate_send_dictionary(
    DICTIONARY *sort_dictionary,
    DICTIONARY *query_dictionary,
    DICTIONARY *drillthru_dictionary,
    DICTIONARY *ignore_dictionary,
    DICTIONARY *pair_one2m_dictionary,
    DICTIONARY *non_prefixed_dictionary );

char *dictionary_separate_send_string(
    dictionary_separate_send_dictionary() );

char *dictionary_separate_hidden_html(
    char *prefix,
    DICTIONARY *dictionary );

/* Private */
DICTIONARY *dictionary_separate_prefixed(
    DICTIONARY *working_post_dictionary,
    char *prefix );
```

FOLDER_MENU

```
/* Usage */
FOLDER_MENU *folder_menu_fetch(
    char *application_name,
    char *session_key,
    char *appaserver_data_directory,
    char *role_name );

/* Input */
char *application_name;
char *session_key;
char *appaserver_data_directory;
char *role_name;
```

```
/* Process */
ROLE *role =
    role_fetch(
        role_name,
        0 /* not fetch_role_attribute_exclude_list */ );

LIST *folder_menu_lookup_folder_name_list(
    char *role_name );

LIST *folder_menu_insert_folder_name_list(
    char *role_name );

if ( role->folder_count )
```

```
{
    char *folder_menu_filename(
        char *application_name,
        char *session_key,
        char *appaserver_data_directory,
        char *role_name );

    LIST *folder_menu_lookup_count_list(
        application_name,
        folder_menu_lookup_folder_name_list(),
        folder_menu_filename() );
}
```



FOLDER_MENU_COUNT

```
/* Usage */
LIST *folder_menu_lookup_count_list(
    char *application_name,
    folder_menu_lookup_folder_name_list(),
    folder_menu_filename() );

FOLDER_MENU_COUNT *folder_menu_count_new(
        char *application_name,
        char *folder_name );

/* Attributes */
```

```
char *folder_name;
unsigned long count;
char *display;

/* Process */
unsigned long folder_menu_count_fetch(
    char *folder_table_name() );

char *folder_menu_count_display(
    folder_menu_count_fetch() );
```

```
/* Public */
void folder_menu_write_count_list(
    char *filename,
    folder_menu_lookup_count_list() );

LIST *folder_menu_read_count_list(
    char *filename );

FOLDER_MENU_COUNT *folder_menu_count_seek(
    char *folder_name,
    LIST *folder_menu_lookup_count_list() );
```

APPLICATION

```
/* Usage */
APPLICATION *application_fetch(
    char *application_name );

/* Attributes */
char *application_name;
char *application_title;
char *user_date_format;
char *relative_source_directory;
int next_session_number;
int next_reference_number;
char *background_color;
char *distill_directory;
char *ghost_script_directory;
char *grace_home_directory;
char *grace_execution_directory;
boolean grace_free_option;
char *grace_output;
boolean frameset_menu_horizontal;
boolean ssl_support;
boolean prepend_http_protocol;
char *chart_email_command_line;
int max_query_rows_for_drop_downs;
int max_drop_down_size;

/* Process */
char *application_primary_where(
    char *application_name );

char *application_system_string(
    char *APPLICATION_SELECT,
    char *application_primary_where() );

APPLICATION *application_parse(
    char *input );

/* Private */
APPLICATION *application_new(
    char *application_name );

APPLICATION *application_calloc( void );

/* Public */
char *application_table_name(
    char *folder_name );

char *application_title_string(
    char *application_name );

char *application_title(
    char *application_name );

char *application_relative_source_directory(
    char *application_name );

char *application_background_color(
    char *application_name );

char *application_distill_directory(
    char *application_name );

char *application_ghost_script_directory(
    char *application_name );

char *application_grace_home_directory(
    char *application_name );

char *application_grace_execution_directory(
    char *application_name );

boolean application_grace_free_option(
    char *application_name );

char *application_grace_output(
    char *application_name );

boolean application_frameset_menu_horizontal(
    char *application_name );

boolean application_ssl_support(
    char *application_name );

char application_ssl_support_yn(
    char *application_name );

boolean application_prepend_http_protocol(
    char *application_name );

char *application_prepend_http_protocol_yn(
    char *application_name );

char *application_chart_email_command_line(
    char *application_name );

int application_max_query_rows_for_drop_downs(
    char *application_name );

int application_max_drop_down_size(
    char *application_name );

char *application_first_relative_source_directory(
    char *application_name );

char *application_http_prefix(
    char *application_name );

void application_reset( void );
```

SESSION

```
/* Usage */
/* If error then exit( 1 ) */
SESSION *session_fetch(
    char *application_name,
    char *session_key,
    char *login_name );

/* Attributes */
char *application_name;
char *session_key;
char *login_name;
char *login_date;
char *login_time;
char *last_access_date;
char *last_access_time;
char *http_user_agent;
char *remote_ip_address;

/* Process */
void session_environment_set(
    char *application_name );

char *session_primary_where(
    char *session_key );

char *session_system_string(
    char *session_primary_where() );

SESSION *session_parse(
    char *input );

if ( !session_parse()
||  strcmp( session->login_name, login_name ) != 0 )
{
    void session_access_failed_message_exit(
        application_name,
        login_name,
        session_current_ip_address() );
}

char *session_current_ip_address(
    char *environment_remote_ip_address() );

boolean session_remote_ip_address_changed(
    char *session->remote_ip_address,
    char *session_currrent_ip_address() );

if ( session_remote_ip_address_changed() )
{
    void session_message_ip_address_changed_exit(
        char *application_name,
        char *session_key,
        char *login_name,
        char *session->remote_ip_address,
        char *session_current_ip_address() );
}

void session_update_access_date_time(
    char *session_key );

void session_purge_temporary_files(
    char *application_name );

/* Private */
SESSION *session_calloc( void );

/* Public */
void session_sql_injection_message_exit(
    char *argv_0,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *session_current_ip_address() );

boolean session_sql_injection_strcmp_okay(
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name );

void session_access_failed_message_exit(
    char *application_name,
    char *login_name,
    char *session_current_ip_address() );

char *session_login_name(
    char *session_key );

LIST *session_system_list(
    char *session_system_string() );

LIST *session_list_fetch(
    char *login_name );

char *session_key(
    char *application_name );

void session_insert(
    char *session_key(),
    char *login_name,
    char *date_now_yyyy_mm_dd_string(),
    char *date_now_hhmm_string(),
    char *left_string(
        destination,
        environment_http_user_agent(),
        80 ),
    char *environment_remote_ip_address() );
```

SESSION_FOLDER

```
/* Usage */
/* Sets appaserver environment and outputs argv. */
/* Each parameter is security inspected. */
/* Any error will exit( 1 ). */
SESSION_FOLDER *session_folder_integrity_exit(
    int argc,
    char **argv,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *folder_name,
    char *state );

/* Process */
if ( !session_sql_injection_strcmp_okay(
        application_name,
        session_key,
        login_name,
        role_name ) )
{
    session_sql_injection_message_exit(
        argv[ 0 ],
        application_name,
        session_key,
        login_name,
        role_name,
        session_current_ip_address() );
}

appaserver_error_argv_append_file(
    argc, argv, application_name );

SESSION *session =
    /* If error then exit( 1 ) */
    session_fetch(
        application_name,
        session_key,
        login_name );

if ( string_strcmp(
        security_sql_injection_escape(
            folder_name ),
        folder_name ) != 0 )
{
    exit( 1 );
}

if ( string_strcmp(
        security_sql_injection_escape(
            state ),
        state ) != 0 )
{
    exit( 1 );
}

boolean session_folder_valid(
    char *state,
    LIST *role_folder_list(
        role_name,
        folder_name ) );

if ( !session_folder_valid() )
{
    session_access_failed_message_exit(
        application_name,
        login_name,
        session_current_ip_address() );
}

/* Private */
SESSION_FOLDER *session_folder_calloc( void );
```

SESSION_PROCESS

```
/* Usage */
/* Sets appaserver environment and outputs argv. */
/* Each parameter is security inspected. */
/* Any error will exit( 1 ). */
SESSION_PROCESS *session_process_integrity_exit(
    int argc,
    char **argv,
    char *application_name,
    char *session_key,
    char *login_name,
    char *role_name,
    char *process_name );

/* Process */
if ( !session_sql_injection_strcmp_okay(
        application_name,
        session_key,
        login_name,
        role_name ) )
{
    session_sql_injection_message_exit(
        argv[ 0 ],
        application_name,
        session_key,
        login_name,
        role_name,
        session_current_ip_address() );
}

appaserver_error_argv_append_file(
    argc, argv, application_name );

SESSION *session =
    /* If error then exit( 1 ) */
    session_fetch(
        application_name,
        session_key,
        login_name );

if ( string_strcmp(
        security_sql_injection_escape(
            process_name ),
        process_name ) != 0 )
{
    exit( 1 );
}

boolean session_process_valid(
    char *process_name,
    LIST *role_process_list( role_name ) );

if ( !session_process_valid() )
{
    session_access_failed_message_exit(
        application_name,
        login_name,
        session_current_ip_address() );
}

/* Private */
SESSION_PROCESS *session_process_calloc( void );
```

POST_LOGIN

```
/* Usage */
POST_LOGIN *post_login_new(
   int argc,
   char **argv );

/* Process */
DICTIONARY *post_login_dictionary( void );

char *sql_injection_escape_application_name =
   security_sql_injection_escape(
      string_low(
         post_login_application_name(
            int argc,
            char **argv,
            post_login_dictionary() ) ) );

char *sql_injection_escape_login_name =
   security_sql_injection_escape(
      string_low(
         dictionary_get(
            “login_name”,
            post_login_dictionary() ) );

boolean post_login_missing_name(
      char *sql_injection_escape_login_name );

char *sql_injection_escape_password =
   security_sql_injection_escape(
      dictionary_get(
         “password”,
         post_login_dictionary() );

boolean post_login_public_name(
   char *sql_injection_escape_login_name );

char *post_login_database_password(
   char *sql_injection_escape_login_name );

boolean post_login_missing_database_password(
   char *sql_injection_escape_password );

boolean post_login_name_email_address(
   sql_injection_esacpe_login_name );

enum password_match_return
   post_login_password_match(
      post_login_missing_name(),
      post_login_missing_database_password(),
      post_login_name_email_address(),
      post_login_public_name(),
      sql_injection_escape_password,
      post_login_database_password() );

if ( post_login_password_match() in
   {  email_login,
      database_password_blank,
      password_fail,
      missing_name } )
{
   char *post_login_ip_address( void );

   char *post_login_location(
      char *POST_LOGIN_CLOUDACUS_LOCATION,
      post_login_ip_address() );

   char *post_login_message(
      post_login_password_match() );

   char *post_login_redraw_index_screen_string(
      sql_injection_escape_application_name,
      post_login_location(),
      post_login_message(),
      appaserver_library_from_php(
         post_login_dictionary() ) );
}

if ( post_login_password_match() in
   {  password_match,
      public_login,
      email_login } )
{
   char *session_key(
      sql_injection_escape_application_name );

   session_insert(
      session_key(),
      sql_injection_escape_login_name,
      date_now_yyyy_mm_dd_string(),
      date_now_hhmm_string(),
      left_string(
         destination,
         environment_http_user_agent(),
         80 ),
      environment_remote_ip_address() );

   char *post_login_output_pipe_string(
      char *POST_LOGIN_EMAIL_OUTPUT_TEMPLATE,
      boolean post_login_name_email_address(),
      char *sql_injection_escape_application_name,
      char *appaserver_parameter_document_root(),
      char *session_key() );

   char *frameset_output_system_string(
      FRAMESET_OUTPUT_EXECUTABLE,
      session_key(),
      sql_injection_escape_login_name,
      post_login_output_pipe_string(),
      appaserver_error_filename() );
}

if ( post_login_password_match() == email_login )
{
   char *post_login_email_link_system_string(
      char *POST_LOGIN_EMAIL_HTTP_TEMPLATE,
      char *POST_LOGIN_RETURN_ADDRESS,
      char *POST_LOGIN_SUBJECT,
      char *sql_injection_escape_application_name,
      char *session_key(),
      char *sql_injection_escape_login_name );
}

/* Private */
POST_LOGIN *post_login_calloc( void );
```

enum password_match_return

```
{ password_match,          public_login,
  database_password_blank, email_login,
  password_fail,           missing_name }
```

APPASERVER Data Structure

APPASERVER_LINK

```
/* Usage */
APPASERVER_LINK *appaserver_link_new(
    char *application_http_prefix(),
    char *appaserver_library_server_address(),
    boolean (application_prepend_http_protocol_yn() == ‘y’),
    char *document_root_directory,
    char *filename_stem,
    char *application_name,
    pid_t process_id,
    char *session_key,
    char *begin_date_string,
    char *end_date_string,
    char *extension );

/* Input */
char *http_prefix;
char *server_address;
boolean prepend_http;
char *document_root_directory;
char *filename_stem;
char *application_name;
pid_t process_id;
char *session_key;
char *begin_date_string;
char *end_date_string;
char *extension;

/* Process */
APPASERVER_LINK_PROMPT *
    appaserver_link_prompt_new(
        http_prefix,
        sever_address,
        prepend_http );

char *tail_half =
    appaserver_link_output_tail_half(
        application_name,
        filename_stem,
        begin_date_string,
        end_date_string,
        process_id,
        session_key,
        extension );

APPASERVER_LINK_OUTPUT *appaserver_link_output_new(
    document_root_directory,
    tail_half );

/* Public */
char *appaserver_link_working_directory(
    char *document_root_directory,
    char *application_name );

void appaserver_link_pid_filename(
    char **prompt_filename,
    char **output_filename,
    char *application_name,
    char *document_root_directory,
    pid_t process_id,
    char *process_name,
    char *extension );
```

APPASERVER_LINK_PROMPT

APPASERVER_LINK_OUTPUT

APPASERVER_LINK_PROMPT

```
/* Usage */
APPASERVER_LINK_PROMPT *appaserver_link_prompt_new(
    char *http_prefix,
    char *server_address,
    boolean prepend_http,
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );

/* Process */
char *appaserver_link_prompt_link_half(
    boolean prepend_http,
    char *http_prefix,
    char *server_address );

char *appaserver_link_prompt_filename(
    appaserver_link_prompt_link_half(),
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );
```

APPASERVER_LINK_OUTPUT

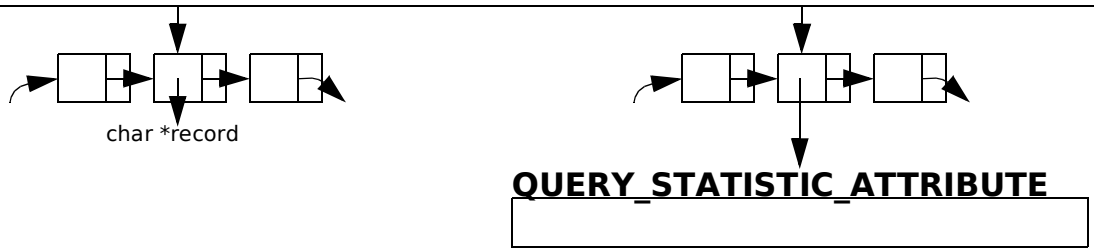
```
/* Usage */
APPASERVER_LINK_OUTPUT *
    appaserver_link_output_new(
        char *document_root_directory,
        char *appaserver_link_output_tail_half() );

/* Process */
char *appaserver_link_output_filename(
    char *document_root_directory,
    appaserver_link_output_tail_half() );

/* Public */
char *appaserver_link_output_tail_half(
    char *application_name,
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );

char *appaserver_link_output_abbreviated_filename(
    char *filename_stem,
    char *begin_date_string,
    char *end_date_string,
    pid_t process_id,
    char *session_key,
    char *extension );
```

POPULATE_HELPER_PROCESS

```
/* Usage */
POPULATE_HELPER_PROCESS *
    populate_helper_process_new(
        char *
            process_parameter->
                populate_helper_process_name ||
            process_set_parameter->
                populate_helper_process_name,
        char *document_root_directory,
        char *application_name,
        pid_t getpid() );

/* Input */
char *name;
char *document_root_directory;
char *application_name;
pid_t process_id;

/* Process */
char *populate_helper_process_output_filename(
    char *name,
    char *document_root_directory,
    char *application_name,
    pid_t process_id );

char *populate_helper_process_prompt(
    char *name,
    char *application_name,
    pid_t process_id );
```

QUERY_STATISTIC

```
/* Usage */
QUERY_STATISTIC *query_statistic_new(
    LIST *folder_attribute_name_list(
        folder_attribute_number_list(
            query->folder->folder_attribute_list ) ),
    LIST *folder_attribute_name_list(
        folder_attribute_date_list(
            query->folder->folder_attribute_list ) ),
    LIST *folder_attribute_name_list(
        folder_attribute_date_time_list(
            query->folder->folder_attribute_list ) ),
    char *query_from_string(),
    char *query_where_string() );

/* Process */
LIST *query_statistic_record_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    char *from_string,
    char *where_string );

LIST *query_statistic_attribute_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    LIST *query_statistic_record_list() );

char *query_statistic_html(
    char *query_from_string(),
    char *query_where_string(),
    query_statistic_attribute_list() );

/* Private */
char *query_statistic_title_html(
    char *from_string,
    char *where_string );

char *query_statistic_heading_html( void );

char *query_statistic_select(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list );
```



QUERY_STATISTIC_ATTRIBUTE

```
/* Usage */
LIST *query_statistic_attribute_list(
    LIST *folder_attribute_number_name_list,
    LIST *folder_attribute_date_name_list,
    LIST *folder_attribute_date_time_name_list,
    LIST *query_statistic_record_list );

QUERY_STATISTIC_ATTRIBUTE *
    query_statistic_attribute_new(
        char *folder_attribute_name,
        boolean is_date,
        int piece_offset,
        LIST *query_statistic_record_list );

/* Process */
QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate =
        query_statistic_aggregate_new(
            is_date,
            piece_offset,
            query_statistic_record_list );

char *query_statistic_attribute_html(
    char *folder_attribute_name,
    boolean is_date,
    QUERY_STATISTIC_AGGREGATE *
        query_statistic_aggregate );
```

QUERY_STATISTIC_AGGREGATE

```
/* Usage */
QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate_new(
        boolean is_date,
        int piece_offset,
        LIST *record_list );

/* Attributes */
unsigned long count;
double sum;
double average;
double minimum;
double maximum;
double range;
double standard_deviation;
double coefficient_of_variation;
double standard_error;
double percent_missing;
char *begin_date;
char *end_date;

/* Process */
char *aggregate_filename =
    timlib_temp_filename();

FILE *output_pipe =
    query_statistic_aggregate_output_pipe(
        boolean is_date,
        char *aggregate_filename );

void query_statistic_aggregate_generate(
        FILE *output_pipe,
        int piece_offset,
        LIST *record_list );

pclose( output_pipe );

QUERY_STATISTIC_AGGREGATE *
    query_statistic_aggregate_read(
        boolean is_date,
        char *aggregate_filename );

timlib_remove_file( aggregate_filename );
```